



单位代码: 10635

西南大學

本科毕业论文（设计）

基于微服务架构的在线销售平台设计与开发

学 院: 计算机与信息科学学院 软件学院

专 业: 软 件 工 程

姓 名: 杨 英 杰

学 号: 222020321062018

指导老师: 刘 波

答辩日期: 2024 年 5 月 18 日

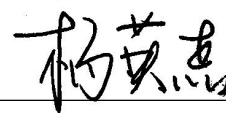
2024 年 5 月

独 创 声 明

本人郑重声明：所提交的毕业论文（设计），是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议。尽我所知，除文中已经注明引用的内容外，本论文（设计）不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体均已在文中以明确方式标明。

声明的法律后果由本人承担。

作者签名：



2024 年 5 月 23 日

毕业论文（设计）使用授权声明

本人完全了解西南大学关于收集、保存、使用毕业论文（设计）的规定。

本人愿意按照学校要求提交论文（设计）的印刷本和电子版，同意学校保存论文（设计）的印刷本和电子版，或采用影印、数字化或其它复制手段保存论文（设计）；同意学校在不以营利为目的的前提下，建立目录检索与阅览服务系统，公布论文（设计）的部分或全部内容，允许他人依法合理使用。

（保密论文在解密后遵守此规定）

作者签名：



2024 年 5 月 23 日

目 录

摘 要	1
Abstract	2
第 1 章 导论	4
1.1 背景	4
1.1.1 销售平台的现状	4
1.1.2 微服务架构在在线销售中的重要性	4
1.1.3 人工智能的现状	5
1.2 开发目标与挑战	5
1.2.1 开发目标	5
1.2.2 面临挑战	7
1.3 论文组织结构	7
第 2 章 相关理论与技术	9
2.1 相关理论介绍	9
2.1.1 B2B2C	9
2.1.2 UML 应用	9
2.1.3 用例模型	9
2.1.4 结构模型	10
2.1.5 行为模型	11
2.1.6 DevOps 和 CICD	12
2.1.7 微服务架构	13
2.1.8 分布式事务	13
2.1.9 SSO 与 OAuth2	13
2.2 相关技术介绍	15
2.2.1 Spring Cloud Alibaba	15
2.2.2 Redis 和 MySQL	16
2.2.3 Docker	16
2.2.4 Kubernetes	17

2.2.5 Seata	17
2.3 本章小结	17
第3章 系统需求分析	18
3.1 用例模型的构建	18
3.1.1 需求的识别与分析	18
3.1.2 候选用例集识别与构造	19
3.1.3 用例分析与抽取	20
3.1.4 高层文本用例开发	21
3.1.5 对话式详细文本用例开发	24
3.2 结构模型的构建	28
3.2.1 初始类图的识别和分析	29
3.2.2 概念类图的绘制	30
3.3 行为模型的构建	32
3.3.1 系统操作识别与用例序列图绘制	32
3.3.2 操作契约定义	37
3.4 本章小结	39
第4章 系统设计	40
4.1 设计原则	40
4.2 功能架构设计	40
4.3 对象序列图设计	41
4.5 数据库设计	45
4.6 本章小结	49
第5章 系统实现	50
5.1 工具与环境	50
5.2 技术架构	50
5.3 核心功能实现	52
5.3.1 i18n 国际化	52
5.3.2 用户登录	52
5.3.3 用户注册	56

5.3.4 用户找回密码	56
5.3.5 首页三级分类	57
5.3.6 浏览商品	58
5.3.7 搜索商品	62
5.3.8 管理购物车	63
5.3.9 订购商品	64
5.3.10 在线咨询	68
5.3.11 个人中心管理	69
5.4 数据库操作实现	71
5.5 挑战问题解决	73
5.6 本章小结	74
第 6 章 系统部署	75
6.1 部署方式	75
6.2 流水线配置	75
6.3 本章小结	76
第 7 章 系统测试	77
7.2 功能测试	77
7.3 非功能测试	81
7.3.1 性能测试	81
7.3.2 安全性测试	85
7.3.3 容错性测试	86
7.3.4 兼容性测试	87
7.4 本章小结	87
第 8 章 总结与展望	88
8.1 结论	88
8.2 展望	89
参考文献	91
致 谢	93

基于微服务架构的在线销售平台设计与开发

杨英杰

西南大学计算机与信息科学学院软件学院，重庆 400715

摘 要：现代在线销售平台的敏捷构建与持续演进成为其在瞬息万变的市场竞争中取得优势的迫切需求。但随着用户需求的日趋复杂、系统规模的持续增长、智能化构件的不断介入，传统单体软件体系架构下的在线销售平台开发与运维均面临着巨大挑战。微服务架构通过将系统构造为一系列规模适中、良构定义、边界清晰的新型软件构件（即微服务），在系统构建、持续开发与运维、故障隔离方面展现出巨大的优势，已成为工业界事实上的体系架构标准；并有望成为智能化时代构建超大规模在线销售平台的绝佳解决方案（winning strategy）。

鉴于此，本课题探索研究并开发了融合 Spring Cloud Alibaba 微服务架构与 Alibaba 通义千问为一体的智慧化微服务体系架构在线销售平台。本平台构建过程应用了工业界最新的基于模型软件工程方法（Model-based Software Engineering, MBSE），针对数据管理、身份验证、自动部署等六大挑战，围绕着需求规约、服务拆分、系统设计等六大任务，开展了研究和开发工作。主要贡献如下：

（1）为严格定义软件需求规约，构建了用例模型、结构模型和行为模型；累计开发了 19 个用例以刻画系统的功能性需求，并开发了各用例的概念类模型与交互模型（用例序列图）；同时针对系统行为，构建了基于霍尔三元组（Hoare Triple）的系统操作契约，对系统行为模型进行了精化。

（2）为正确构造系统体系架构，在操作契约后置条件基础上，应用通用职责分配设计模式（General Responsibility Assignment Software Patterns, GRASP），构造了对象交互模型（即对象序列图）；并基于概念类模型进一步导出了系统的设计类模型，为后续系统实现中的代码逻辑编写和数据库实现指引了方向。

（3）在实现层面，使用了 Spring Cloud Alibaba 作为微服务开发框架，应用了 Nacos 做配置下发和微服务注册与发现。结合 OpenFeign, Zipkin, Spring Cloud Gateway, Sentinel, Seata 等组件分别实现微服务的远程调用、链路监控、网关路由控制、流量控制和分布式事务的管理。基于 Docker + Kubernetes 进行项目的部署，通过 Alibaba Codeup 进行 DevOps，部署流水线实现自动化部署。

（4）在测试方面，使用了 JMeter 测试重放工具和 Apifox 对该平台进行了充分的系统性测试，内容涵盖其功能和非功能两个方面。相关测试结果表明，相比于单体架构系统，微服务架构系统在

实际具有更好的功能正确性和更好的并发特性及高可用性等性能；有望推广到真实的工业应用中。

关键词：微服务架构；基于模型软件工程；通用职责分配设计模式；在线销售平台；智能软件工程

Design and Development of Online Sales Platform based on Micro-Service Architecture

YANG Yingjie

School of Computer and Information Science, School of Software, Southwest University, Chongqing 400715, PR China

Abstract: The agile construction and continuous evolution of modern online sales platforms has become an urgent need to gain an advantage in the ever-changing market competition. However, with the increasing complexity of user requirements, the continuous growth of system scale, and the continuous intervention of intelligent components, the development and operation and maintenance of online sales platforms under the traditional monolithic software architecture are facing great challenges. Micro-service architecture, by constructing the system into a series of new software components (i.e., micro-services) with moderate scale, good architectural definition, and clear boundaries, shows great advantages in system construction, continuous development and operation and maintenance, and fault isolation, and has become the architectural standard in the industrial world; and it is expected to be an excellent solution for constructing a mega-scaled online sales platform in the era of intelligentization (Winning Strategy). It is also expected to be an excellent solution for building a mega-scale online sales platform (winning strategy) in the era of intelligence.

In view of this, this project explores and develops an intelligent micro-service architecture online sales platform integrating Spring Cloud Alibaba micro-service architecture and Alibaba Tongyi Large Model. The platform construction process applies the latest Model-based Software Engineering (MBSE) methodology in the industry, and the research and development work has been carried out to address the six major challenges, such as data management, authentication, and automatic deployment, and centers around six major tasks, such as requirement specification, service disassembly, and system design. The main contributions are as follows:

(1) In order to rigorously define the software requirement statute, the use case model, structural model, and behavioral model were constructed; 19 use cases were cumulatively developed to portray the functional requirements of the system, and the conceptual class model and interaction model (use case sequence diagram) of each use case were developed; meanwhile, for the system behavior, the system operation contract based on the Hoare Triple was constructed, and the system behavioral model was The system behavior model is refined.

(2) In order to correctly construct the system architecture, the object interaction model (i.e., object sequence diagram) is constructed by applying the General Responsibility Assignment Software Patterns (GRASP) on the basis of the post-conditions of the operation contract; and the design class model of the system is further derived based on the conceptual class model, which provides a good basis for the subsequent system architecture. The design class model of the system is further derived based on the conceptual class model, which guides the direction of code logic writing and database implementation in the subsequent system realization.

(3) At the implementation level, Spring Cloud Alibaba is used as the micro-service development framework, and Nacos is applied to do configuration issuance and micro-service registration and discovery. Combined with OpenFeign, Zipkin, Spring Cloud Gateway, Sentinel, Seata and other components to realize remote invocation of micro-services, link monitoring, gateway routing control, traffic control and distributed transaction management respectively. Based on Docker + Kubernetes for the deployment of the project, through Alibaba Codeup for DevOps, deployment pipeline to achieve automated deployment.

(4) In terms of testing, JMeter test replay tool and Apifox were used to conduct sufficient systematic testing of the platform, covering both its functional and non-functional aspects. The relevant test results show that compared with the monolithic architecture system, the micro-service architecture system has better functional correctness and better concurrency characteristics and high availability and other performances in practice; it is expected to be promoted to real industrial applications.

Key words: Micro-services; Model-based Software Engineering; Assignment of Responsibilities

Design Pattern; Online Retail Platform; Intelligent Software Engineering

第 1 章 导论

1.1 背景

1.1.1 销售平台的现状

现时代，京东、淘宝、当当和拼多多等在线销售平台系统拥有着庞大的用户群体和广泛的品牌合作伙伴，而如盒马、远方好物等在线销售平台也如雨后春笋般不断冒尖。随着人们生活质量、消费能力的提高和网上支付的完善，在线购物正融入人们生活的方方面面。

我国第 52 次《中国互联网发展状况统计报告》显示，到 2023 年 6 月为止，我国在线购物的用户规模已达 8.84 亿人，较 2022 年 12 月增长 3880 万人，占网民整体的 82%^[29]。同时，据国家统计局相关数据显示，截止至 2023 年 12 月，网上零售额已达 154264 亿元，比 2022 年全年增长 11%^[4]。



图 1.1 2011—2023 年全国网上零售额

Fig.1.1 National e-commerce transactions from 2011-2023

从相关数据不难看出，我国人民对在线购物的需求是日益递增的。消费者对便捷、高效、优质的在线购物需求随着互联网的发展愈来愈多。

1.1.2 微服务架构在在线销售中的重要性

随着在线销售平台的不断完善，需求功能正在不断增多。对于传统单体架构的挑战愈来愈大，如随着项目功能模块增多，可维护性逐渐下降^[33]；项目逐渐臃肿，每次添加修改都需要重新编译乃至更新全部生产环境的现有项目，部署时间增加，伸缩性较差；

功能之间耦合严重等^[6]。

为了解决这些问题，引入微服务架构是非常重要且必需的。微服务架构相比于单体架构而言，具有更低的响应时间，更大的吞吐量，更优的容错机制和更少的部署时间^[1]。因此在在线销售的庞大流量和海量并发中，引入微服务架构是必须的。在历年 618 和双 11 中，新时代的微服务架构的在线销售平台也不负众望地支撑起了海量用户，做到了高并发、高可用、高性能^[2]。

1.1.3 人工智能的现状

人工智能在在线销售中的应用层面十分广阔，简要概述为用户、商家、平台和仓储四个部分。提供包括个性化推荐、智能客服、智能化定价、智能化采销、智能仓库等功能^[17]。人工智能在在线销售领域的应用整合了海量的数据，构建了更加贴合使用者偏好的用户模型，在降低运营成本的同时，提高了用户体验。

而随着 ChatGPT 在 2023 年的出现，人工智能技术将迎来井喷，更人性化的智能客服，更高的人工劳动力和创造力也随之而来，人工智能的应用将迎来新一波的增长^[30]。

1.2 开发目标与挑战

1.2.1 开发目标

本论文的目标是设计实现一个基于微服务架构的在线销售平台，通过使用最新的微服务相关技术，以解决单体架构项目在大量需求迭代中产生的问题，达到高可用、高并发、高性能的效果，同时保证该在线销售平台的基本业务功能的实现。

通过微服务架构的基本设计要求，本论文将该在线销售平台的主要功能模块拆分为以下 8 个独立服务（商品、订单、搜索、仓储、支付、优惠券、用户、权限），通过 OpenFeign 组件完成他们之间的相互调用。同时对于外部请求而言，使用 ELB 做物理负载均衡器，再结合 Spring Gateway 做统一 API 网关，请求经过统一网关后再逐步转发到对应微服务。

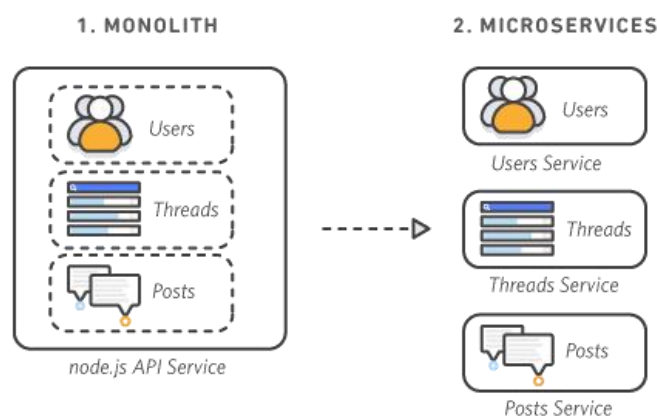


图 1.2 微服务划分

Fig.1.2 Micro-services Partition

为了实现开发目标，需要完成如下工作：

1. 系统需求分析：本系统属于 B2B2C 模式，需要同时面向顾客、商家和平台三端，因此主要通过这三个方面进行系统的需求分析，通过构建用例、数据和行为模型，充分了解在线销售系统的业务需求以及他们之间的关联关系，从而进行合适的服务模块拆分。
2. 系统服务拆分：依照系统的相关需求，针对系统服务进行拆分，使得每个服务之间完全独立，松耦合。本论文依照上述系统相关需求，将核心服务分为以下 8 个（商品、订单、搜索、仓储、支付、优惠券、用户、权限）。
3. 系统架构设计：前端采用 Vue、Vue Router、Element Plus、Pinia、Axios、ESLint、Prettier 分别完成前端技术开发、路由管理、页面样式设计、数据持久化管理、Restful 请求管理、代码规范管理、代码格式化；后端采用使用主流微服务开发框架 Spring Cloud Alibaba，通过 Nacos、OpenFeign、Zipkin、Spring Cloud Gateway、Sentinel、Seata 分别完成服务注册发现与统一配置管理、服务远程调用、服务链路监控、统一网关路由、流量控制与降级保护、分布式事务管理。
4. 业务模块实现：使用 Visual Studio Code 和 IntelliJ IDEA 分别对前端和后端进行开发，前后端分离，通过 Axios 进行通信，数据使用 Json 传输。
5. 业务自动部署：本论文采用阿里云 Codeup 实现 DevOps，通过设置分支合并钩

子完整代码自动化部署。前端采用 keepalive + Nginx 集群实现高可用，后端采用 Kubernetes 进行部署，同时结合 ELB 实现物理方面的负载均衡。此外，采用 Redis 集群实现高可用，采用 MySQL 一主一从，实现读写分离。

6. 系统可用测试：结合 JMeter 和 Apifox，对系统进行功能和非功能测试。

1.2.2 面临挑战

本论文设计开发中将会面临如下挑战：

1. 系统设计挑战：将软件按功能划分为多个独立的服务，确保它们可以协同工作，以及设计他们的服务内通信和分布式事务的相关处理是将会面临的第一个挑战。
2. 数据管理挑战：在微服务架构中，每个服务由于拥有不同的数据库，可能会出现数据不一致和不同步的问题，如何进行数据管理，是将会面临的第二个挑战。
3. 网络延迟挑战：由于服务进入分布式设计，网络延迟很有可能存在性能瓶颈，如何设计合理的超时机制、重试逻辑、报错提示以处理服务故障是将会面临的第三个挑战。
4. 身份验证挑战：由于微服务将服务进行拆分，每个微服务如何验证用户身份就成了一个新的问题。每个服务都对用户进行验证将会极大程度浪费资源，而不对用户进行身份验证，数据得不到合理的保护，因此建立一个合理的身份验证机制是第四个挑战。
5. 配置部署挑战：随着服务数量的提升，管理和部署这些服务将会变得越来越复杂，如何合理地管理配置和部署流程，设置自动化部署流程和编排工具汇是第五个挑战。
6. 可观测和监控：跟踪和监控问题在微服务中将会比单体应用更为复杂和困难，如何结合监控组件完成微服务系统的监控将会是第六个挑战。

1.3 论文组织结构

本文各部分对应章节及其大致内容如下：

第一章，导论，介绍销售平台现状、微服务架构在其中的应用、人工智能的现状与

应用、开发的目标和将会面临的挑战。

第二章，理论技术，介绍系统所用理论和技术。

第三章，需求分析，详细描述了系统的功能、体系结构以及操作流程，并通过用例模型、结构模型和行为模型进行细致建模，主要借助 UML 图示进行阐释。

第四章，系统设计，深入探讨系统的进一步功能流程设计和数据库结构设计。

第五章，系统实现，介绍开发工具与环境、前后端技术架构、前后端交互接口实现、核心代码实现和架构软件部署实现（Redis 集群、MySQL 集群等）。

第六章，系统部署，介绍自动化部署方案。

第七章，系统测试，介绍功能和非功能测试的过程和结果。

第八章，总结和展望。

第 2 章 相关理论与技术

2.1 相关理论介绍

2.1.1 B2B2C

在传统的商业模式中，我们常见的是 B2C 模式，即公司（Business）的产品在官网或者自营平台销售给消费者（Consumer）。而 B2B2C 模式，则是在 B2C 模式的基础上进行了延伸，新增了第三方平台，例如：淘宝、京东等。最终形成商家（Business）、平台（Business）和消费者（Consumer）的局面，即公司通过第三方平台出售自己的商品给消费者^[20]。B2B2C 模式旨在优化原有的 B2C 模式，使 B 端和 C 端之间的交易更便捷、科学和流畅。随着时间的推移，这种模式逐渐演变为平台化服务，使 B 端和 C 端都成为平台的用户，从而形成一个新的产业链条——B2B2C。

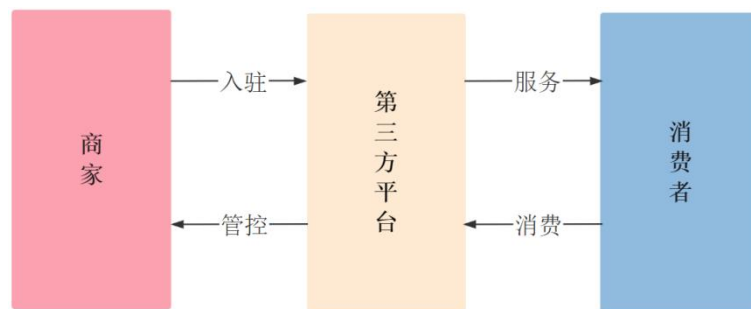


图 2.1 B2B2C 关系

Fig.2.1 B2B2C Relationship

2.1.2 UML 应用

UML 是一种面向对象的建模语言。它并不是一套面向对象的建模方法，因此并没有描述如何进行系统建模，而是给出了一套用于建模的元素和表示符号，并定义了他们的语义^[15]。目前，UML 已成为面向对象建模语言中的最主流语言^[24]。本论文将结合基于模型软件工程方法建模的标准流程^[32]，完成系统需求规约的分析与系统设计。

2.1.3 用例模型

用例模型是基于模型软件工程方法建模中需求分析阶段的三大模型之一的核心模型。用例是系统的业务功能单元，从定义来说，所谓用例就是特定参与者为达成其具体的业务目标，通过与系统交互，所执行的一系列完整的活动（动作）序列^[10]。每个用例

描述了单个或多个场景，展示了系统是如何与最终用户或其他系统进行互动的。这些场景明确了用户或系统可以通过系统实现什么功能，从而分析出具体的业务目标。

其识别要点有如下三点：第一，序列是否完整（能讲出从开始到过程发展到结束的全过程）；第二，是否具有业务价值（谁使用系统的目的是什么，即进行老板测试）；第三，场景是否周全（具有业务执行成功和执行失败的场景）。如果满足这三个识别要点，那么就可以认为该候选用例可以作为用例。同时，对于某一子业务级别的候选用例被多个用例所使用的情況，即使它不能够通过“老板测试”，也可将其单独作为用例。

用例图是由参与者、用例、边界以及它们之间的关系组成的视图，用于描述系统的功能。它展示了系统或子系统的功能行为。其主要包括的四种关系分别是使用关系、继承关系、包含关系和拓展关系，简单示例如下：

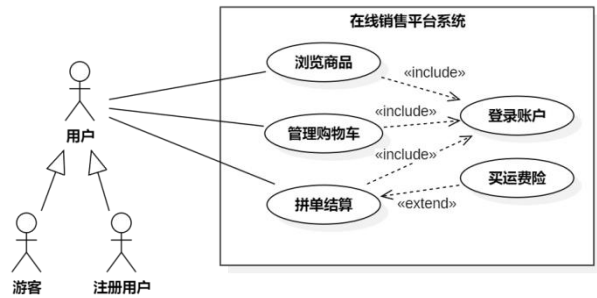


图 2.2 用例图中的四种关系

Fig.2.2 Four Types of Relationships in Use Case Diagram

2.1.4 结构模型

结构模型作为基于模型软件工程方法建模中需求分析阶段的第二部分，描述了系统的概念类、属性以及这些概念类之间的相互关系。通过概念类图，结构模型展现了系统的静态结构。它描述了系统中数据的结构和关系，一般包括实体、属性和关系三个要素，并且可以用来帮助开发人员在设计系统时明确数据的概念和关系。在系统设计部分，结构模型的概念类图也是重要的基础元素，结合所得到的概念类图和用例，可以得到对应的对象序列图和设计类图，进而实现数据的存储、查询、分析和管理工作[27]。

结构模型的建模主要聚焦在以下四个过程：筛选概念和属性、识别关系种类、定义关系性质和定义关系重数。

在筛选概念和属性部分，主要从此挑选依据进行概念类挑选的考量：一切需要被软件系统管理、控制和修改的对象类别。如果满足此依据，那么就可以认为此对象能够作为一个概念类。在此基础上，再根据属性的定义进行属性的识别：属性的选取意味什么，

属性的类型意味什么，属性的取值范围意味什么。通过这两组依据，就能够识别出初始的概念类，一个用例的概念类组合图，称之为初始类图。

在识别关系种类部分，主要从以下四种关系进行考量，分别是：一般、继承/泛化关系、聚合和组合。在其中，基本的依赖性是通过一般关系来描绘的，这种关系存在于各类之间。而继承或泛化关系则着重于展现子类与父类之间的紧密连接。同时，聚合与组合关系同时描绘整体和部分的联系，区别在于整体与部分的生命周期是否同步，以及部分是否能属于多个整体。为了简化表示，我们采用以下符号：在聚合和组合关系中，用菱形符号指向整体，而小箭头则用来指示部分（这个部分可以省略）。

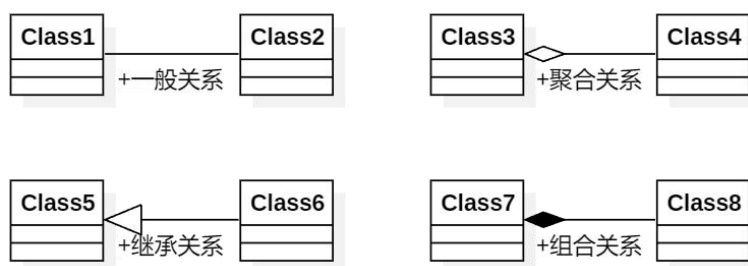


图 2.3 类图中的四种关系

Fig.2.3 Four Types of Relationships in Class Diagram

在定义关系的性质部分，主要聚焦在关系的度、角色、命名和方向。所谓关系的度即产生关系时参与的类的个数，如一元关系和二元关系等。

在定义关系的重数部分，毋庸置疑重点在于重数。一个类与另外一个类产生关系的时候，约束一个类与另外一个类产生的关系时的各自实例的数量就是重数。

2.1.5 行为模型

行为模型是基于模型软件工程方法建模中需求分析阶段的最后一个核心模型。它描述了系统的动态行为，包含对用例所涉及的参与者与系统之间交互过程的刻画。该模型通过序列图和操作契约展示了系统内是如何进行交互的。主要用于帮助开发人员在设计系统时明确各个组成部分之间的交互关系和行为规则，进而实现系统的设计、开发、测试和维护。此部分主要产出物为用例序列图和操作契约。

所谓的用例序列图就是将系统当作一个黑盒，通过抽取详细文本用例中蕴含的相关信息，从而展现出来的系统提供的服务的图。而系统操作契约主要定义系统操作执行前后导致系统状态的变化：对象被创建或销毁、对象属性被更新、对象之间的连接关系被建立或销毁。具体而言，开发系统操作契约的核心工作是构造霍尔三元组（Hoare Triple）

中的 Pre-condition（前置条件）和 Post-condition（后置条件），其表示如下：

{Per - Condition} m() {Post - Condition}

其中 m()代表的是系统操作，Pre-condition（前置条件）定义系统状态在系统操作在执行之前必须满足的假设条件。例如：哪些对象在执行之前必须存在；哪些对象的某些属性的取值应该满足什么要求；哪些对象之间存在着连接(link)Post-condition(后置条件)定义系统操作完成后，当前系统发生状态变化后所应呈现的结果。这些状态变化后的结果包括：哪些对象被创建或销毁；哪些对象属性被更新；哪些对象之间的连接关系被建立或销毁。

这样形容可能太过于抽象，下面我以简单的登录交互过程为示例，不考虑实际生产环境中的复杂校验和权限认证，一个简单的用例序列图示例如下。

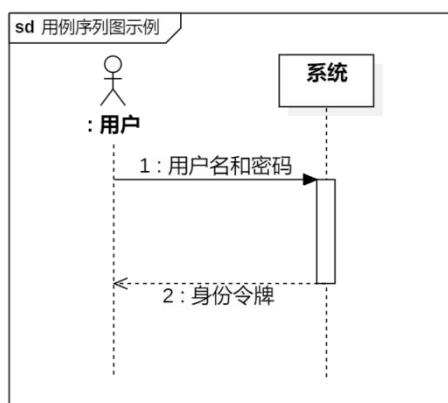


图 2.4 用例序列图示例

Fig.2.4 Use Case Sequence Diagram

2.1.6 DevOps 和 CICD

DevOps 强调开发和运维人员之间的紧密沟通和合作，它是一种文化、运动或实践。它通过自动化软件交付和架构变更的过程，实现了软件构建、测试和发布的加速、频繁性和可靠性^[13]。



图 2.5 DevOps 生命周期

Fig.2.5 DevOps Lifecycle

CI 的全称为 Continuous Integration, CD 的全称为 Continuous Deployment, 表示持续交付。企业一旦引入 CICD, 就可以将其部分或者全部自动化, 而且加快交付流程, 降低沟通成本以及错误成本^[12]。

2.1.7 微服务架构

微服务架构是根据应用系统的业务需求, 分服务划分原有架构体系, 形成的一个企业级应用分布式体系结构^[34]。与单体架构不同的是, 功能和数据被按照模块进行分割, 使得每个服务都拥有自己的数据库, 并可以独立部署运行。每个服务之间不需要考虑技术的统一, 不受编程语言和版本的限制。

其优势在于:

1. 功能独立, 易于开发和维护;
2. 灵活性高, 不限制语言和开发平台;
3. 子服务可独立部署, 可持续集成和交付;
4. 容错性高, 单个服务不会影响其他服务;
5. 易于拓展。

目前, 微服务体系结构已被很多公司应用在自己的大体量系统中^[23]。

2.1.8 分布式事务

在分布式系统中, 由于一个业务操作需要多个微服务共同参与, 为了确保数据的一致性, 每个服务的事务都必须同时成功或失败。这一次完整的业务操作就叫分布式事务, 每个服务的事务就是一次分支事务, 整个业务就称之为全局事务。

2.1.9 SSO 与 OAuth2

Single Sign-on (SSO) 就是只要进行一次验证, 就可以在多系统上进行登录, 是随着软件系统数量的增多和企业信息化的不断深入而随之提出的一种概念。常见的单点登录实现方式之一是访问票据^[11]。

Open Authorization 1 是一种安全协议, 该框架能够帮助用户在不提供访问网站或应用的登录信息的情况下, 通过第三方应用进行授权, 达到登录目标网站的目的^[7]。例如: 在使用拼多多应用的时候, 可以通过第三方微信进行授权登录, 使用微信的登录接口完成拼多多应用的登录操作, 从而达到微信用户访问拼多多资源的目的。使用该协议可以

方便用户通过单一应用授权达到登录大多数应用的目的，这无疑方便了用户使用不同系统^[9]。

Open Authorization 2（OAuth2.0）授权协议则是 OAuth1.0 后的完善版本。尽管 OAuth1.0a 修复了 OAuth1.0 中存在的安全漏洞——会话固化攻击，在安全性上已经没有问题了，但其单一的授权流程和复杂的签名过程都太过于简陋，因此 OAuth2.0 诞生了。OAuth2.0 所提供的新特性主要有：授权流程多样性、认证方式无加密以及 Access Token 时效化^[16]。

授权流程多样性是指除原本单一的授权码模式（Authorization Code Grant）外，新增了隐式授权模式（Implicit Grant），资源所有者密码授权模式（Resource Owner Password Credentials Grant）以及客户端授权模式（Client Credentials Grant）^[22]。目前主流且最安全的是授权码模式，因此这里对其他模式不做讲解。所谓授权码模式就是通过认证获取到一块访问令牌，可以通过此访问令牌访问受保护的资源。该模式的时序图如下所示：

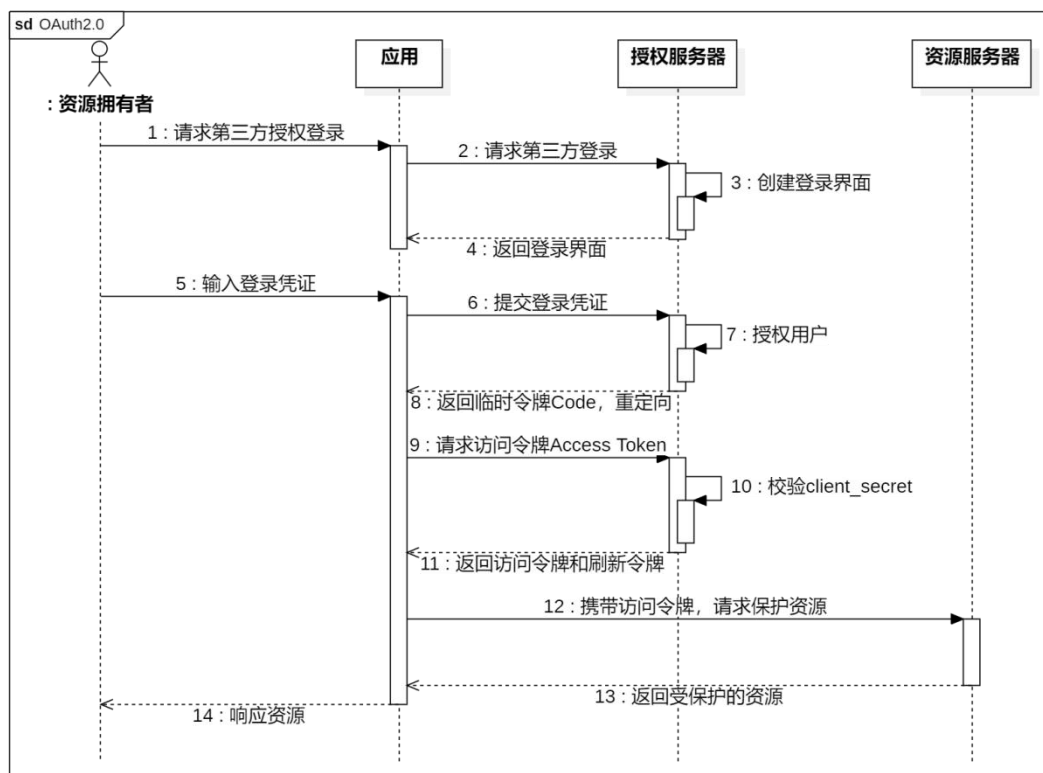


图 2.6 OAuth2.0 授权码模式时序图

Fig.2.6 Authorization Code Grant Sequence Diagram

认证方式无加密是指去掉了 OAuth1.0 中每次 token 传输的签名加密，改用 HTTPS 传输来保证数据的安全性，因此 OAuth2.0 也完全不兼容 OAuth1.0。而 Access Token 时效化是指将 OAuth1.0 中长期有效的 Token 设置了短期过期时间，并使用 Refresh Token

作为凭证来刷新 Access Token。

本文将结合 SSO 和 OAuth2.0, 使用 Spring Cloud Gateway 作统一权限集中管理实现包含第三方授权的单点登录。因部分主流应用未开放个人开发者 OAuth2.0 授权功能（微信、支付宝等），本文只实现了 QQ、Github、Google 的第三方登录功能。同时，还提供主流的账户密码、手机号验证码登录功能。

2.2 相关技术介绍

2.2.1 Spring Cloud Alibaba

Spring Cloud 是 Spring 官方推出的为了解决分布式微服务架构问题的一站式体系方案，通过 Spring Boot 脚手架，它使得微服务系统的构建变得简单易用。Spring Cloud 是一套微服务规范和构建标准。

Spring Cloud Alibaba 则是阿里巴巴推出的第二代实现的关键组成部分，它继承了 Spring Cloud Netflix 的核心架构思想，并进行了性能优化。随着 Spring Cloud Netflix 停止更新和维护，Spring Cloud Alibaba 成为主流框架。

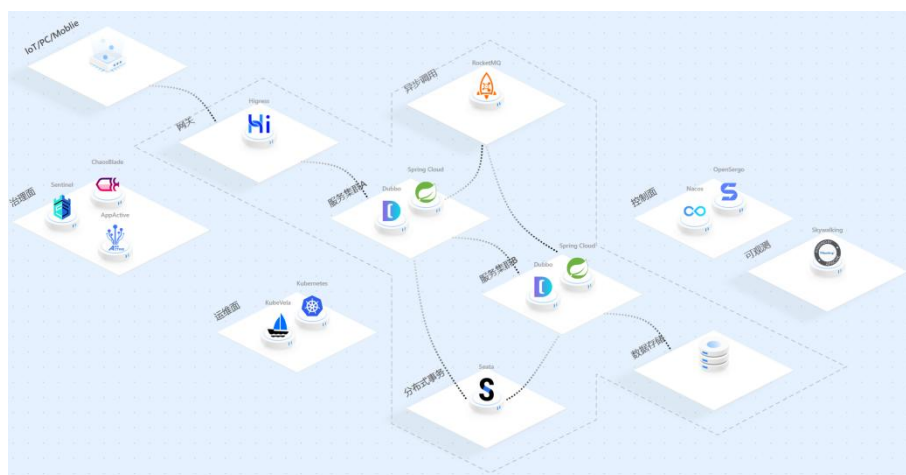


图 2.7 Spring Cloud 全景图

Fig.2.7 Spring Cloud Panorama

本次使用的相关微服务技术如下：

1. 配置中心：Nacos
2. 注册与发现：Nacos
3. 远程调用：OpenFeign
4. 链路监控：Zipkin
5. 统一网关路由：Spring Cloud Gateway

6. 流控、降级、保护：Sentinel
7. 异步调用：RocketMQ
8. 分布式事务：Seata

2.2.2 Redis 和 MySQL

Redis（Remote Dictionary Server）是开源的非关系型数据库，它基于内存的数据存储方式，将被更新的数据异步持久化到硬盘中。虽然 Redis 是非关系型数据库，但是它仍然可以存储 List、Sets 类型的数据，甚至还能完成排序、自增等操作，并保证了操作的原子性^[31]。

MySQL 是方兴未艾的关系型数据库管理系统之一，由于它玲珑小巧、省繁从简、开源免费，大多数网站都采用该系统。部分企业基于 MySQL 自行二次开发升级。它支持各种存储引擎，用户可以自行设置以获取更高的性能^[8]。

2.2.3 Docker

Docker 容器打包了一个应用程序及其所有依赖，使得该应用程序能在不同的设备环境中轻松、快速且可靠地切换。Docker 容器的运行是将软件和环境隔离开的，和每个软件运行在单独的虚拟机上类似，但又有所不同。区别在于，Docker 容器共用系统操作系统，因此占用空间更小、启动速度更快、消耗资源更少。

其容器具备以下特性：

1. 标准化：标准统一，能够在不同环境中移植。
2. 轻量级：共享操作系统内核，无需独立操作系统，减少了资源消耗。
3. 安全性：提供强大的隔离功能，运行的应用程序具有最高级别的安全性。

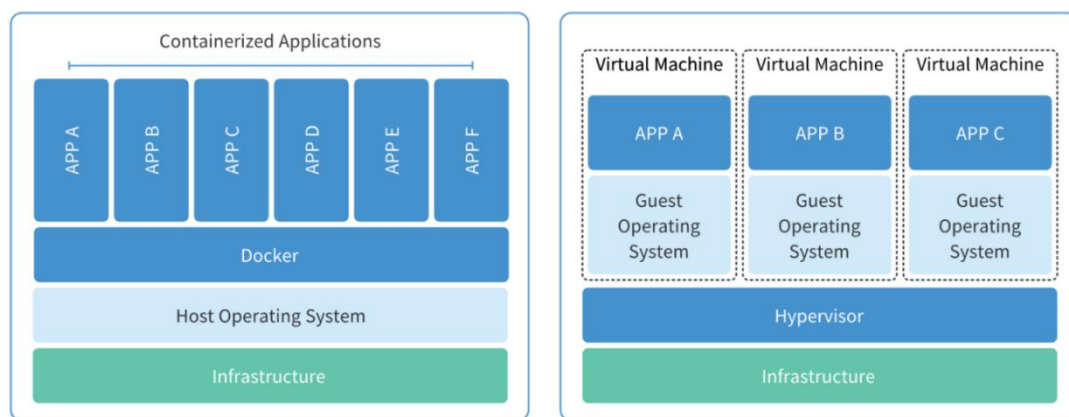


图 2.8 容器和虚拟机对比

Fig.2.8 Comparing Containers and Virtual Machines

2.2.4 Kubernetes

Kubernetes，常被称为 K8s，是 Google 结合 15 年生产环境的运维经验和开源社区的创意实践推出的一个开源系统^[19]。K8s 方便管理和服务发现，支持容器化应用程序的自动部署、伸缩和管理。

2.2.5 Seata

Seata 是一款开源的分布式事务解决方案。在开源之前，Seata 在阿里系内部作为应用架构层的数据一致性中间件，成功支撑了历年双 11 的经济体稳定运行，并为上层业务提供了强大的技术支持。经过多年的积累和发展，Seata 的商业化产品已经在阿里云和金融云上成功销售。

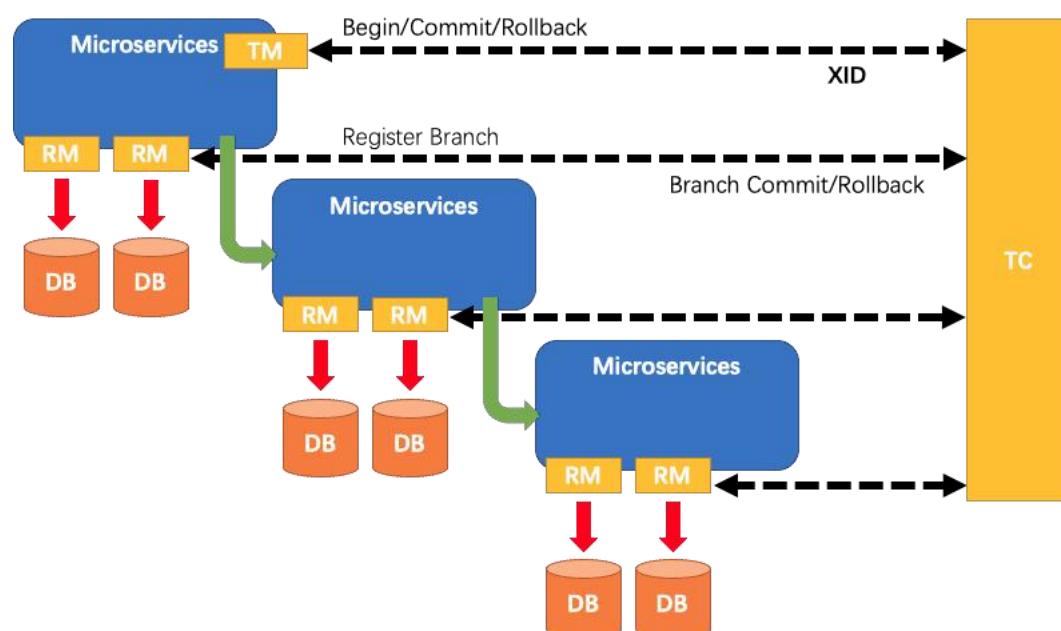


图 2.9 Seata 分布式事务生命周期
Fig.2.9 Seata Distributed Transaction Lifecycle

2.3 本章小结

本章主要介绍了在线销售平台开发过程中所涉及的相关理论和技术。首先介绍了销售平台中涉及的 B2B2C 概念以及使用的系统分析设计方法。然后针对该方法需求分析阶段中的相关模型进行了介绍。最后对实现过程中所涉及的核心微服务相关理论技术进行了介绍。

第3章 系统需求分析

3.1 用例模型的构建

为了找出在线销售平台的功能点，要先进行用例模型的构建。因此要先对该平台的应用例进行识别，从识别到的候选用例集中找到真正合适的应用例^[28]。并基于这些用例构建在线销售平台的应用例图。最终得到所有的功能点设计。

3.1.1 需求的识别与分析

此销售平台系统的业务需求如下，顾客角度的业务场景为：顾客打算通过此在线销售平台进行新手机的购买，因此顾客打开了该在线销售平台，通过搜索框进行搜索或通过分类进行搜索，进入手机搜索的搜索结果页面。顾客想要挑选指定处理器骁龙 8+的手机，因此通过筛选操作筛选出处理器为骁龙 8+的手机商品，选择感兴趣的手机商品点击进入详细页面，选择合适的销售属性（颜色、内存等）后加入购物车或直接下单。下单完成后进行支付，支付成功后商品开始运输。收货后进行商品评价。

商家角度的业务场景为：商家通过在线销售平台商家端系统进行商品的发布、下架以及库存设置。商家查看顾客的下单结果进行商品的发货与管理。商家管理自己的店铺。

平台角度的业务场景为：平台通过后台管理系统进行商家的管理、发布商品的审核和提供平台客服服务。

基于上述业务场景，可以得到在使用该在线销售平台所需要的大致的业务需求，同时结合实际的用户需求，总结得到了具体业务的需求功能：

表 3.1 业务需求
Table 3.1 Business Requirement

编号	需求功能		
	商家侧	顾客侧	平台侧
1	个人信息管理	个人信息管理	管理平台的商家
2	管理售卖的商品	搜索想购的商品	管理销售的商品
3	管理自己的店铺	管理购物车	提供咨询服务
4	管理售卖的订单	管理收货地址	
5	与顾客聊天	管理支付方式	
6	咨询平台客服	管理购买的订单	
7		管理个人的评价	
8		管理会员	

9	管理个人优惠券
10	管理收藏商品/店铺
11	管理浏览记录
12	查看推荐的商品
13	咨询商家/平台客服

3.1.2 候选用例集识别与构造

在上述业务需求中，尽可能找出了全部潜在用例，分析主要参与者与目标系统之间所发生的业务交互活动，并构造候选用例集。把相应的交互活动都填入到“参与者—业务目标”表的“业务目标”列，作为目标系统的候选用例。由于这些用例并不一定属于真正的用例，在识别之前都属于潜在用例，因此在这些潜在用例后用（？）进行标注。最终发现如下候选用例集：

表 3.2 参与者与业务目标

Table 3.2 Participants and Business Objectives

编号	业务目标 - 具体用户（角色）		
	小黄（商家）	小明（顾客）	小蓝（平台管理员）
1	登录账户？	登录账户？	登录账户？
2	输入账户密码？	输入账户密码？	输入账户密码？
3	注册账户？	注册账户？	注册账户？
4	管理账户信息？	管理账户信息？	管理账户信息？
5	管理商品？	浏览商品？	管理店铺？
6	管理店铺？	搜索商品？	管理商品？
7	管理订单？	管理购物车？	在线咨询？
8	在线咨询？	管理收货地址？	
9		登录账户？	
10		管理支付方式？	
11		订购商品？	
12		管理订单？	
13		管理评价？	
14		管理会员？	
15		管理优惠券？	
16		管理商品收藏？	
17		管理店铺关注？	
18		管理浏览记录？	
19		在线咨询？	
20		查看帮助手册？	

3.1.3 用例分析与抽取

对于业务需求来说，想要作为用例有两个要求，第一是需要满足老板测试，另一个则是满足以下三个条件的其中之一：

1. 创建或者删除对象，对象的属性发生变更；
2. 做了某些操作之后，导致对象之间的连接关系发生创建或者销毁；
3. 同时需要有至少两种的状态发生改变。

因此用例的识别判断标准，即满足老板测试且满足条件数量大于等于一。

对于用户小明而言，浏览商品、搜索商品、管理购物车、管理收货地址、管理支付方式、订购商品、管理订单和管理评价这八个候选用例的逻辑为：用户浏览推荐的商品、对想要的商品进行搜索，将它加入购物车中，并自主选择是否在购物车中修改它的数量或者规格参数，然后选择要购买的商品进行订购下单，填写个人地址和支付方式等信息后，点击结算完成商品订购，此时生成对应的订单，用户可以对订单和评价进行管理。显然这些用例都会引起多种状态发生变化，也可以通过老板测试，因此它们都可以作为用例。而对于管理账户信息、管理会员、管理优惠券、管理商品收藏、管理店铺关注和浏览记录这些候选用例而言，既可以满足对象连接关系发生改变，又可以满足至少两种状态发生变化，同时满足老板测试，因此他们也可以作为用例。而对于在线咨询来说，当用户打开新的在线咨询窗口的时候，客服和用户之间的连接被创建了，满足条件，也可以说用户使用该系统的目的是咨询获得想要的信息，因此也可以作为用例。而查看帮助手册并没有发生两种以上的状态变化，同时也不满足老板测试，因此查看帮助手册无法被识别成用例。

对于商家小黄和平台管理员小蓝而言，管理账户信息、管理商品、管理店铺、管理订单和在线咨询候选用例与用户面一致，满足条件且满足老板测试，因此可以作为用例存在。这里不单独赘述。

而对于登录账户而言，如果需要登录账户，会输入账户密码或选择其他登录方式，账户登录状态发生改变，并生成新的身份令牌，显然有两种及以上的状态发生改变。但是由于该用例属于子功能级别用例，不能说使用这个系统的目的是登录账户，因此登录账户这个用例并不能通过老板测试。但是由于其被多个用例所使用，因此尽管登录账户这个候选用例不能通过老板测试，仍然可以让其作为一个单独的用例存在。而对于注册账户来说，此候选用例与登录账户类似，尽管不满足老板测试，但是登录账户等功能并

不能绕开此用例，因此可以强制作为用户用例单独存在。同时，对于输入账户密码来说，由于该过程只存在一次状态变化（账户状态发生改变），因此输入账户密码不应该作为用例存在^[25]。

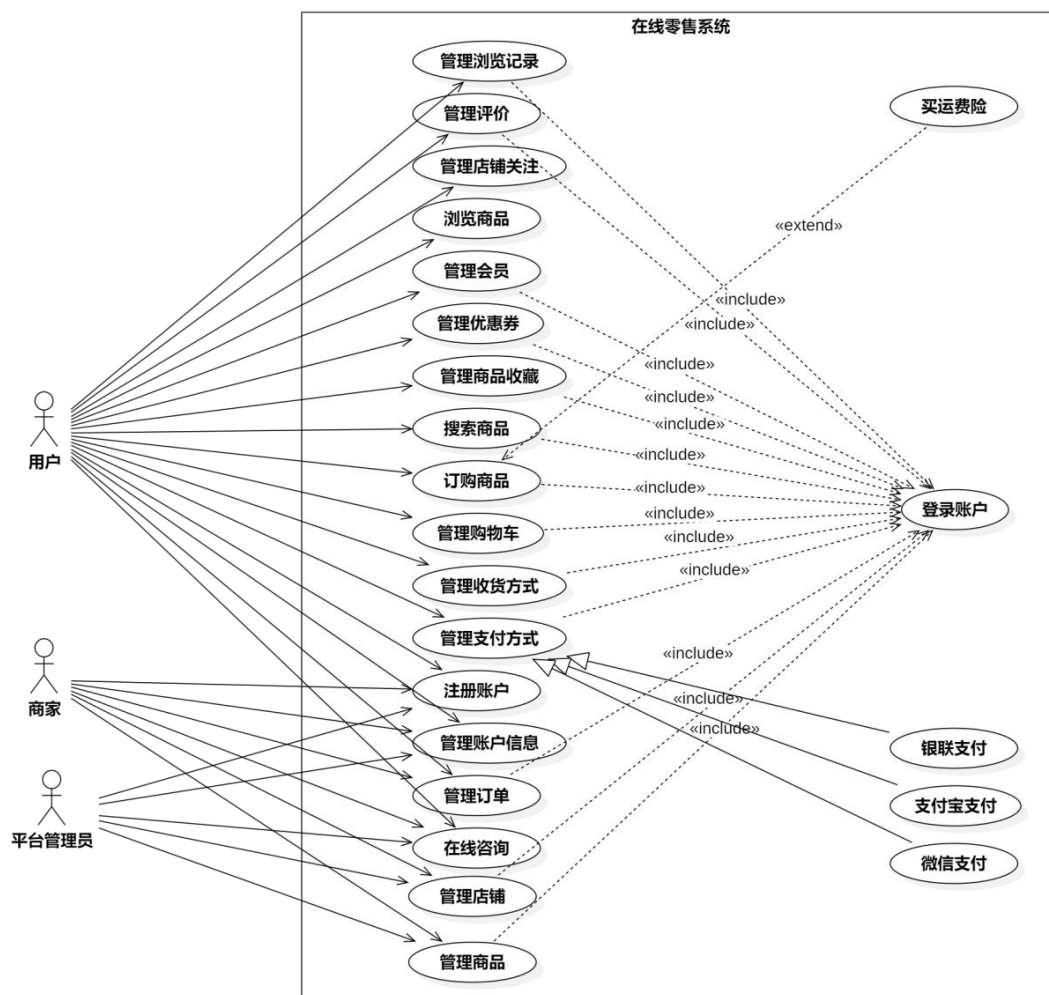


Fig.3.1 Online Retail System Use Case Diagram

基于上述用例图，撰写其高层文本用例描述，由于篇幅有限，这里详细描述核心功能的高层文本用例，对于管理类的用例，以管理购物车为典范，描述如下所示：

Table 3.3 Browse Products High-level Text Use Case

2. 业务目标	允许用户浏览系统中的商品信息，查看商品的详细信息
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客
5. 过程摘要	开始于：用户成功访问在线销售平台，并进入商品浏览界面 执行过程：用户通过滚动页面或者点击“下一页”等操作来查看系统展示的商品列表。继续查看系统展示的该商品的详细信息。选择查看该商品的评价信息。用户通过返回、收藏、添加到购物车等操作继续浏览其他商品信息。 结束于：用户浏览结束后退出商品浏览界面。

表 3.4 搜索商品高层文本用例
Table 3.4 Search Products High-level Text Use Case

主要元素	说明
1. 用例名称	搜索商品
2. 业务目标	允许用户通过输入关键词搜索商品，允许用户通过分类字段筛选搜索商品，允许用户进行连级筛选
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客
5. 过程摘要	开始于：用户成功访问在线销售平台。 执行过程：用户在搜索框选择店铺或者商品后，输入关键词进行搜索。系统根据关键词搜索匹配的商品信息并显示在搜索结果页。用户通过搜索结果页上的商品信息来进一步筛选结果或者点击查看商品详细。 结束于：用户搜索商品结束后进入商品详细页或退出搜索结果页。

表 3.5 管理购物车高层文本用例
Table 3.5 Manage Cart High-level Text Use Case

主要元素	说明
1. 用例名称	管理购物车
2. 业务目标	允许用户管理自己的购物车
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客
5. 过程摘要	开始于：用户暂时完成了商品浏览并需要对待购买商品进行管理。 执行过程：用户进入他的购物车并对相应内容进行管理。可管理内容，查看指定商品的详细信息，修改指定商品数量，移除指定商品，选定某几项商品进行结账。选定某几项商品进行总价查看。 结束于：用户完成操作，继续浏览或者开始支付对应的商品。

表 3.6 订购商品高层文本用例
Table 3.6 Book Products High-level Text Use Case

主要元素	说明
1. 用例名称	订购商品
2. 业务目标	引导用户填写提交准确的收货信息和支付方式，完成商品下单操作
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客
5. 过程摘要	开始于：用户提交商品订购请求。 执行过程：用户提交商品订购请求后，系统会构建支付表单，表单包括待购商品列表及价格，用户选择支付方式和收货地址。用户可针对挑选的商品进行备注操作，选择合适的优惠方式。 结束于：用户确认所有的信息准确无误后，点击支付按钮，支付成功完成商品订购。

表 3.7 在线咨询高层文本用例
Table 3.7 Online Consultation High-level Text Use Case

主要元素	说明
1. 用例名称	在线咨询
2. 业务目标	允许用户通过在线窗口进行店铺在线咨询和平台客服在线咨询
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客和商家
5. 过程摘要	开始于：用户或商家点开客服服务按钮进入在线咨询页 执行过程：用户或商家进入在线咨询页，对商家或平台发起咨询。人工智能客服将提供常见问题解答，同时也可以唤醒人工客服进行问题咨询。 结束于：用户或商家获取到结果后，对本次咨询进行评价。

表 3.8 登录账户高层文本用例
Table 3.8 Account Sign in High-level Text Use Case

主要元素	说明
1. 用例名称	登录账户
2. 业务目标	允许已经注册的用户登录到在线销售平台中
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客
5. 过程摘要	开始于：用户访问在线销售平台。 执行过程：用户在登录页面选择密码，验证码或第三方，进行登录。如果验证通过，系统将用户状态设置为已登录，并允许用户访问系统中的相关功能，如果不匹配，系统提示错误告警。用户在系统中

执行相关操作，比如浏览商品信息、添加商品到购物车、下单等。
结束于：用户完成登录，可以访问系统中的相关功能。

表 3.9 注册账户高层文本用例
Table 3.9 Account Sign up High-level Text Use Case

主要元素	说明
1. 用例名称	注册账户
2. 业务目标	允许用户在在线销售系统中创建一个新的账户
3. 用例级别	业务功能级别
4. 主要参与者	使用该在线销售平台的顾客
5. 过程摘要	开始于：用户访问在线销售平台。 执行过程：用户在注册页面输入个人信息。系统验证信息。用户通过激活邮件完成激活。 结束于：用户成功注册并能够登录系统。

3.1.5 对话式详细文本用例开发

基于上述高层文本用例描述，进一步开发出其对应的对话式详细文本用例，在实际系统开发过程中，应该先判断高层文本用例的优先级，再针对优先级高的用例先行开发详细文本用例：

表 3.10 浏览商品详细文本用例
Table 3.10 Browse Products Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户成功访问该平台。	
2. 用户进入浏览商品界面。	3. 系统展示商品列表，包括商品的名称、图片、简介等基本信息。
4. 用户通过滚动页面或者点击“下一页”等操作来查看更多的商品信息。	5. 系统判断是否已经浏览完所有商品。若已经浏览完所有商品则无法继续向下滚动或者点击“下一页”。
6. 用户点击具体的商品，进入该商品的详情页面。	7. 若用户选择的商品有具体详细信息，系统跳转到详情页面并展示。
8. 用户可以查看该商品的评价信息。	9. 如果该商品有评价信息，系统将会显示该商品的评价信息，包括评价内容、评价时间、评价用户等。
10. 用户通过返回、收藏、添加到购物车等操作继续浏览其他商品信息。	11. (1) 如果用户点击“返回”后再浏览其他商品信息，系统将会首先返回商品浏览页面，随后推荐新商品。 (2) 如果用户点击“收藏”后再浏览其

他商品，系统首先将该商品放入收藏商品的列表里，随后自动跳转至商品推荐界面。

- (3) 如果用户选择将商品添加到购物车。系统将该商品加入购物车。

12. 用例结束于用户浏览商品结束。

Alternative Courses:

Line 3: 无法正常展示商品列表，提示用户“该商品列表加载失败，请重试”。

Line 7: 无详细信息，提示用户“该商品无详细信息，请选择其他商品查看”。

表 3.11 搜索商品详细文本用例

Table 3.11 Search Products Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户成功访问该平台。	
2. 用户在主页面的搜索框中选择店铺搜索或关键词搜索，输入店铺名或者输入商品类型、名称等关键词，然后点击“搜索”。	3. 系统根据用户输入的关键词搜索匹配的商品并显示。对于搜索结果过多的情况，系统会通过精确的搜索算法将商品列表按照相关性和热度进行排序。
4. 用户通过筛选结果页上的商品信息（例如：品种、价格、评价等）来进一步细化搜索结果。	5. 系统根据商品信息进一步细化搜索，并显示搜索结果页。
6. 用例结束于用户搜索商品结束。	

Alternative Courses:

Line 3: 查询无返回结果，提示“未搜索到相关商品，请重新查询”。

表 3.12 管理购物车详细文本用例

Table 3.12 Manage Cart Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户暂时完成了商品浏览并需要对待购买商品进行管理。	
2. 用户查看指定商品的详细信息。	3. 系统接收到用户请求，跳转到该商品的具体详情页面。系统展示该商品的详细信息。
4. 用户修改指定商品数量。	5. 系统对用户指定的修改数量进行判定，如果库存量满足修改订单操作，则成功进行修改。
6. 用户移除指定商品。	7. 系统在购物车中移除用户指定移除的商品，删除该商品与用户购物车的联系。
8. 用户选定某几项商品进行总价查看。	9. 系统将用户选定的几件商品进行组合，

将组合数据发送到后端接口。后端将组合数据当作初始计算数据,同时计算该订单能够使用的优惠券和积分等优惠方式,将优惠后的最佳价格进行返回。用户可以查看到这几件商品的最优价格总量。

10. 用户选定某几项商品进行结账。
11. 系统将用户选定的几件商品进行组合,将组合数据发送到订购商品接口,进行后续的结账操作。
12. 用例结束于用户执行完所有的操作后退出购物车界面或者进行结账。

Alternative Courses:

Line 5: 库存不足,取消本次修改操作并提示“库存数量超过指定数量最大”。

Line 11: 存在商品可购买数量不足,取消结账操作并提示“商品已售罄,是否移除售罄商品?”。

表 3.13 订购商品详细文本用例

Table 3.13 Book Products Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户在购物车内或商品详情页选择结算。	
2. 用户选择收货地址和收件人姓名、手机,或自行填写新的收货地址等信息,核对无误后确认该地址。	3. 系统查看地址、姓名、手机填写是否无误,如果符合填写标准则返回订单确认界面,显示商品的详细购买信息。
4. 用户调整好商品数量、配送服务、运费险、开具发票、订单备注等信息。用户核对价格明细,选择优惠方式。	5. 系统将显示用户所拥有的所有优惠方式,供用户确认优惠方式之后回滚到价格明细界面。
6. 用户确认所有信息无误后,提交订单,进入支付。	7. 系统核查支付情况,支付成功后生成订单。
8. 用例结束于用户支付成功,订单成功生成。	

Alternative Courses:

Line 3: 信息有误,结算失败。

表 3.14 在线咨询详细文本用例

Table 3.14 Online Consultation Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户成功访问该平台。	
2. 用户或商家点击在线咨询按钮发起线上咨询。	3. 系统接收到用户请求,跳转到在线咨询界面,唤起人工智能服务。
4. 用户发起咨询,可选择快捷问题进行提	5. 系统智能根据用户提问进行回话。

问。

6. 用户需要人工服务，唤起人工服务。
7. 系统接入人工服务，回答用户问题。
8. 用户咨询结束，结束对话。
9. 系统确认咨询结束，弹出咨询评价给用户。
10. 用例结束于用户咨询结束，对本次咨询进行评价打分。

Alternative Courses:

Line 4: 数据库中不存在提出问题的答案，给出错误答复。

表 3.15 登录账户详细文本用例

Table 3.15 Account Sign in Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户访问该平台，并点击登录按钮。	
2. 用户进入登录界面。	3. 系统显示登录页面，用户可选密码、验证码、三方登录。
4. (1) 用户选择密码登录。通过输入用户名/邮箱/电话作为账户名，然后输入密码进行登录。	5. 系统验证用户的登录信息是否匹配，匹配成功，生成用户身份令牌，设置该令牌有效时间为 2h，每次请求接口都会重新刷新。设置该用户登录状态为已登录，并返回该令牌给用户。
(2) 用户选择验证码登录。接收验证码进行登录。	
(3) 用户选择第三方登录。点击第三方 logo，在第三方页面完成登录。	
6. 用例结束于用户本地存储身份 token。	

Alternative Courses:

Line 5: 输入登录信息不匹配，登录失败。

Line 8: 用户令牌过期，请重新登录。

表 3.16 注册账户详细文本用例

Table 3.16 Account Sign up Detail Text Use Case

参与者活动	系统响应
1. 用例开始于用户进入该平台，并点击注册按钮。	
2. 用户进入注册界面。	3. 系统显示注册页面，要求用户输入个人信息。
4. 用户输入个人信息，并点击确认按钮。	5. 系统验证输入的账号是否满足唯一性，同时验证密码是否属于强密码。系统生成新的账户，并将用户 ID、用户名、密码、邮箱、联系电话等信息保存到数据库中。生成一封账户激活邮件发给用

户。

6. 用户打开邮箱查看激活邮件。
7. 用户点击激活邮件中的链接完成激活。
8. 系统验证激活是否有效，如果有效，将用户的状态设置为已激活，用户注册成功。
9. 用例结束于获得自己的账户。

Alternative Courses:

Line 5: 输入账号密码不满足要求，无法注册需要重新输入符合规范的账户密码。

Line 8: 激活链接超时失效，用户需要重新申请系统发送激活邮件。

3.2 结构模型的构建

在构建了用例模型后，基于电商平台系统的用例图和作为案例的用例们，结合相关材料文档，以及在线购物软件系统的业务背景、工作环境以及知识术语体系，我从中发掘出以下名词，根据实体、场所和角色三种类别对名词进行简单分类，得到了如下概念发现表。

表 3.17 概念发现分类

Table 3.17 Concept Discovery Classification

角色	用例功能	实体	场所
用户	浏览商品	推荐商品项、商品基本信息	平台首页
	搜索商品	商品详细页信息、用户评价	平台首页 搜索结果页
	管理购物车	购物车、商品信息、商品项、价格总计、赠品信息、商品详细页信息、商品选择信息、库存信息、订单信息	购物车页面、商品详细页面
	管理收货地址	收货地址信息	收货地址管理界面
	管理支付方式	支付方式信息	支付方式管理界面
	订购商品	购物车、商品项、商品信息、价格总计、赠品信息、商品详细页信息、商品选择信息、库存信息、订单信息	购物车页面、订单结算页面
	管理浏览记录	浏览记录信息	浏览记录页面
	管理评价	商品评价信息	评价管理页面
	管理店铺关注	店铺关注信息	关注收藏页面
	管理商品收藏	商品收藏信息	关注收藏页面
	管理优惠券	优惠券信息	优惠券管理页面
	管理会员	会员信息	会员管理页面

用户	在线咨询	用户信息、商家信息	客服页面
商家	管理订单	订单信息	订单管理页面
商家	管理店铺	店铺信息、商品信息	店铺管理页面
管理员	管理商品	商品信息	商品管理页面
用户	管理账户信息	用户信息	账户管理页面
商家	登录账户	用户信息	登录页面
管理员	注册账户	用户信息	注册页面

3.2.1 初始类图的识别和分析

概念候选集的识别与构造板块，通过上述的概念发现发类图，并进一步结合具体的概念是否需要发生改变，是否需要去手动去变更其属性和状态^[26]，通过分析为什么在该用例中要识别这些概念，确定了最终的初始类图。

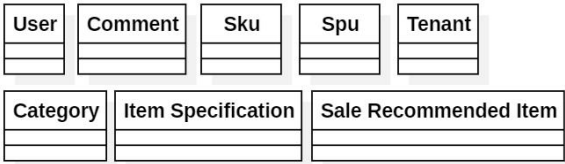


图 3.2 浏览商品初始类图

Fig.3.2 Browse Products Initial Class Diagram

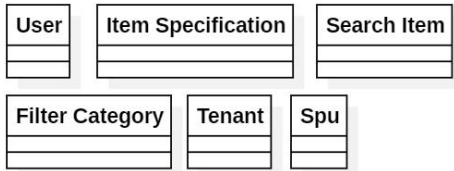


图 3.3 搜索商品初始类图

Fig.3.3 Search Products Initial Class Diagram

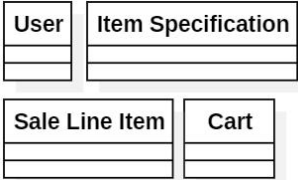


图 3.4 管理购物车初始类图

Fig.3.4 Manage Cart Initial Class Diagram

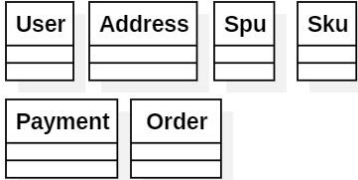


图 3.5 订购商品初始类图

Fig.3.5 Book Products Initial Class Diagram

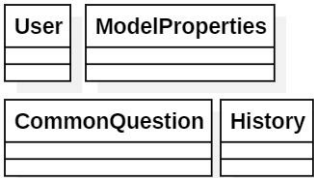


图 3.6 在线咨询初始类图

Fig.3.6 Online Consultation Initial Class Diagram

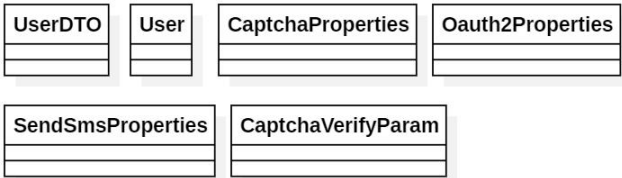


图 3.7 登录账户初始类图

Fig.3.7 Sign in Initial Class Diagram

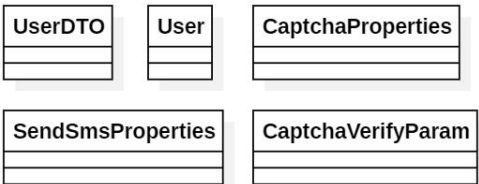


图 3.8 注册账户初始类图

Fig.3.8 Sign up Initial Class Diagram

3.2.2 概念类图的绘制

通过定义类间关系（一般关系、继承/泛化关系、聚合关系和组合关系），识别关系间的重数，最终得到的概念类图如下所示。

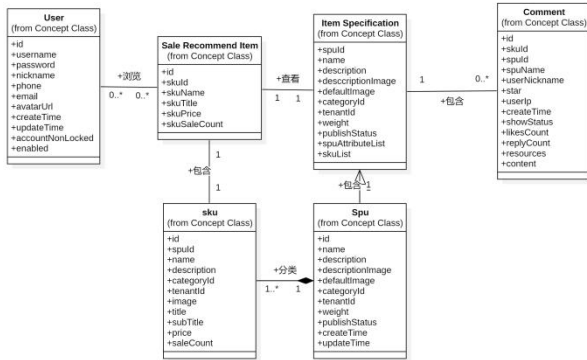


图 3.9 浏览商品概念类图

Fig.3.9 Browse Products Concept Class Diagram

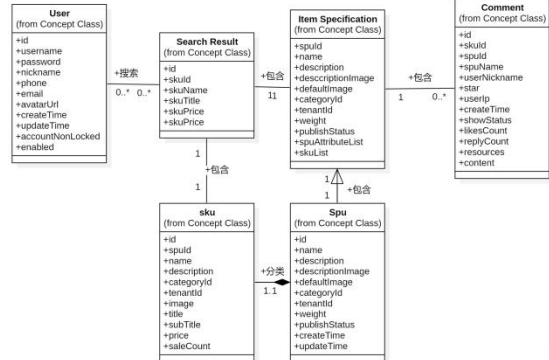


图 3.10 搜索商品概念类图

Fig.3.10 Search Products Concept Class Diagram

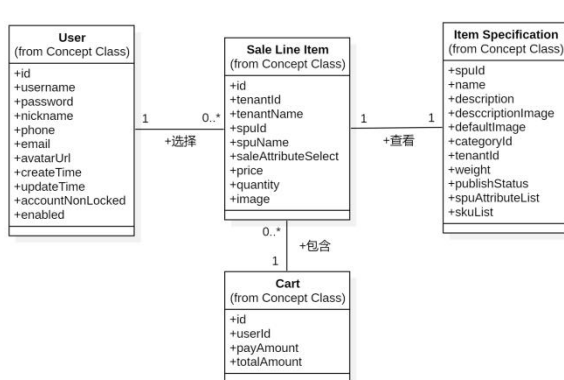


图 3.11 管理购物车概念类图

Fig.3.11 Manage Cart Concept Class Diagram

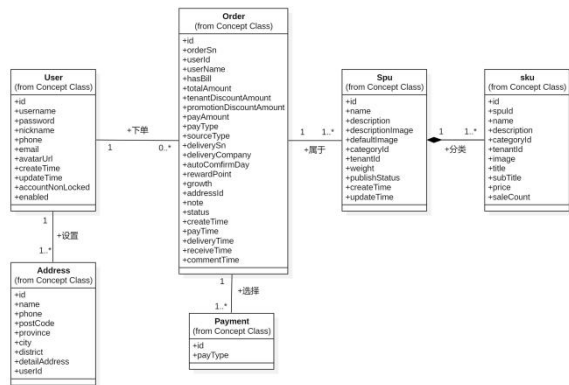


图 3.12 订购商品概念类图

Fig.3.12 Book products Concept Class Diagram

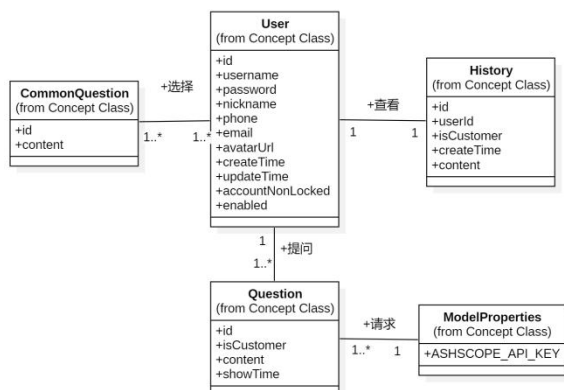


图 3.13 在线咨询概念类图

Fig.3.13 Online Consultation Concept Class Diagram

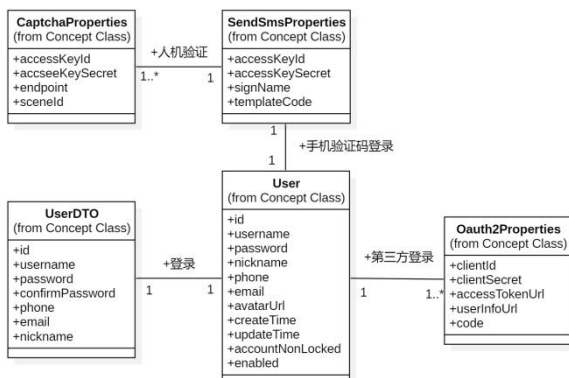


图 3.14 登录账户概念类图

Fig.3.14 Sign up Concept Class Diagram

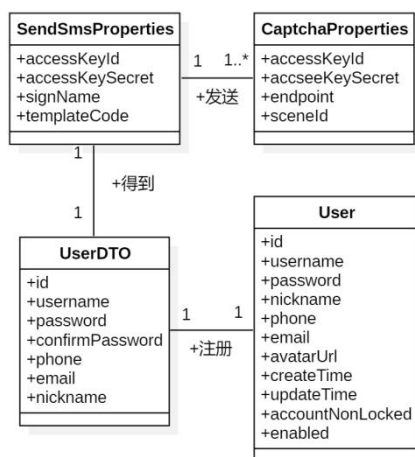


图 3.15 注册账户概念类图

Fig.3.15 Sign up Concept Class Diagram

3.3 行为模型的构建

在系统操作识别与用例序列图构建板块，根据详细文本用例来得到对应的动作序列，并判断在这些动作序列中，哪些动作会引发系统响应事件，将它们提取出来作为系统操作^[3]。判断其中可能出现的事件，使用 **Fragment** 控制段将其表达出来，根据最新的 UML 标准^[35]，绘制出相对应的用例序列图。

3.3.1 系统操作识别与用例序列图绘制

基于上述详细文本用例，总结出对应的动作序列如下所示：

1. 浏览商品动作序列：

- (1) 用户进入首页；
- (2) 用户请求推荐商品；
- (3) 系统返回 SKU 列表，包括商品销售项的名称、图片和价格；
- (4) 用户通过滚动页面或者点击“下一页”等操作来查看更多的商品信息；
- (5) 系统返回更多的 SKU 列表；
- (6) 用户点击具体的 SKU，进入该 SKU 所属的商品整体页面；
- (7) 系统返回该 SPU 的所有信息；
- (8) 用户查看该 SPU 的评价；
- (9) 系统返回该 SPU 的评价。

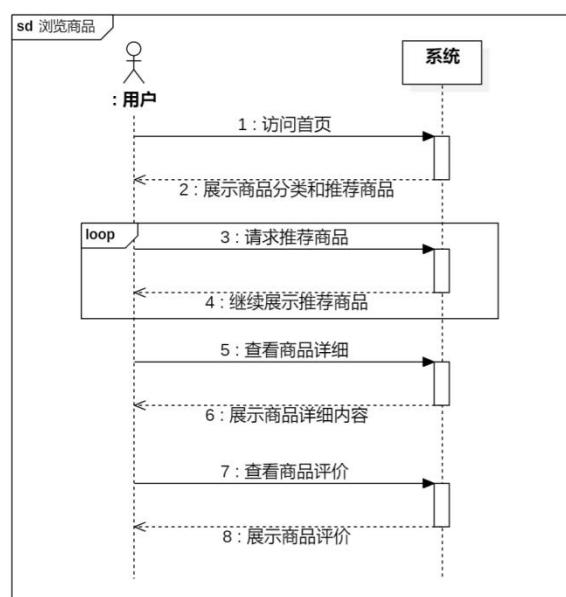


图 3.16 浏览商品用例序列图

Fig.3.16 Browse Products Use Case Sequence Diagram

2. 搜索商品动作序列：

- (1) 用户打开搜索框，进入搜索页面；
- (2) 用户输入关键词进行“搜索”；
- (3) 系统返回匹配的商品；
- (4) 用户通过筛选结果页的商品信息（品种、价格等）来进一步细化搜索结果；
- (5) 系统返回进一步细化搜索得到的搜索结果页。

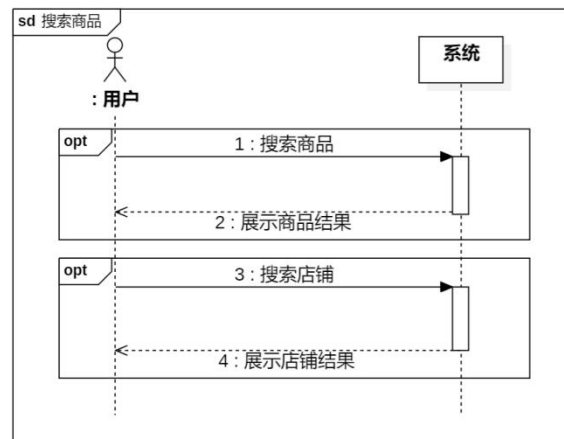


图 3.17 搜索商品用例序列图

Fig.3.17 Search Products Use Case Sequence Diagram

3. 管理购物车动作序列：

- (1) 用户查看指定的商品信息；
- (2) 系统展示商品信息；
- (3) 用户修改指定商品数量；
- (4) 系统判断是否能够修改，如果可以修改则返回展示修改后的数量；
- (5) 用户移除指定商品；
- (6) 系统返回移除结果；
- (7) 用户选定某几项商品进行总价查看；
- (8) 系统返回优惠后的商品总价；
- (9) 用户选定某几项商品进行结账；
- (10) 系统返回结账结果。

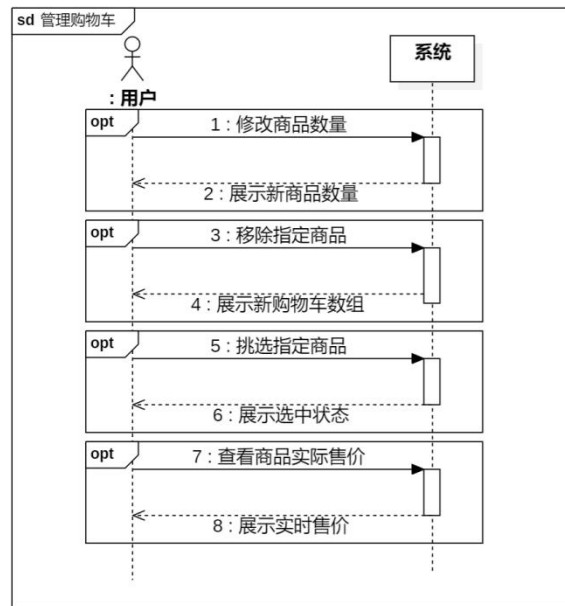


图 3.18 管理购物车用例序列图

Fig.3.18 Manage Cart Use Case Sequence Diagram

4. 订购商品动作序列:

- (1) 用例开始于用户完成了购物车内商品的选择，并点击结算键；
- (2) 用户在确认订单界面，选择自己收货的地址信息；
- (3) 更改想要的商品的数量，选择不同的配送服务和支付途径；
- (4) 系统返回最终的价格以及优惠手段。

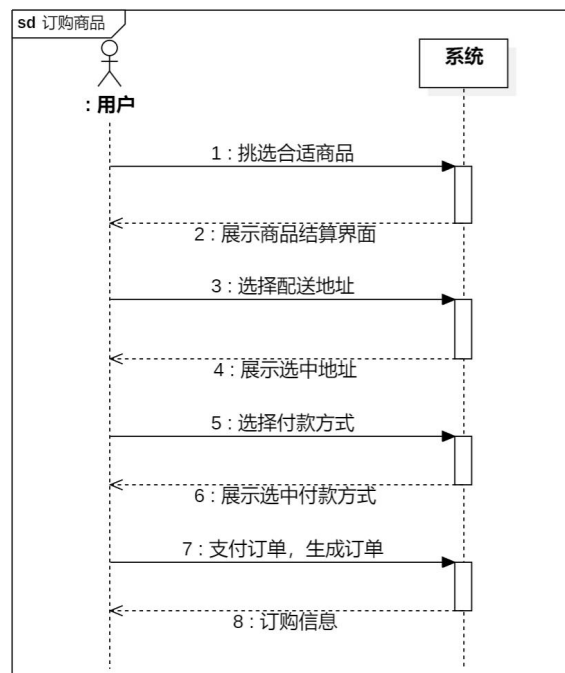


图 3.19 订购商品用例序列图

Fig.3.19 Book Products Use Case Sequence Diagram

5. 在线咨询动作序列：

- (1) 用户进入网站点击咨询按钮；
- (2) 用户通过聊天框发起询问；
- (3) 用户通过快捷输入快速发起询问；
- (4) 用户输入“人工客服”寻求人工帮助；
- (5) 系统针对用户提问给出针对性意见；
- (6) 系统识别到人工客服寻求请求，接入人工客服；
- (7) 人工客服对用户做出回应；
- (8) 询问结束退出问咨询界面。

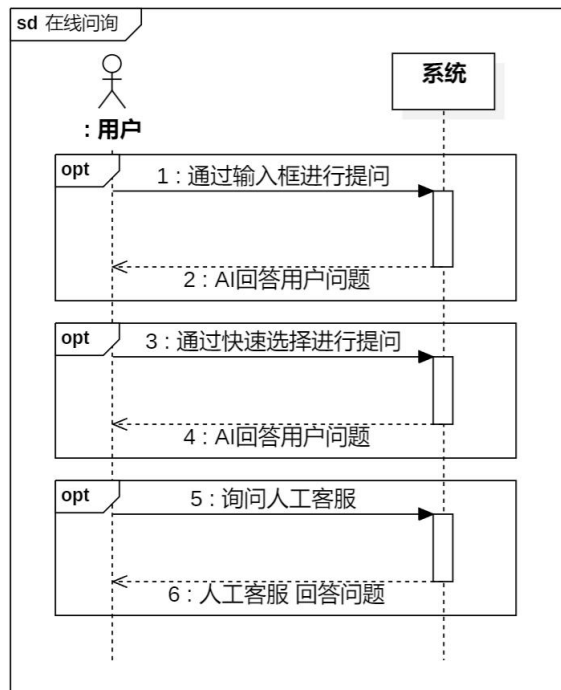


图 3.20 在线问询用例序列图

Fig.3.20 Consult Online Use Case Sequence Diagram

6. 登录账户动作序列：

- (1) 用户选择登录方式；
- (2) 系统显示登录页面；
- (3) 系统验证登录是否通过；
- (4) 如果登录成功，系统将该用户信息加入 token 进行返回。

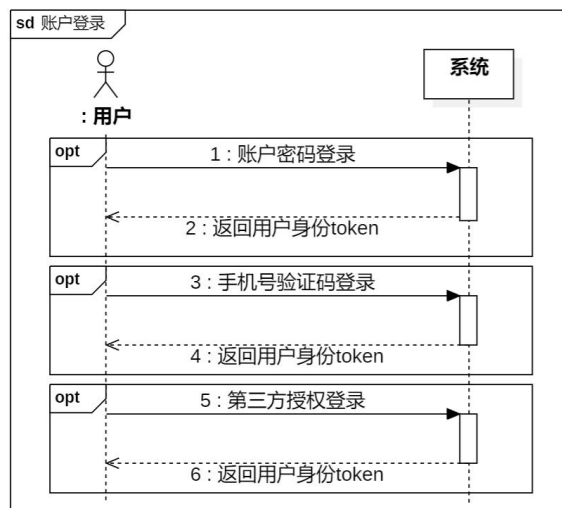


图 3.21 账户登录用例序列图

Fig.3.21 Sign in Use Case Sequence Diagram

7. 注册账户动作序列:

- (1) 用户输入注册信息;
- (2) 系统验证输入的注册信息是否正确;
- (3) 用户申请激活账户;
- (4) 系统发送激活邮件;
- (5) 用户点击激活链接完成激活;
- (6) 系统验证邮件激活是否有效。

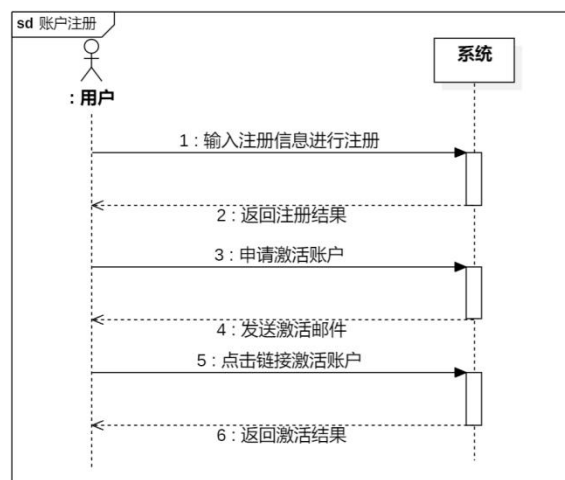


图 3.22 账户注册用例序列图

Fig.3.22 Sign up Use Case Sequence Diagram

3.3.2 操作契约定义

根据系统操作定义对应的操作契约，以下将从每个实例用力序列图中抽取出一个系统操作，根据此系统操作进行对应的操作契约撰写，具体如下。

表 3.18 用户浏览商品详细信息—操作契约表

Table 3.18 View detail information for products in Operation Contract

主要元素	说明
1. 系统操作	boolean viewItemComment(quantity)
2. 交叉引用	浏览商品（用例）
3. 前置条件	用户进入该销售平台； 用户进入商品详细页面。
4. 后置条件	创建 Comment（创建实例）； Comment.id 赋值为传入的 id（修改属性）； Comment 与 CommentItem 关联（形成关联）。

表 3.19 用户进行详细搜索—操作契约表

Table 3.19 Search Products Carefully in Operation Contract

主要元素	说明
1. 系统操作	Product searchItem(itemType,itemName)
2. 交叉引用	搜索商品（用例）
3. 前置条件	用户进入该销售平台； 用户输入搜索内容，点击搜索按钮。
4. 后置条件	创建 saleLineItem（创建实例）； saleLineItem.type 赋值（修改属性）； saleLineItem.name 赋值（修改属性）； saleLineItem 与 ItemSpecification 关联（形成关联）； saleLineItem 与搜索展示框关联（形成关联）。

表 3.20 用户修改商品数量—操作契约表

Table 3.20 User modifies the quantity of goods in Operation Contract

主要元素	说明
1. 系统操作	boolean changeGoodQuantity(quantity)
2. 交叉引用	管理购物车（用例）
3. 前置条件	用户点击购物车内某商品的数量加号按钮； 商品的库存未达到限额。
4. 后置条件	创建 GoodLineItem 的实例（创建实例）； GoodLineItem.quantity 设置为 quantity（修改属性）； GoodLineItem 与当前 ItemSpecification 关联（形成关联）。

表 3.21 商品结算—操作契约表

Table 3.21 Products Settlement in Operation Contract

主要元素	说明
1. 系统操作	boolean Settlement(ProductList)
2. 交叉引用	订购商品（用例）
3. 前置条件	用户选择了购物车中某项或某几项商品； 用户点击了购物车中的结算按钮； 商品的库存不为 0。
4. 后置条件	创建 Product Settlement 的实例 PS（创建实例）； PS 与当前 Product 关联（形成关联）； PS.DeliverInfo 赋值到 DeliverInfo（修改属性）； PS.quantity 赋值到 Quantity（修改属性）； PS.price 赋值为 Price（修改属性）； PS 被关联到 Product Information（形成关联）。

表 3.22 用户智能客服问询—操作契约表

Table 3.22 Consult by AI in Operation Contract

主要元素	说明
1. 系统操作	String consult(content)
2. 交叉引用	在线问询（用例）
3. 前置条件	用户输入内容进行问询； 内容发送成功。
4. 后置条件	创建 ConsultConent 的实例（创建实例）； ConsultConent 内容不断被 push（修改属性）； ConsultConent 与当前 User 关联（形成关联）。

表 3.23 生成 Token 令牌—操作契约表

Table 3.23 Generate Token in Operation Contract

主要元素	说明
1. 系统操作	boolean generateToken(User)
2. 交叉引用	登录账户（用例）
3. 前置条件	用户输入登录信息； 用户登录成功。
4. 后置条件	创建 Token 的实例（创建实例）； Token.id 设置为 user.id（修改属性）； Token.time 设置为 2h+随机数（修改属性）； User 与当前 Token 关联（形成关联）。

表 3.24 系统发送激活邮件—操作契约表

Table 3.24 Activation Email Sent by System in Operation Contract

主要元素	说明
1. 系统操作	<code>boolean sendActivationEmail()</code>
2. 交叉引用	注册账户（用例）
3. 前置条件	用户必须处于注册状态； 该用户账户未激活。
4. 后置条件	创建 User 的实例（创建实例）； User.enabled 设置为 true（修改属性）； User.accountNonLocked 设置为 true（修改属性）； User 与 Role 关联（形成关联）。

3.4 本章小结

本章针对该在线销售平台系统进行了需求分析，使用基于模型软件工程方法，分别依次通过用例模型、数据模型和行为模型三个维度进行了需求建模。分析出该系统所涉及的功能用例，并最终得到了系统的功能用例、概念类图、用例序列图和系统的操作契约。

第4章 系统设计

在本章中，将结合需求分析阶段得到的两个重要产物（概念类图和系统操作契约），并依照用例序列图，设计出对应的对象序列图，这将为系统的代码实现指引方向。同时，将基于概念类图初步对系统数据库进行设计，完善系统数据层。

4.1 设计原则

系统的组成与架构、接口与交互流程在系统设计阶段被定义，有部分设计原则是需要被遵循的，本销售平台的设计将遵循以下四项原则^[5]：

1. 单一职责原则：类与类、模块与模块之间的职责应该进行划分，保证职责的单一性，解耦合，提高可维护性。
2. 开放封闭原则：实体对扩展开放，对修改封闭，提高拓展性和稳定性。
3. 接口隔离原则：接口独立且隔离，仅提供需要被使用的接口。
4. 可复用性原则：模块复用性高、避免大面积重复代码的出现。

4.2 功能架构设计

根据需求分析中得到的系统功能用例，将本论文销售平台整体划分为以下几个功能服务，对应的系统功能架构图如下所示：

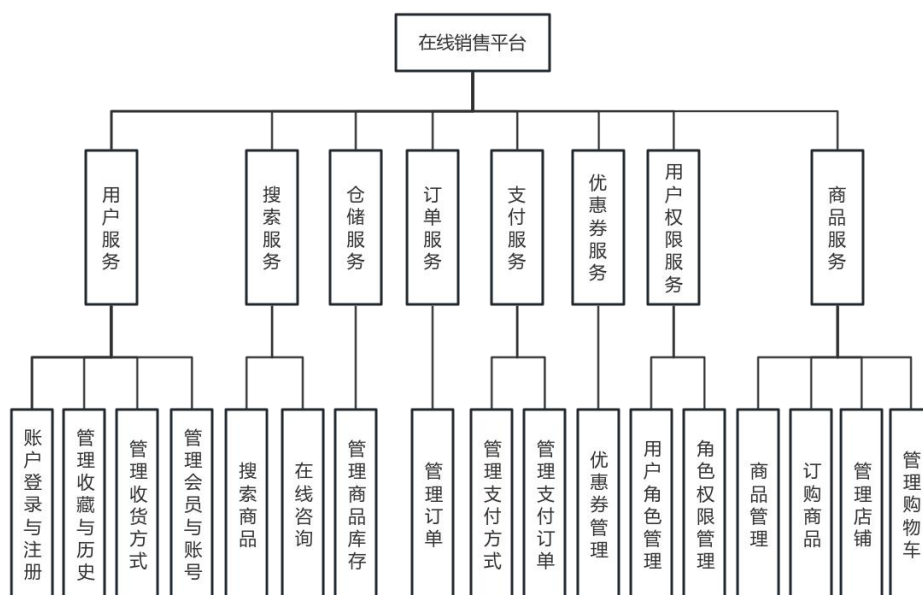


图 4.1 在线销售平台功能架构设计

Fig.4.1 E-Commerce Platform Function Design of Architecture

4.3 对象序列图设计

本小节将通过构建对象序列图的方式，展示平台核心功能用例的设计和建模。在系统操作契约中，我们可以由霍尔三元组得到一组非常重要的产出物，即后置条件。这些后置条件具体来说有以下三种：

- 1. 对象被创建或者销毁
- 2. 对象的属性发生变更
- 3. 对象之间的链接关系被创建或者销毁

将这三类职责分别安置在对应的对象上面，就可以得到对应的对象序列图设计^[21]。不同于用例序列图的是，对象序列图将系统中的对象分职责区分好，而不仅仅是黑盒。由于实际用例过多，本论文将以需求分析中展示的七个示例用例继续深入，给出他们的对象序列图。其中，浏览商品用例将展开叙述，其他仅给出对应的对象序列图，用以展示类与类之间的交互逻辑关系。

1. 浏览商品对象序列图：在浏览商品用例中，除概念类图中识别到的商品分类、商品推荐列、商品信息和评价信息外，还涉及首页和搜索结果页两个实体对象，由于篇幅限制，只展示了核心业务逻辑，用户从首页挑选类别，浏览商品后，点开商品详细页面查看商品的基本信息和用户的相关评论，从而完成浏览商品动作。类与类之间的交互逻辑关系如下所示。

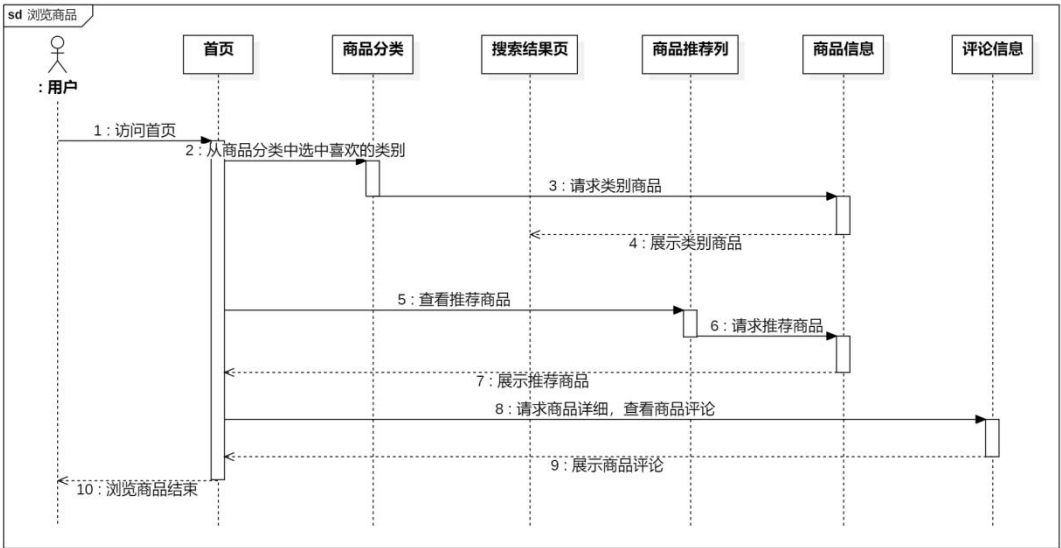


图 4.2 浏览商品对象序列图
Fig.4.2 Browse Products Object Sequence Diagram

2. 搜索商品对象序列图：

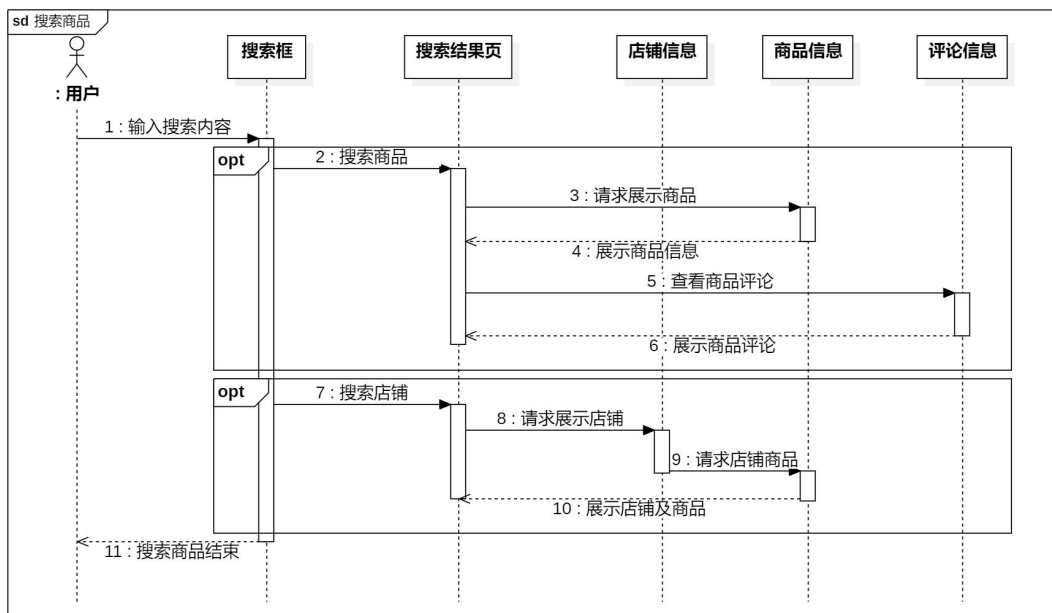


图 4.3 搜索商品对象序列图
Fig.4.3 Search Products Object Sequence Diagram

3. 管理购物车对象序列图：

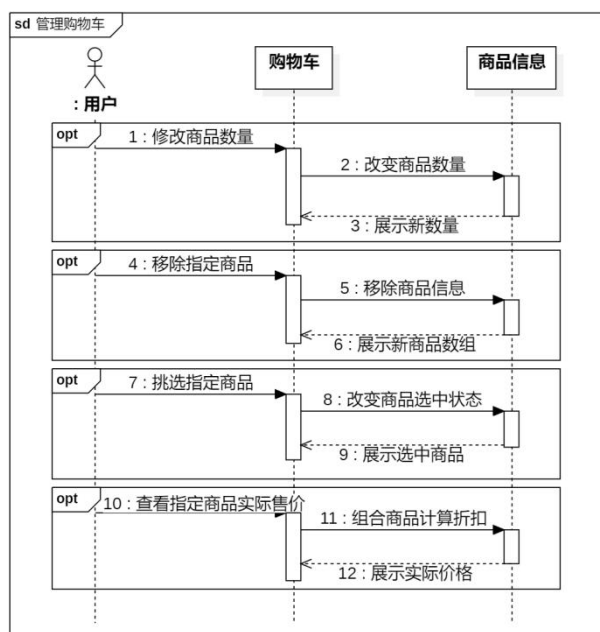


图 4.4 管理购物车对象序列图
Fig.4.4 Cart Management Object Sequence Diagram

4. 订购商品对象序列图:

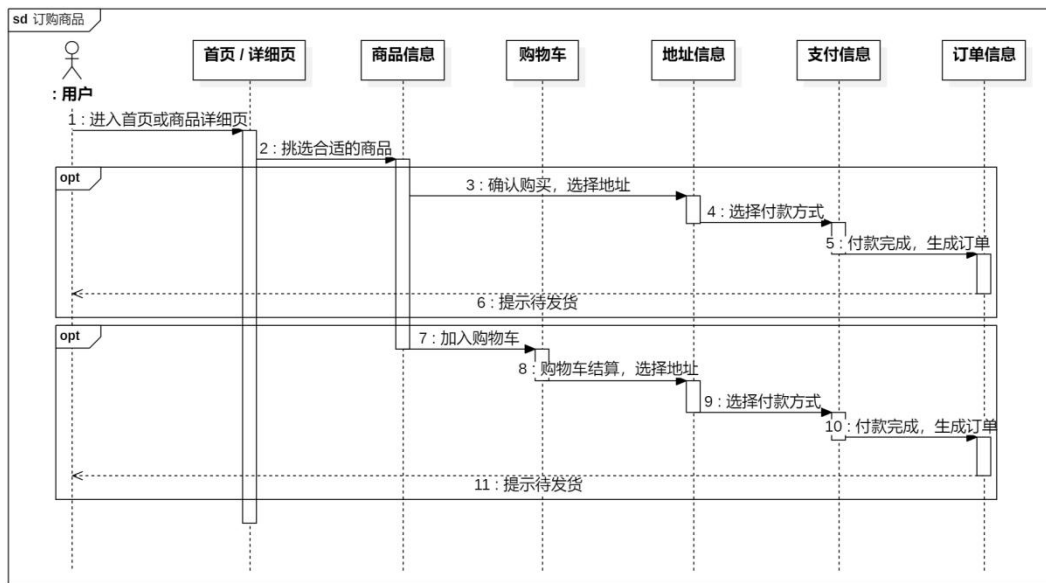


图 4.5 订购商品对象序列图

Fig.4.5 Book Products Object Sequence Diagram

5. 在线咨询对象序列图:

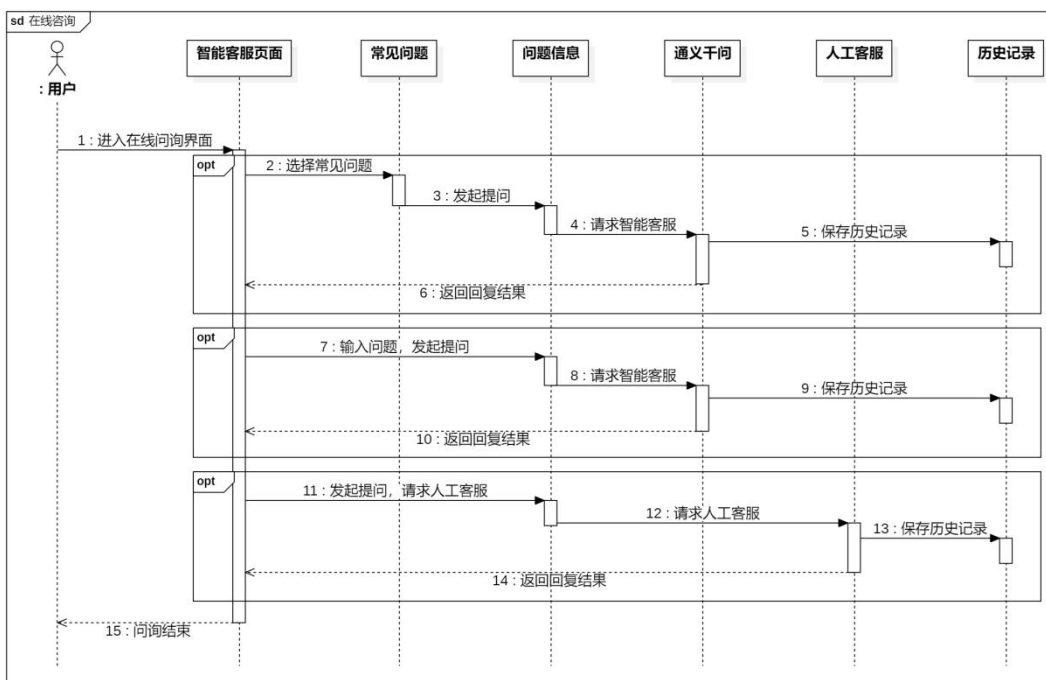


图 4.6 在线咨询对象序列图

Fig.4.6 Online Consultation Object Sequence Diagram

6. 登录账户对象序列图：

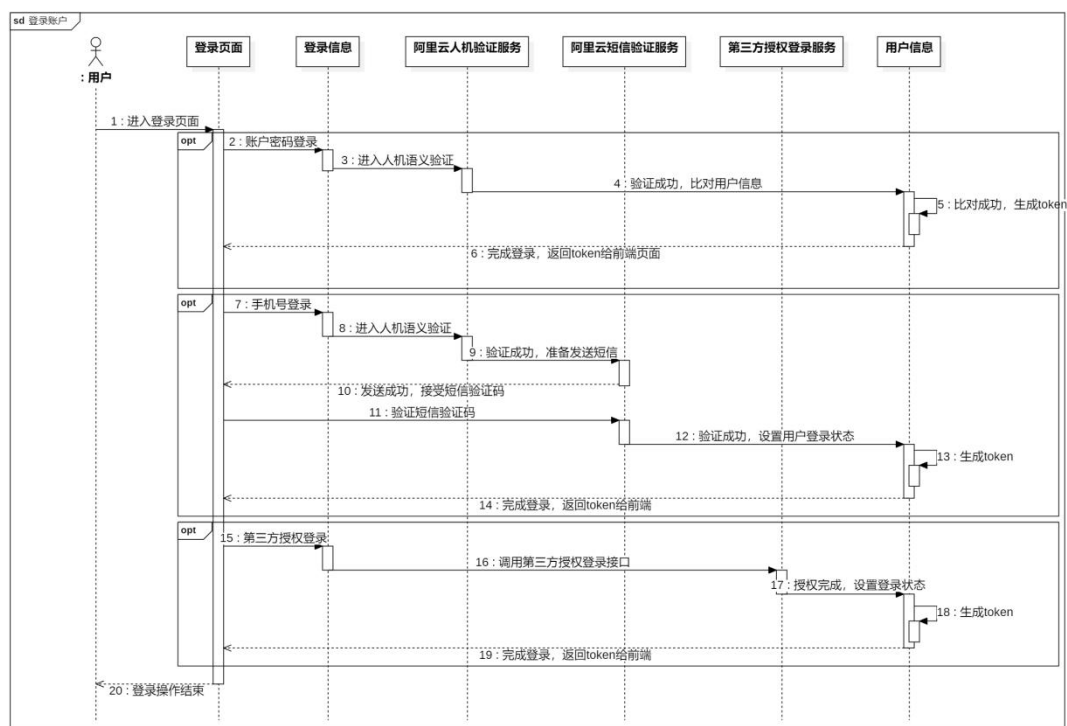


图 4.7 登录账户对象序列图

Fig.4.7 Login Account Object Sequence Diagram

7. 注册账户对象序列图：

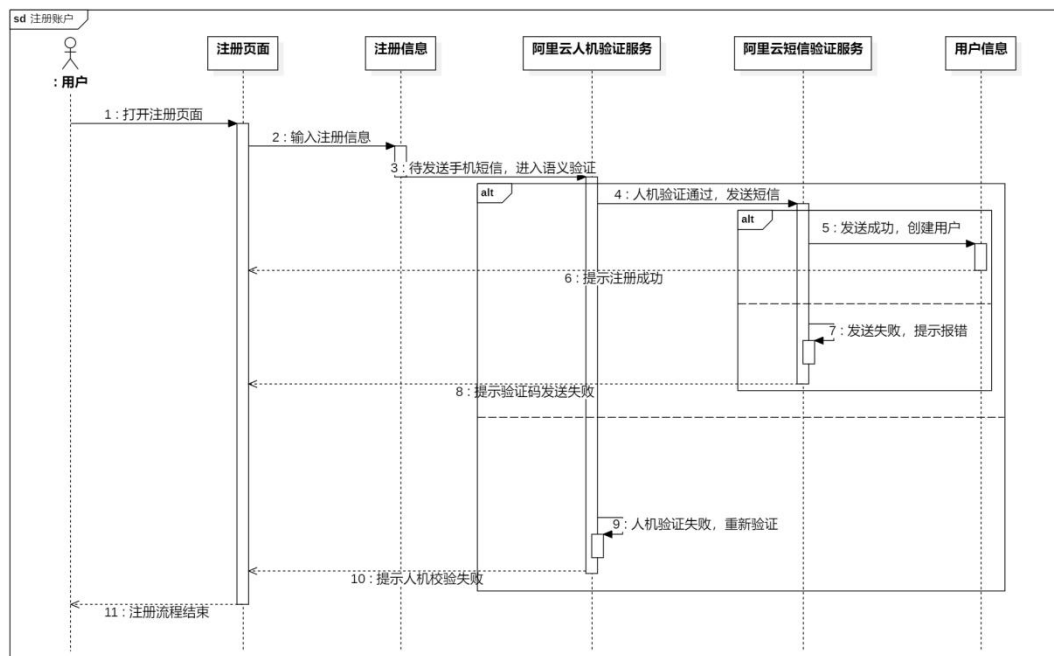


图 4.8 注册账户对象序列图

Fig.4.8 Registered Account Object Sequence Diagram

4.5 数据库设计

为了确保数据库表命名的一致性并强化命名规范，在本论文中，所有数据库表名将由对应的微服务模块名称与相关表的英文术语组合构成。本论文数据表主要涉及用户、商品、订单、支付、优惠券、用户权限（RBAC）、仓储模块，每一类的数据表都涉及该系统的微服务功能模块，相同类的数据表命名前缀相同。

表 4.1 数据表分模块
Table 4.1 Database Table - Divided by Module

表分类	前缀	描述
用户模块	user_	主要存储了用户的信息、收藏的店铺、收藏的商品、登录日志、成长值、积分、等级、第三方授权、地址信息等。
商品模块	product_	主要存储了商品的属性、分类、店铺信息、商品信息、SPU 信息、SKU 信息、商品评论等。
订单模块	order_	主要存储了订单信息、订单发票信息、订单操作历史、订单退款信息、订单退款原因等。
支付模块	pay_	主要存储了支付单号信息等。
优惠券模块	coupon_	主要存储了优惠券信息等。
用户权限模块 (RBAC)	rbac_	主要存储了角色信息和权限信息以及用户、角色和权限三者关系。
用户模块	user_	主要存储了用户的信息、收藏的店铺、收藏的商品、登录日志、成长值记录、积分记录、会员等级、第三方授权、地址信息等。
仓储模块	stock_	主要存储了商品的库存信息和货物订购信息。

在具体实现时，结合数据库三大范式，避免传递依赖，例如 RBAC 模型中，涉及用户类、角色类和权限类，将使用五张表来阐述他们之间的关系，分别是用户表、角色表、权限表和他们的相互关系所涉及的两张表。为了提高性能和效率，在部分表中添加了部分冗余字段^[14]。具体的数据库表设计如下所示，由于篇幅有限，这里仅列出商品模块的相关数据表。

表 4.2 商品属性表（product_attribute）
Table 4.2 Product Attribute (product_attribute)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	属性 id
name	varchar	30			属性名称
need_search	tinyint				需要检索 [0—否，1—是]

icon	varchar	40	属性的图标
value_select	varchar	255	可选值列表 [逗号分隔]
attribute_type	tinyint		属性的类型 [0—基本, 1—销售, 2—基本&销售]
enabled	tinyint		启用状态 [0 - 禁用, 1 - 启用]
category_id	bigint		所属的分类

表 4.3 商品分类表 (product_category)

Table 4.3 Product Category (product_category)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	分类 id
name	varchar	30			分类名称
parent_id	bigint				父分类 id
category_level	int				层级
show_status	tinyint				显示 [0—不显示, 1—显示]
sort	int				排序, 数字越大越靠后
icon	varchar	40			图标地址
product_unit	varchar	50			计量单位
product_count	int				商品数量

表 4.4 店铺表 (product_tenant)

Table 4.4 Tenant (product_tenant)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	店铺 id
name	varchar	30			店铺名
logo	varchar	255			店铺 logo 地址
description	longtext				店铺介绍
show_status	tinyint				显示 [0—不显示,1—显示]
sort	int				排序

表 4.5 分类店铺关系表 (product_category_tenant_relation)

Table 4.5 The Relation between Category and Tenant (product_category_tenant_relation)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	关系自增 id
tenant_id	bigint				店铺 id
category_id	bigint				分类 id
tenant_name	varchar	30			店铺名
category_name	varchar	30			分类名

表 4.6 商品 SPU 信息表 (product_spu_info)

Table 4.6 SPU Information (product_spu_info)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	商品 id
name	varchar	200			商品名称
description	varchar	1000			商品描述
description_image	varchar	255			商品描述长图
default_image	varchar	255			默认展示图片
category_id	bigint				所属分类 id
tenant_id	bigint				店铺 id
weight	decimal	18			商品重量
publish_status	tinyint				上架状态 [0 — 下架, 1 — 上架]
create_time	datetime				商品 SPU 创建时间
update_time	datetime				商品 SPU 信息更新时间

表 4.7 商品 SPU 属性值表 (product_spu_attribute_value)

Table 4.7 SPU Attribute Value (product_spu_attribute_value)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	属性值自增 id
spu_id	bigint				商品 id
attribute_id	bigint				属性 id
attribute_name	varchar	200			属性名
attribute_value	varchar	200			属性值
sort	int				排序

表 4.8 商品 SPU 评论表 (product_spu_comment)

Table 4.8 SPU Comment (product_spu_comment)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	评论 id
sku_id	bigint				sku_id
spu_id	bigint				spu_id
spu_name	varchar	255			商品名字
user_nick_name	varchar	255			会员昵称
star	tinyint	1			评分
user_ip	varchar	64			会员 ip
create_time	datetime				创建时间
show_status	tinyint	1			显示 [0—不显示, 1—显示]
likes_count	int				点赞数

reply_count	int		回复数
resources	varchar	1000	评论图片/视频
content	text		内容
user_avatar	varchar	255	用户头像
comment_type	tinyint		评论类型

表 4.9 商品 SPU 二级评论表 (product_spu_comment_replay)

Table 4.9 SPU Second Level Comment (product_spu_comment_replay)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	关系 id
comment_id	bigint				评论 id
replay_id	bigint				回复 id

表 4.10 商品 SKU 信息表 (product_sku_info)

Table 4.10 SKU Information (product_sku_info)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	skuId
spu_id	bigint				spuId
name	varchar	255			sku 名称
description	varchar	2000			sku 介绍描述
category_id	bigint				所属分类 id
tenant_id	bigint				品牌 id
image	varchar	255			图片
title	varchar	255			标题
subtitle	varchar	2000			副标题
price	decimal	18			价格
sale_count	bigint				销量

表 4.11 商品 SKU 销售属性值表 (product_sku_sale_attribute_value)

Table 4.11 SKU Sale Attribute Value (product_sku_sale_attribute_value)

字段名称	类型	长度	不为空	主键	字段说明
id	bigint		√	√	属性值自增 id
sku_id	bigint				sku_id
attribute_id	bigint				attr_id
attribute_name	varchar	200			销售属性名
attribute_value	varchar	200			销售属性值
sort	int				排序

4.6 本章小结

本章通过系统分析阶段的产出物（用例序列图、概念类图和系统操作契约），进行了系统更加详细的设计，最终得到了对应的对象序列图。同时在概念类图的引导下，进行了数据库设计，由于篇幅有限，以商品模块作为示例展示了大致的数据库设计。

第 5 章 系统实现

5.1 工具与环境

本论文所使用的开发工具和软件配置如下所示：

1. 后端 IDE：IntelliJ IDEA 2023.1.1
2. 前端 IDE：Visual Studio Code
3. 数据库可视化软件：Navicat Premium 16
4. 数据库：MySQL 8.0.13
5. 缓存：Redis 7.2.4
6. 检索中间件：Elasticsearch 8.13.0, Kibana 8.13.0
7. 配置与服务注册发现中心：Nacos 2.2.3
8. 消息队列：RocketMQ 5.2.0
9. 流控保护：Sentinel 1.8.6
10. 分布式事务管理：Seata 2.0.0
11. 日志模块：Elasticsearch 8.13.0, Logstash 8.13.0, Kibana 8.13.0
12. 开发系统：Window 11
13. 服务器系统：Alibaba Cloud Linux Release 3
14. 接口测试：Apifox, Knife4j

5.2 技术架构

本论文使用 Vue 作为前端开发框架，结合 Nginx 集群实现前端高可用。用户可通过 H5 应用、微信小程序和手机 APP 实现该销售平台的访问。使用 Vue 结合 Taro 实现多种小程序快速开发。

本论文使用 Spring Cloud Alibaba 作为后端微服务框架，使用 Kubernetes 实现后端高性能、高可用、高并发。在请求进来前，使用 Spring Cloud Gateway 和 Spring Cloud LoadBalancer 作为服务网关和负载均衡器，在微服务板块，使用 Nacos 作为配置中心和服务注册与发现中心，使用 OpenFeign 作为服务远程调用组件，使用 Seata 完成分布式事务管理，并使用 Sentinel 对请求进行流量控制与降级保护。在数据层，使用 MySQL 数据库和 Redis 缓存，使用 RocketMQ 作为消息队列，使用 Elasticsearch 作为检索中间

件。并结合 Minio、Alibaba Captcha、Alibaba SMS 作文件存储、人机验证和短信服务中间件。在日志板块，使用 ELK 组件进行日志的采集、分析与可视化。在智能客服方面，由于 Spring AI 在国内并不适用，因此通过自定义结合通义千问作为大模型智能客服

本论文使用 Kubernetes 和 Alibaba Codeup 实现 Pipeline，通过设置上推 master 发布分支触发器，自动编译打包合并到 master 分支的代码，生成制品，并上推到测试环境容器进行测试。

最终，本论文的核心技术框架选型如下图所示：

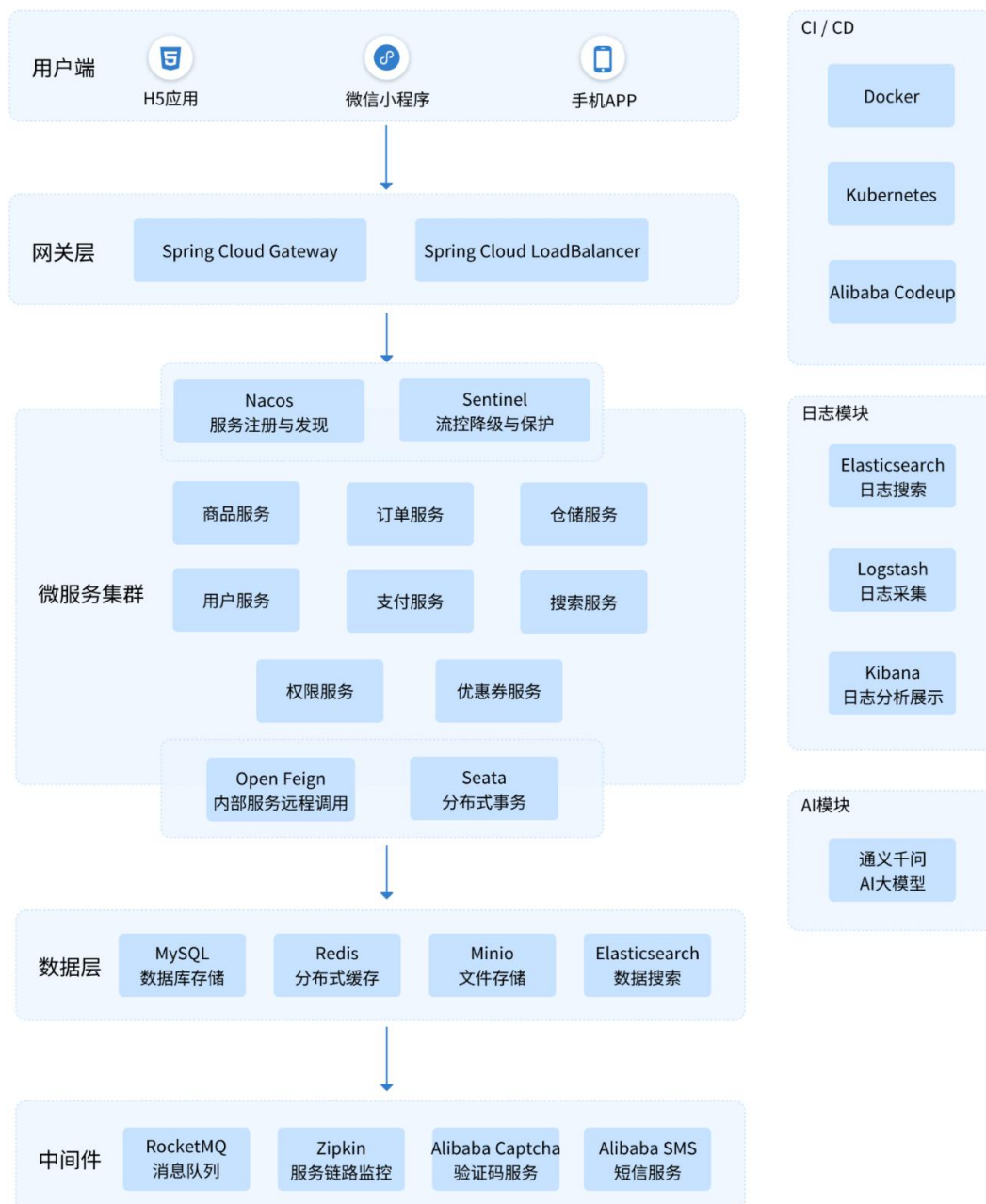


图 5.1 技术框架

Fig.5.1 Service Technology Framework

5.3 核心功能实现

在核心功能实现部分，由于大部分管理类功能实现和界面表示都相类似，这里仅以部分作为示例进行功能实现展示。

5.3.1 i18n 国际化

使用 vue-i18n@next 作为项目内容 i18n 的核心，结合 i18n pro 以及 Element Plus 中的 configuration provider 组件实现组件的 i18n。我将转换按钮封装成了组件，通过组件可完成语言切换。以登录页面作为展示，实际实现效果如下：

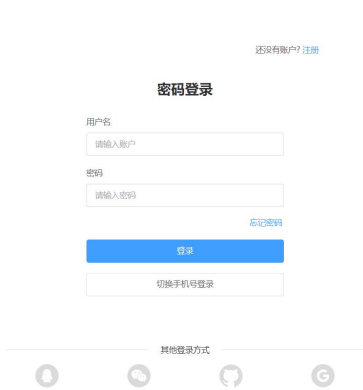


图 5.2 中文登录
Fig.5.2 Login in Chinese

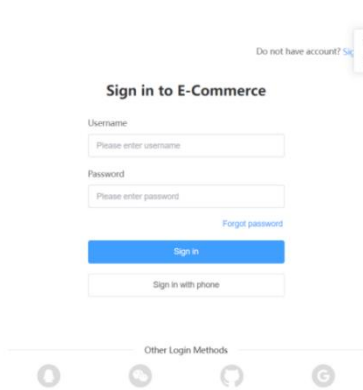


图 5.3 英文登录
Fig.5.3 Login in English

5.3.2 用户登录

用户可以通过密码登录、验证码登录以及三方授权登录进行该平台登录。登录成功后，服务器将返回该用户的身份 token 作为登录令牌，以便于该用户后续访问其他页面。三种登录页面展示效果如下所示。



图 5.4 密码登录
Fig.5.4 Login with Password



图 5.5 手机号验证码登录
Fig.5.5 Login with Phone and Code

针对登录、注册以及发送验证码功能，为了有效预防爬虫，使用了阿里云的人机验

验证码 2.0。为了避免提高正常用户的操作体验，通过人机验证和风险识别两种方案相结合，提高正常用户操作的无感知和流畅度的同时，达到对机器人的有效预防。本文使用的人机验证方式为空间推理，所谓的空间推理就是选择图片上满足语义的图形，从而达到验证的效果。下面是一个简单的空间推理验证示意图：

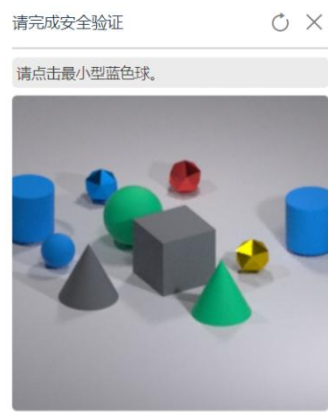


图 5.6 空间推理待验证

Fig.5.6 Spatial Reasoning Verification

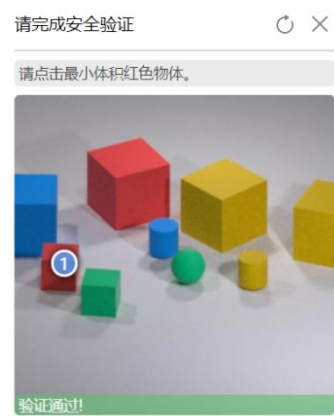


图 5.7 空间推理验证成功

Fig.5.7 Spatial Reasoning Verification

当通过语义验证码验证后，前端将语义验证码结果字符串整体作为请求头发送给服务器，服务器申请发送手机验证码，并将语义验证结果和业务请求结果（手机验证码发送情况）返回给前端页面，若都成功，将会收到短信验证码如下，并将验证码存入 Redis 等待手机登录验证。



图 5.8 短信验证码

Fig.5.8 SMS Verification Code

本系统通过阿里云发送短信验证码，通过 `accessKeyId` 和 `accessKeySecret` 申请与阿里云短信服务器建连，发送验证码的业务代码逻辑简单，这里不做过多阐述，仅对发送短信工具类附上核心代码。

```
1. public static Map<String, Object> sendSms(String phoneNumber, String code) {
2.     // ... 配置认证信息
3.     // ... 配置客户端
4.     // 请求参数配置
5.     SendSmsRequest sendSmsRequest = SendSmsRequest.builder()
```

```

6.     .signName(sendSmsProperties.getSignName())
7.     .templateCode(sendSmsProperties.getTemplateCode())
8.     .phoneNumbers(phoneNumber)
9.     // {"code": "123456"}
10.    .templateParam("{\"code\":\"" + code + "\"}")
11.    .build();
12.    // 请求短信服务接口
13.    try {
14.        CompletableFuture<SendSmsResponse> response = client.sendSms(sendSmsRequest);
15.        SendSmsResponse resp = response.get();
16.        SendSmsResponseBody body = resp.getBody();
17.        if (SUCCESS_CODE.equals(body.getCode())) {
18.            log.info("Send SMS successfully, bizId is {}, requestId is {}", body.getBizId(),
                , body.getRequestId());
19.            return Map.of("isSuccess", true, "message", "验证码发送成功");
20.        } else {
21.            log.info("Send SMS failed, code is {}, message is {}", body.getCode(), body.ge
                tMessage());
22.            return Map.of("isSuccess", false, "message", body.getMessage());
23.        }
24.    } catch (Exception e) {
25.        log.error("Send SMS failed, exception is {}", e.getMessage());
26.    } finally {
27.        client.close();
28.    }
29.    return Map.of("isSuccess", false, "message", "验证码发送失败");
30. }

```

图 5.9 短信验证码发送核心代码

Fig.5.9 Send SMS Verification Code Core Code

由于账户密码登录流程过于简单，即判断用户是否存在，比对用户信息，生成 token 返回，这里不做过多赘述。针对手机验证码流程为示例，由于频繁调用语义验证将对用户体验造成非常不好的影响，因此额外加入风险验证，当判断当前用户所处环境安全的情况下，执行无感验证操作，不弹出对应的语义验证弹窗，直接通过该次语义验证，对应的时序图如下所示：

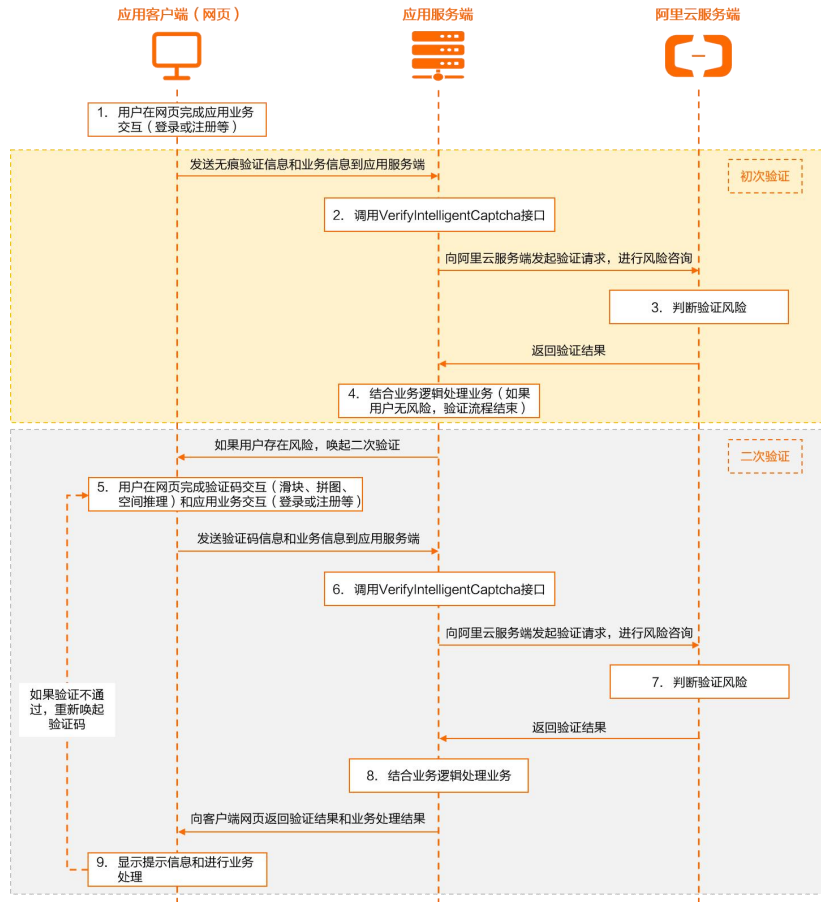


图 5.10 人机识别无痕验证对象序列图

Fig.5.10 Machine Identification Non-inductive Verification Object Sequence Diagram

本论文通过修改验证信息附加方式，将验证信息附加在请求头上，解耦了人机识别验证和业务请求，更方便后续业务代码的修改和添加新的人机验证请求。

由于本论文的服务器是微服务的，因此不通过 session 进行权限认证，而通过 token 进行身份验证，有效期为 2h 加上随机数，当访问页面的时候，将会不断刷新该令牌的有效期。核心代码通过 Gateway 微服务的 JwtGlobalFilter 过滤器实现。

```

1. @Override
2. public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
3.     ServerHttpRequest request = exchange.getRequest();
4.     // ... 指定路径放行
5.     // 获取请求头中的 token
6.     List<String> headers = request.getHeaders().get("Authorization");
7.     if (headers == null || headers.isEmpty()) {
8.         throw new JwtException(JwtStatus.NON_JWT.getMessage());
9.     }
10.    String token = headers.get(0).substring("Bearer ".length());
11.    // 判断缓存中是否存在该 token
12.    Map map = (Map) redisTemplate.opsForValue().get(token);
  
```

```

13. if (map == null) {
14.     throw new JwtException(JwtStatus.USER_SIGN_OUT.getMessage());
15. }
16. User user = BeanUtil.fillBeanWithMap(map, new User(), false);
17. // 解析 token
18. Map<String, Object> claims = JwtUtil.parseToken(token);
19. if (claims == null) {
20.     throw new JwtException(JwtStatus.NON_PAYLOAD.getMessage());
21. }
22. // ... 延长 token 时间
23. // ... 判断用户权限
24. // 传递用户信息
25. CustomContext customContext = new CustomContext(user.getId(), token);
26. ServerWebExchange webExchange = exchange.mutate()
27.     .request(builder -> builder.header("user", JSONUtil.toJsonStr(customContext)))
28.     .build();
29. return chain.filter(exchange);
30. }

```

图 5.11 JwtGlobalFilter 核心代码
Fig.5.11 JwtGlobalFilter Core Code

5.3.3 用户注册

本论文额外提供用户注册功能，此外，当使用手机号验证码登录或者第三方授权登录的时候，如果所登录的账户不存在，将会自动创建账户，随机生成十二位密码，密码开启强验证，后续如果使用账户密码登录，需要重置密码。

5.3.4 用户找回密码

当用户输入邮箱后，服务端将通过 `spring-boot-starter-mail` 将重置密码页面地址（附带 token）发送给用户，通过生成 256 位加密密钥并存入 Redis，保证此地址只能被使用一次。此密钥将会被附加在请求体中，随着用户请求页面一起带给服务端。用户直接通过重置密码页面即可对自己的账户进行密码重置操作。服务端接收到该 token 和重置的密码后，会从 token 中解析出对应的用户信息，找到指定用户后进行密码的重置操作，最终达到修改密码的效果。

[还没有账户? 注册](#)

找回密码

密码

确认密码

图 5.12 找回密码
Fig.5.12 Forget Password

5.3.5 首页三级分类

对商品整体进行分类使得商品类别结构更加清晰明了，用户可以根据分类信息快速地找到自己需要的商品，而商家也可以快速定位自己的商品属于什么类别。在三级分类中，第一级分类提供了大致的方向，第二级分类进一步细化，最后一级分类则提供了具体的、详细的类别点。通过三级分类，提高用户的满意度和体验。因此本论文在首页提供三级分类菜单推荐，通过 10 个大类、120 个二级分类和多个三级分类，准确为用户提供产品划分。



图 5.13 三级分类推荐
Fig.5.13 Three-level Classification Recommendation

通过 Java8 的特性 Stream 流，对查询出的全部类别信息进行父子结构组装，最终返回商品的三级分类信息，核心实现逻辑如下。

```

1. @Override
2. public List<CategoryVO> getList() {
3.
4. // 如果redis 中存在则直接返回

```

```

5. List<CategoryVO> categoryListFromRedis = (List<CategoryVO>) redisTemplate.opsFor
   Value().get("categoryList");
6. if (categoryListFromRedis != null) {
7.     return categoryListFromRedis;
8. }
9.
10. // 查询所有分类, 组装父子结构
11. List<Category> categoryList = categoryMapper.getListTree();
12.
13. // 1. 一级分类获取
14. List<CategoryVO> firstCategories = categoryList.stream()
15.     .filter(category -> category.getParentId() == 0)
16.     .map(CategoryVO::new)
17.     .toList();
18.
19. // 2. 子分类获取
20. List<CategoryVO> categoryVOs = firstCategories.stream()
21.     .peek(category -> category.setChildren(getChildren(category.getId(), categoryL
   ist)))
22.     .sorted(Comparator.comparingInt(CategoryVO::getSort))
23.     .collect(Collectors.toList());
24.
25. // 3. 存入 redis
26. redisTemplate.opsForValue().set("categoryList", categoryVOs);
27. return categoryVOs;
28. }

```

图 5.14 获取三级分类核心代码
Fig.5.14 Get Three-level Classification Core Code

5.3.6 浏览商品

用户通过首页进行商品的浏览，本论文参考京东商城的商品展示逻辑，使用 SKU 检索进行商品展示，而不是采用 SPU 检索。若用户开启了个性化推荐服务，将通过大模型分析用户画像，推荐出最符合用户想法的商品信息。

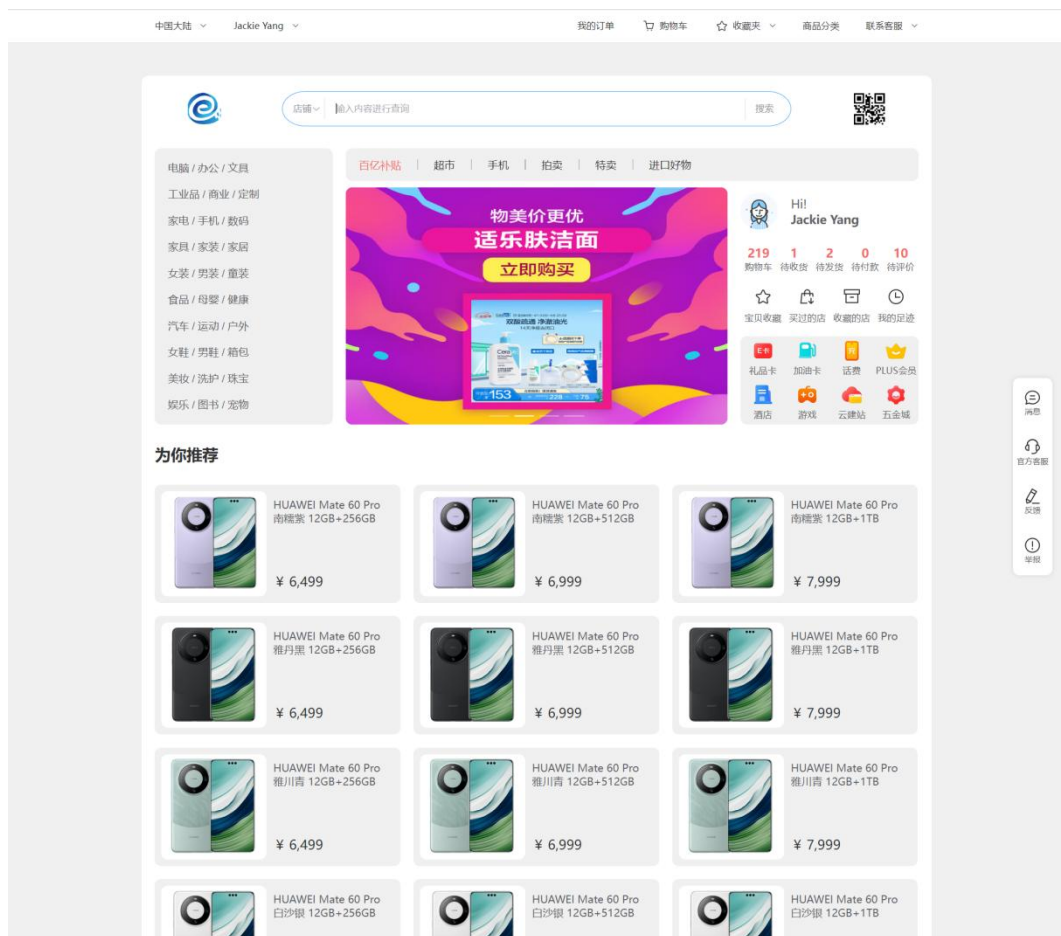


图 5.15 首页商品浏览

Fig.5.15 Browse Products in Home Page

点击进入详细商品页面时，通过携带 SKU 信息的方式，在商品详细页自动选择对应的 SKU 信息。商品详细页由商品基础信息、销售信息以及商品详细信息和 SPU 商品评价四部分组成。用户可在商品详细页查看商品图片、销售信息以及商品的详细信息。由于此处核心实现是将商品 SKU 信息拼接进 SPU 信息中去，与三级分类获取拼接方式类似，这里不做过多核心代码阐述。最终能得到结果如下，由于 JSON 过长，大部分相同内容已省略。

```

1. {
2.   "code": 200,
3.   "message": "商品列表获取成功",
4.   "data": [{
5.     "spuId": 28,
6.     "name": "Apple/苹果 iPhone 15 (A3092)",
7.     "description": "Apple/苹果 iPhone 15 (A3092)",
8.     "descriptionImage": "http://47.108.55.112:9000/e-commerce-platform/spu/e1679c3d57cc448f89fedf0b37580989.png",

```



```

9.      "defaultImage": "http://47.108.55.112:9000/e-commerce-platform/spu/7d55fff6
      5fed49309311d19a57c47653.jpg",
10.      "categoryId": 456,
11.      "tenantId": 12,
12.      "weight": 0.335,
13.      "publishStatus": true,
14.      "spuAttributeList": [{
15.          "id": 80,
16.          "attributeId": 7,
17.          "attributeName": "入网型号",
18.          "attributeValue": "A3092",
19.          "sort": null
20.      }...],
21.      "skuList": [{
22.          "skuId": 29,
23.          "spuId": 28,
24.          "name": "Apple/苹果 iPhone 15 (A3092) 蓝色 128GB",
25.          "description": null,
26.          "categoryId": 456,
27.          "tenantId": 12,
28.          "image": "http://47.108.55.112:9000/e-commerce-platform/sku/ea2bb479d20
      a401a856e6a8b83fb7d35.jpg",
29.          "title": "Apple/苹果 iPhone 15 (A3092) 128GB 蓝色 支持移动联通电信 5G 双卡
      双待手机",
30.          "subTitle": "【现货抢购！享白条 12 期免息！】",
31.          "price": 5199,
32.          "saleCount": 123,
33.          "skuAttributeList": [{
34.              "id": 54,
35.              "attributeId": 9,
36.              "attributeName": "颜色",
37.              "attributeValue": "蓝色",
38.              "sort": null
39.          }...]}]}]}

```

图 5.16 SPU 信息结果 JSON

Fig.5.16 Result for SPU Information JSON

通过对该 JSON 进行数据处理，渲染 SKU 图片，销售信息、商品基本信息和商品评价，最终得到页面效果图如下所示。

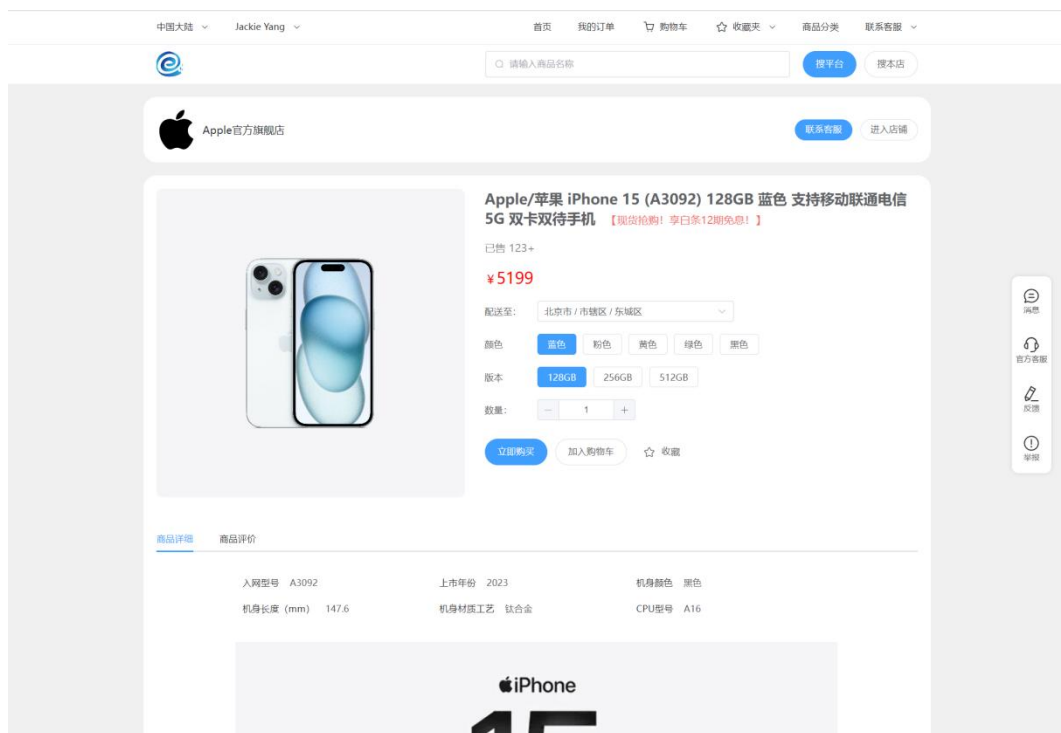


图 5.17 商品详情页（含详细信息）

Fig.5.17 Products Detail Page Including Detail Information

商品评价将不以 SKU 信息单独存储，而是采用 SPU 存储，也就是说，一件商品将会包括它的全部分类的商品评价。用户可以通过点击商品详情页的商品评价，查看用户评价，可以看到购买时间、评分、购买品种以及点赞和回复。

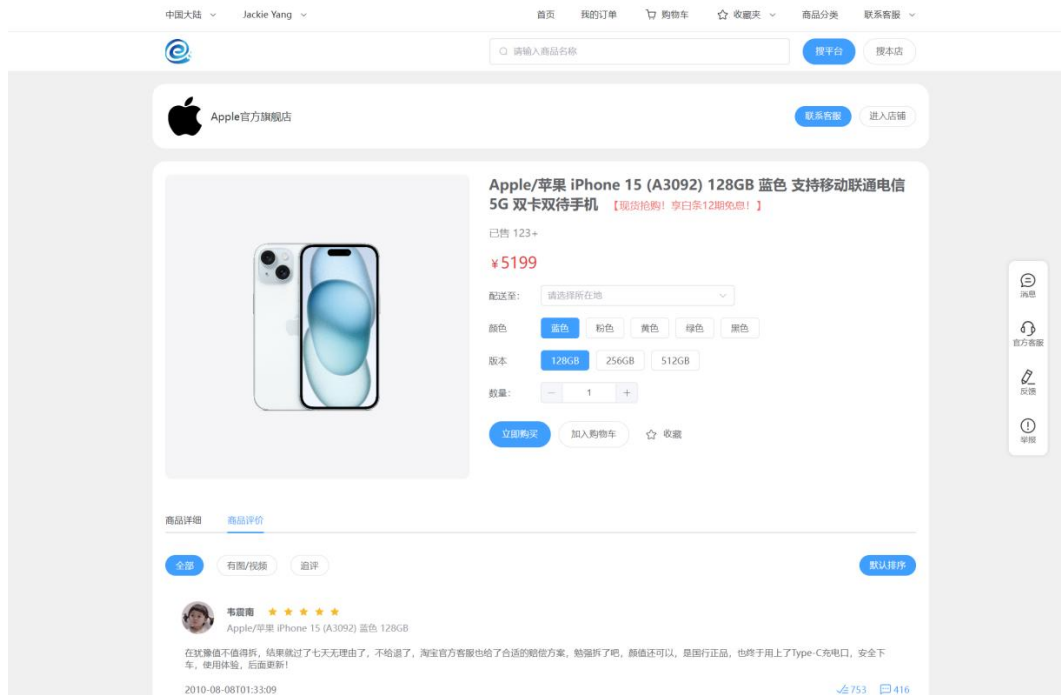


图 5.18 商品详情页（含商品评价）

Fig.5.18 Products Detail Page Including Comments

5.3.7 搜索商品

用户通过上方的搜索框进行搜索，发起搜索请求后，将通过 es 进行商品检索，es 将为商品提供远超 MySQL 的检索速度。从而快速选择出符合用户搜索信息的商品。此搜索详细页由搜索框、检索区域和商品信息组成，并结合分页插件实现页面信息定量存储。

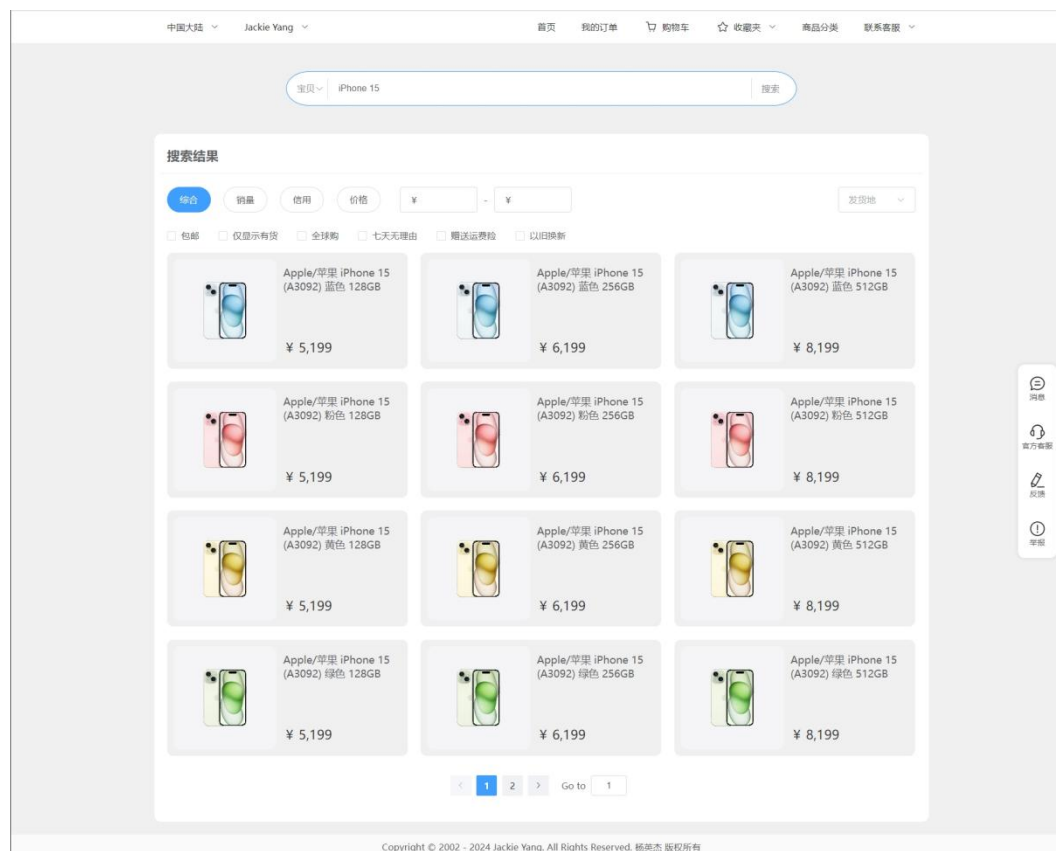


图 5.19 商品搜索结果

Fig.5.19 Result for Product Search

通过 spring-boot-starter-data-elasticsearch 包与 Elasticsearch 8 进行连接，通过传入 Keyword，针对商品和店铺进行搜索，商品搜索的核心实现如下。

```
1. @Override
2. public List<SkuSaveVO> searchProducts(String keyword) throws IOException {
3.     // 在 es 中搜索商品
4.     SearchResponse<SkuSaveVO> response = elasticsearchClient.search(s -> s
5.         .index("product")
6.         .query(q -> q
7.             .match(t -> t
8.                 .field("name")
9.                 .query(keyword)
10.            )
11.        )
12.    );
```

```

11.     ,
12.     SkuSaveV0.class
13. ;
14. log.info("search result: {}", response);
15. List<Hit<SkuSaveV0>> resultHits = response.hits().hits();
16. // 格式化返回结果
17. return resultHits.stream().map(Hit::source).toList();
18. }

```

图 5.20 商品搜索核心代码

Fig.5.20 Product Search Core Code

可在 Kibana 中观测 Elasticsearch 的索引和文档，并在 Kibana 中查看可用的检索词汇和检索效果，在浏览文档中直接输入 Keyword 就能完成搜索，此部分的搜索与通过接口 Keyword 进行搜索效果相同。

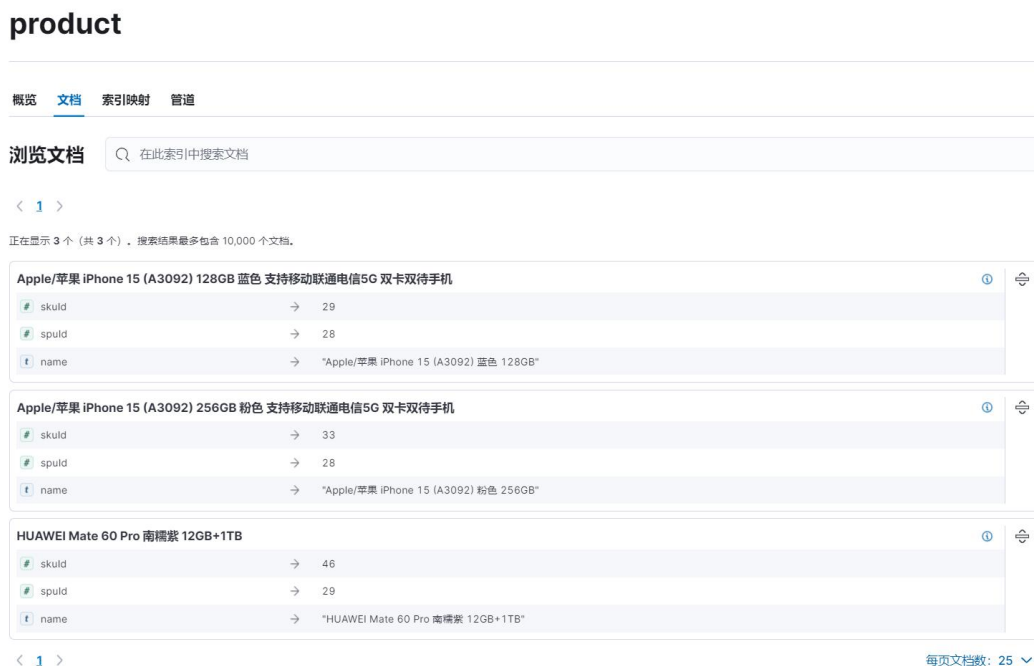


图 5.21 Elasticsearch 索引和文档观测

Fig.5.21 Elasticsearch Indexing and Document Observation

5.3.8 管理购物车

在商品详细页面中添加商品至购物车后，用户可以进入购物车界面以管理所选商品。在此环节，用户有权修改商品数量、将其加入收藏夹或从购物车中移除。同时，用户有机会挑选心仪的商品进行结算。系统将根据所选商品的实时数据，更新最终价格。

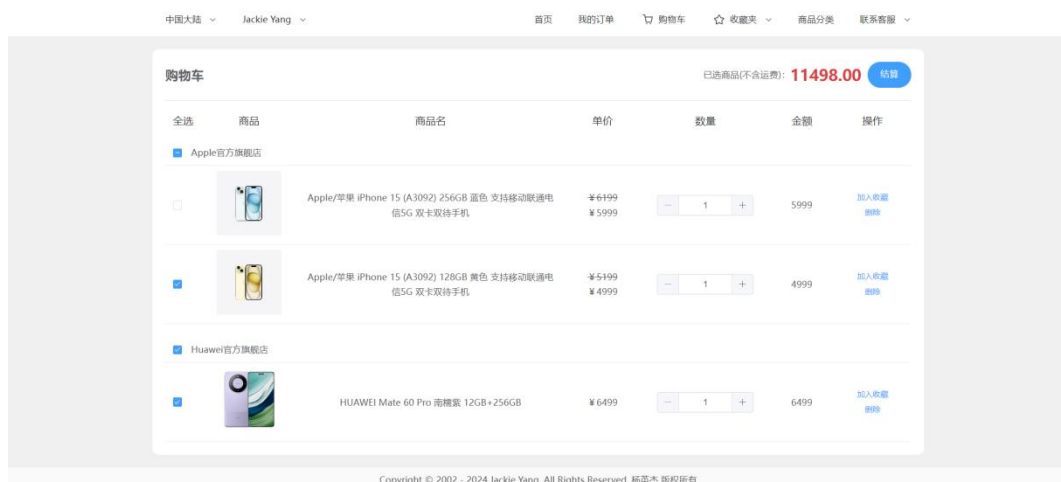


图 5.22 购物车管理

Fig.5.22 Cart Management

5.3.9 订购商品

用户在确认选择特定商品和相关销售选项后，可以通过购物车或商品详情页进行商品订购。系统将用户选择的商品加入待购买序列。

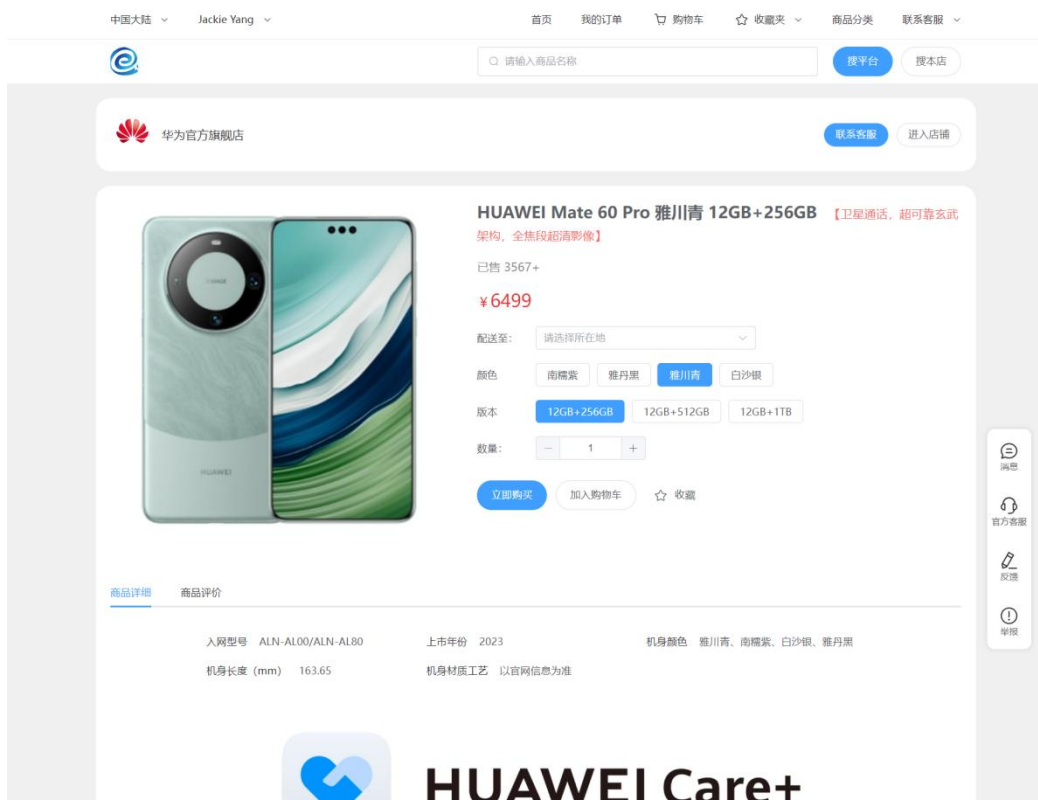


图 5.23 商品详情页（立刻购买）

Fig.5.23 Buy in Product Detail Page

用户在购物车中进行结算操作，选择自己的收货地址，确认商品参数和折扣后，点

击确认订单进入付款流程。

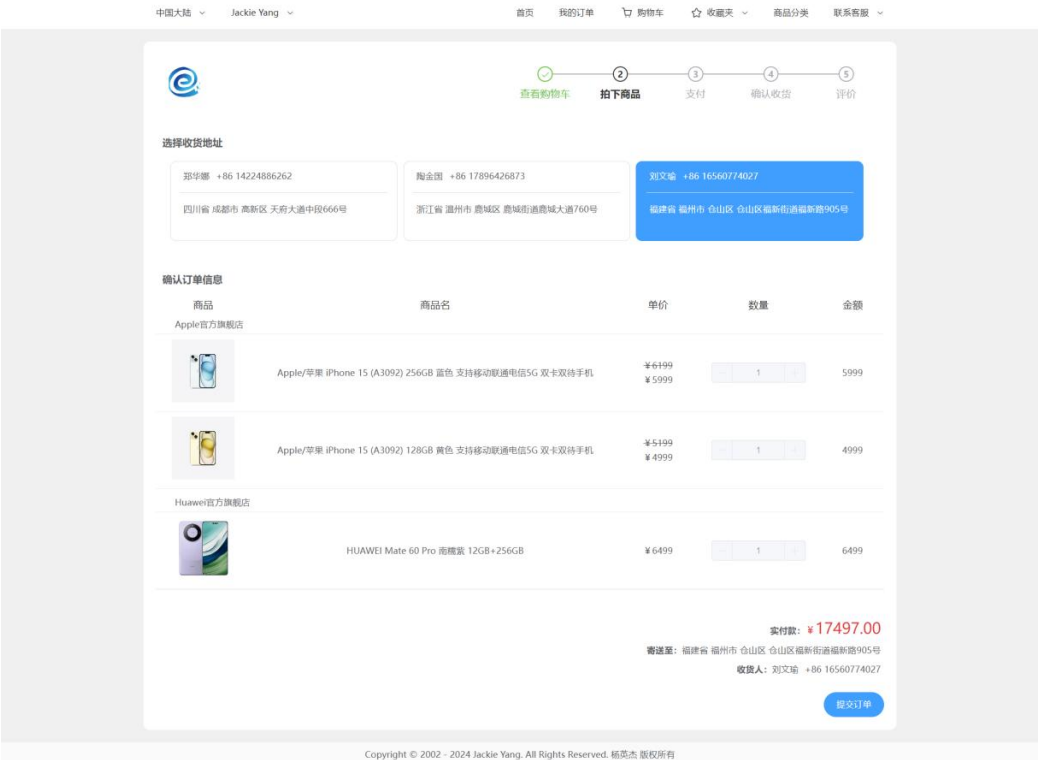


图 5.24 商品结算页面

Fig.5.24 Products Settlement page

用户在购物车界面执行结算操作，挑选自己的收货地址，核实商品参数和享受的折扣后，点击确认订单以进入支付环节，跳转支付宝沙盒进行模拟支付。



ICP证：合字B2-20190046

图 5.25 支付宝沙盒结算界面
Fig.5.25 Alipay Sandbox Settlement Page

一旦支付完成，第三方支付系统将回调信息返回，系统显示支付成功的提示信息，并将该笔交易订单记录到支付交易数据表中。

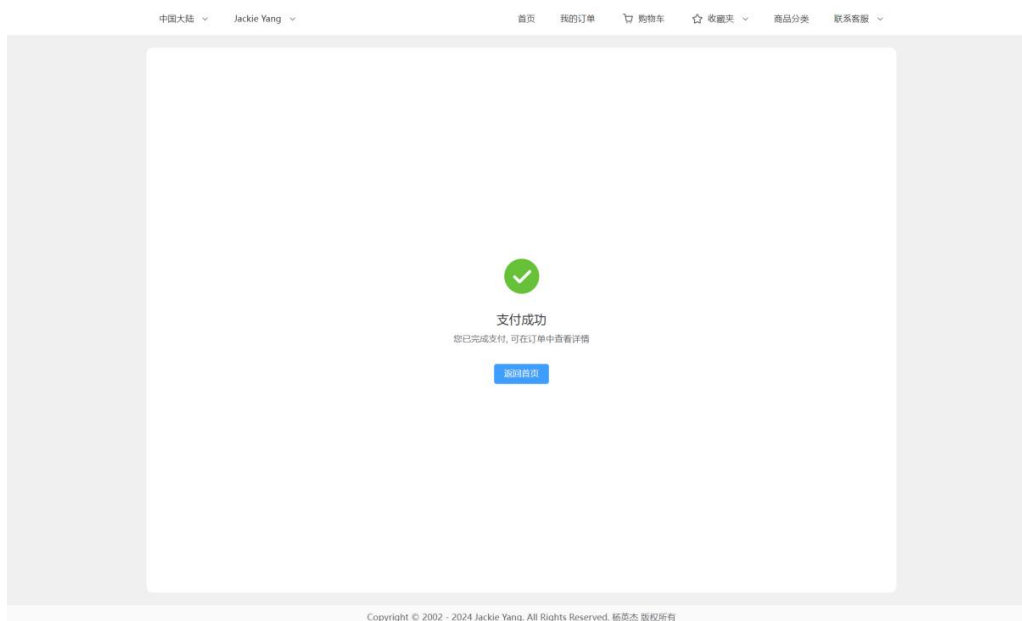


图 5.26 支付成功页面

Fig.5.26 Payment Success Page

若因未知原因导致支付失败，通过 Seata 将事务回退，将商品退回到用户购物车中，并弹出支付失败提示，提示用户返回购物车重新进行结算。

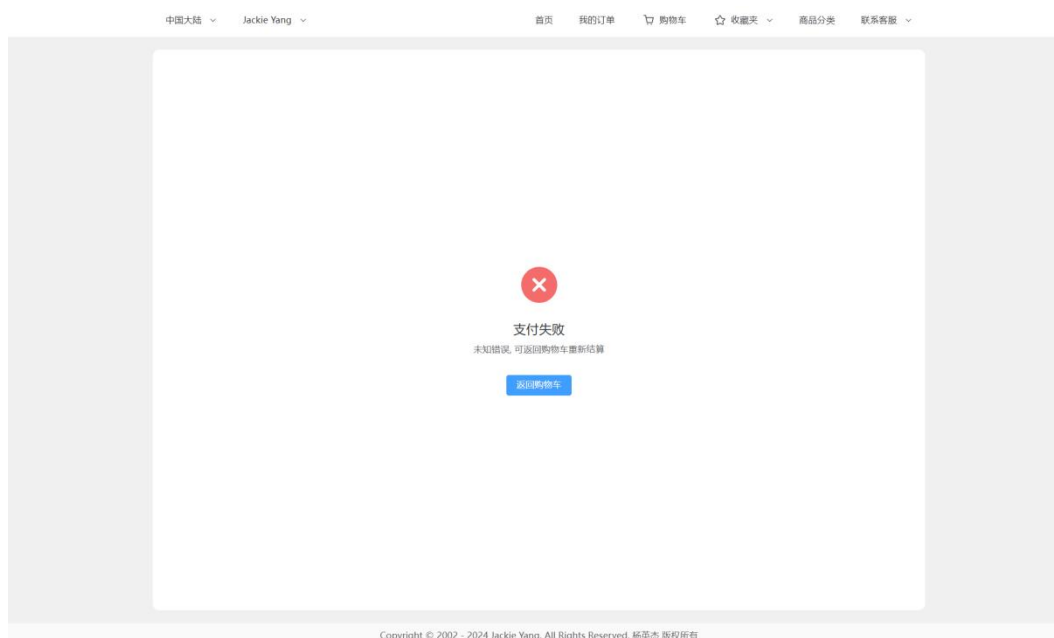


图 5.27 支付失败页面

Fig.5.27 Payment failed Page

用户购物完成后，可在我的订单栏目查看过往订单，并利用筛选框选择特定的订单状态。对于处于“待评价”状态的商品，可在交易操作区域进行商品评价。

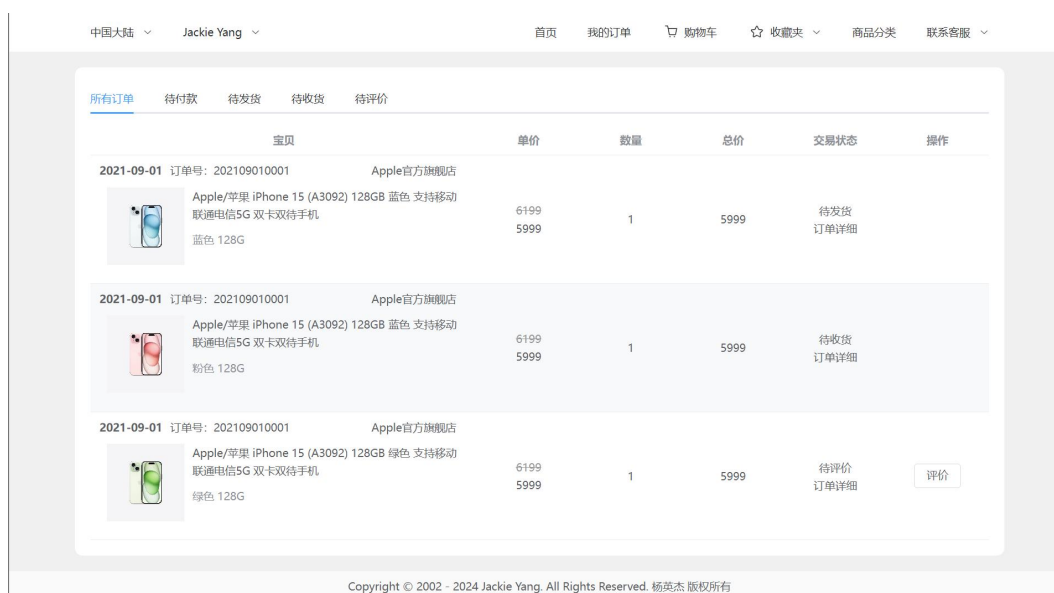


图 5.28 订单管理页面

Fig.5.28 Order Management Page

下订单的过程中涉及微服务调用和分布式事务，这里对核心业务逻辑进行说明，并附上核心实现代码。当请求下订单接口时，总共进行四个步骤。分别是调用仓储服务扣除商品库存，调用支付服务完成支付，生成新的订单和调用商品服务扣除购物车商品。通过 Seata 保证分布式事务的一致性。当任一微服务调用失败时，Seata 会启动事务回溯，保证事务回到最初的状态，保证库存、支付和订单的统一，其解决方案如下。

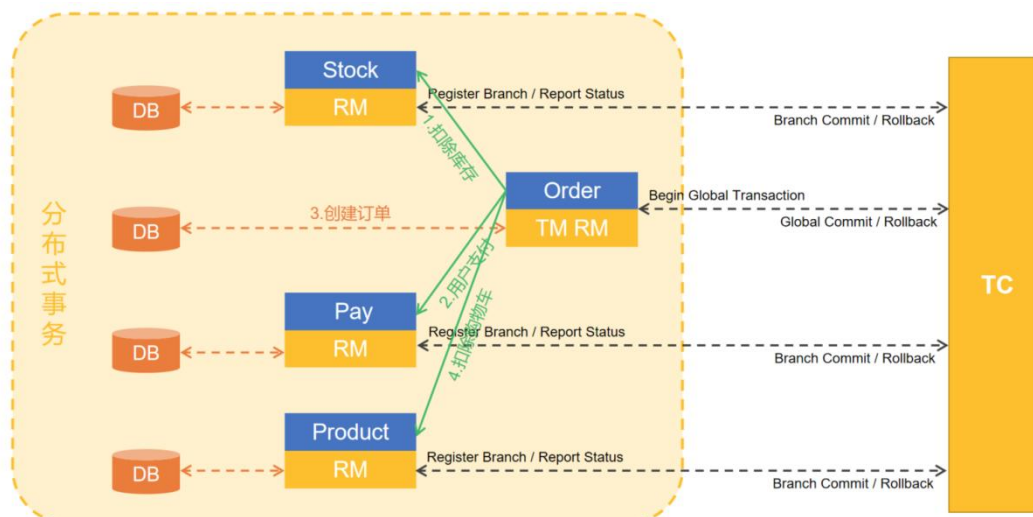


图 5.29 新增订单分布式事务解决方案

Fig.5.29 Add Order Distributed Transaction Solution

通过注解 `@GlobalTransactional` 无侵入的完成全局事务标注，忽略方法的详细调用，新增订单的核心代码实现部分如下。

1. `@Override`


```

2. @GlobalTransactional
3. public void addOrder(OrderDTO orderDTO) {
4.     // 扣除库存
5.     stockClient.decreaseStock(getSkuStockMap(orderDTO.getOrderItemList()));
6.     // 扣除金额
7.     payClient.decreaseWalletAmount(orderDTO.getPayAmount());
8.     // 生成订单
9.     makeOrder(orderDTO);
10.    // 删除购物车
11.    productClient.deleteCartProducts(orderDTO.getCartIdList());
12. }

```

图 5.30 新增订单核心代码
Fig.5.30 Generate Order Core Code

5.3.10 在线咨询

在线咨询结合阿里巴巴通义千问大模型，推出智能化客服，有效减轻了人工客服的需求和压力。用户可以通过首页右侧的官方客服按钮进入在线咨询聊天框。



图 5.31 官方客服按钮
Fig.5.31 Official Customer Service Button

在线咨询聊天框中，用户可以通过点击快捷询问的方式，快速对通义千问发起问题提问。也可以手动输入问题进行提问。此外，用户可以手动输入人工客服，从而唤醒人工服务。

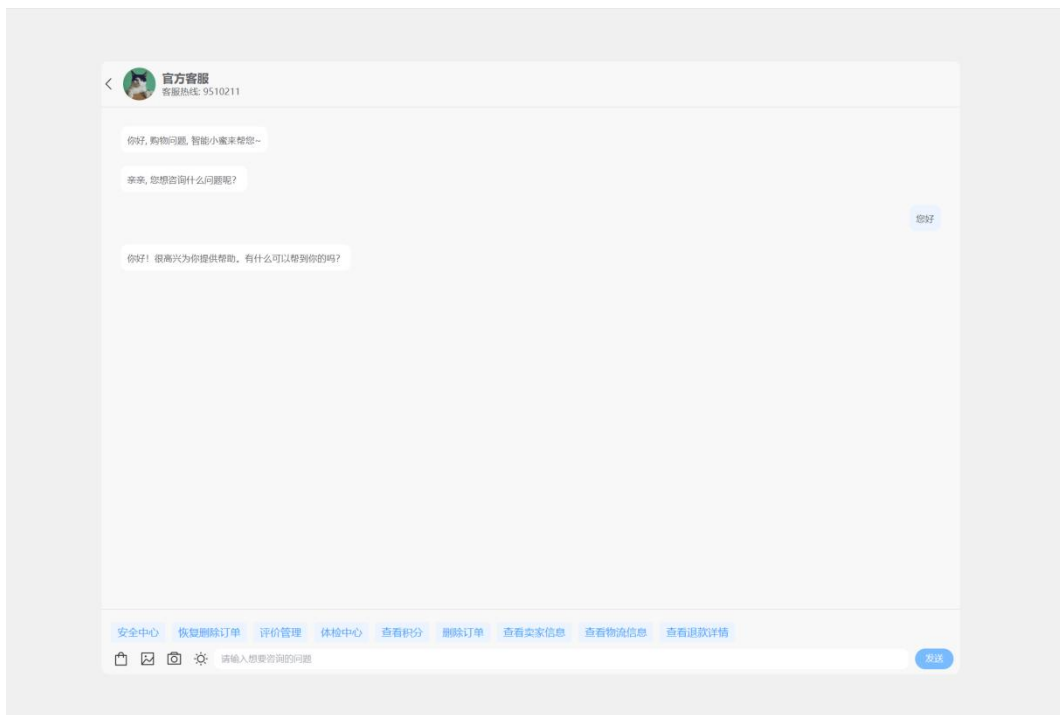


图 5.32 在线咨询界面
Fig.5.32 Online Consultation Page

5.3.11 个人中心管理

对于个人板块的管理主要放在个人中心管理处，因此集成了管理收货方式、管理支付方式、管理账户信息和授权第三方多种功能。用户可在个人中心进行支付方式的管理。

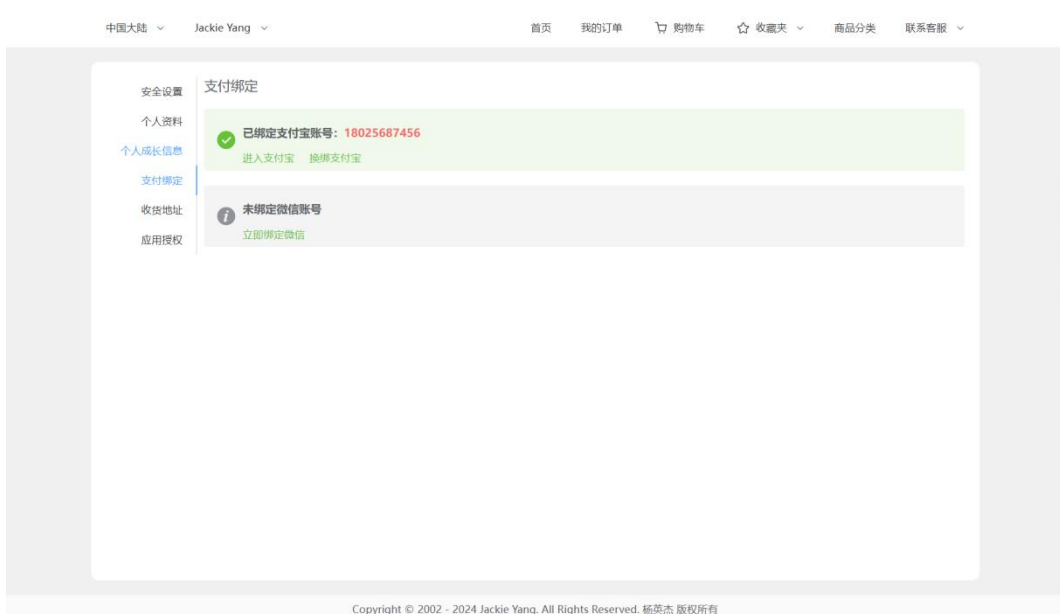


图 5.33 管理支付方式
Fig.5.33 Payment Management

用户可在个人中心进行地址管理。绑定地址后，在购买商品的时候即可快速选择个

人地址，而不需要多次手动输入地址。

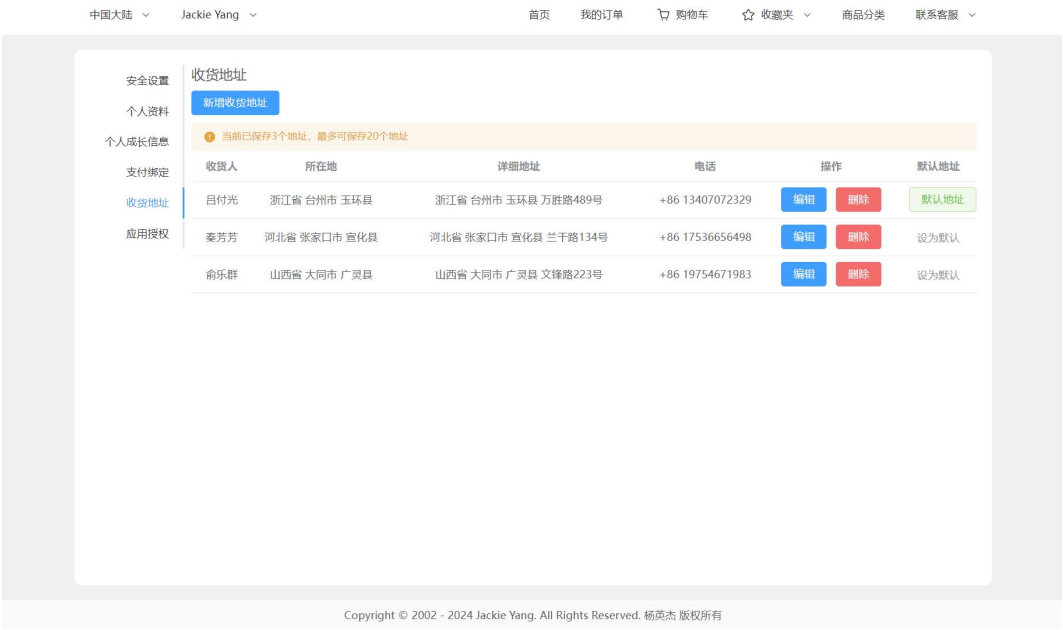


图 5.34 收货地址管理
Fig.5.34 Delivery Address Management

用户可在个人中心进行第三方应用授权管理。通过授权，可以通过第三方应用如菜鸟驿站等查看个人信息或者快递等信息。

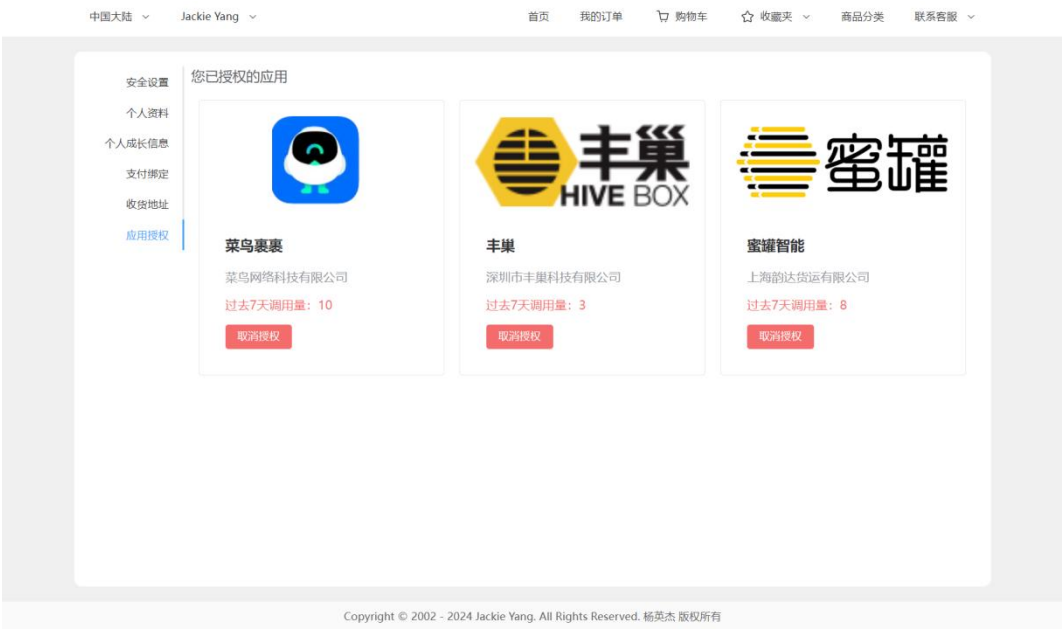


图 5.35 第三方授权管理
Fig.5.35 Third-party Authorization Management

用户可在个人中心进行个人资料管理，通过基本信息和头像栏目管理自己的个性资料，通过保存修改按钮进行个人数据保存操作。

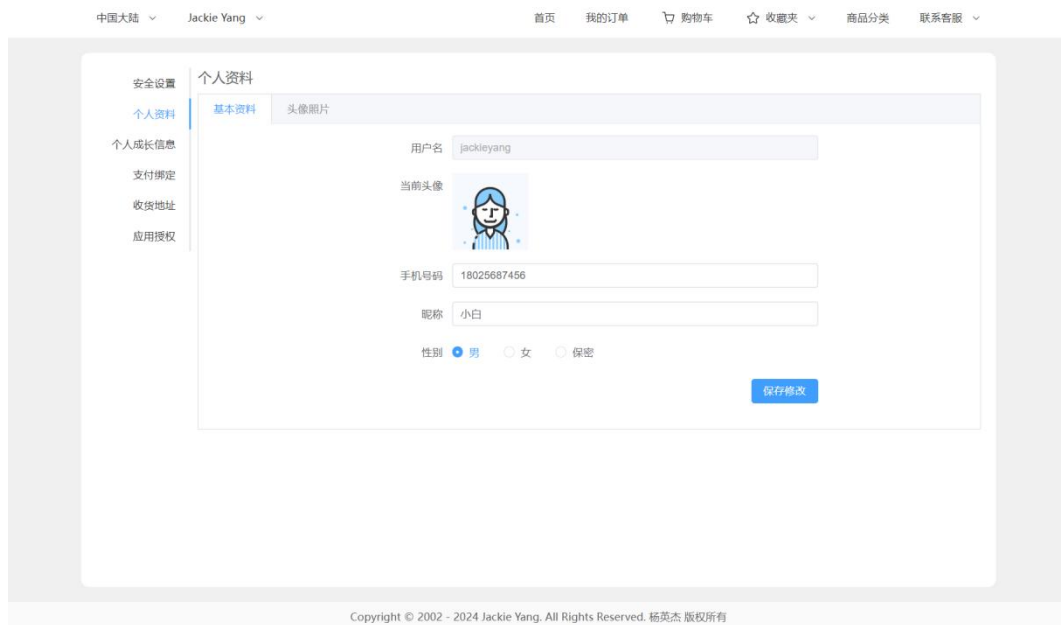


图 5.36 个人资料管理

Fig.5.36 Personal Information Management

用户可在个人中心进行账户信息管理，可自行设置账户基础属性、安全属性与隐私设置，开关个性化推荐等操作。



图 5.37 账户信息管理

Fig.5.37 Account Information Management

5.4 数据库操作实现

本论文所使用的数据库为 MySQL，使用 InnoDB 引擎，因此所有的文本字符集采用

utf8mb4 统一编码规范,排序规则统一采用 utf8mb4_general_ci。基于 keepalived + MySQL cluster 实现数据库集群,实现高并发、高可用和高性能。

在服务端的数据操作层,使用 MyBatis 作为轻量化数据操作框架,简化链接数据库的操作,通过在 Nacos 配置中心上传 MyBatis 的对应配置后,微服务将会在加载完 bootstrap 配置文件后,自动加载对应的 Nacos 中的 MyBatis 配置,同时由于 bootstrap 配置文件的存在,Nacos 中的云端配置可以轻松实现指定配置自定义。

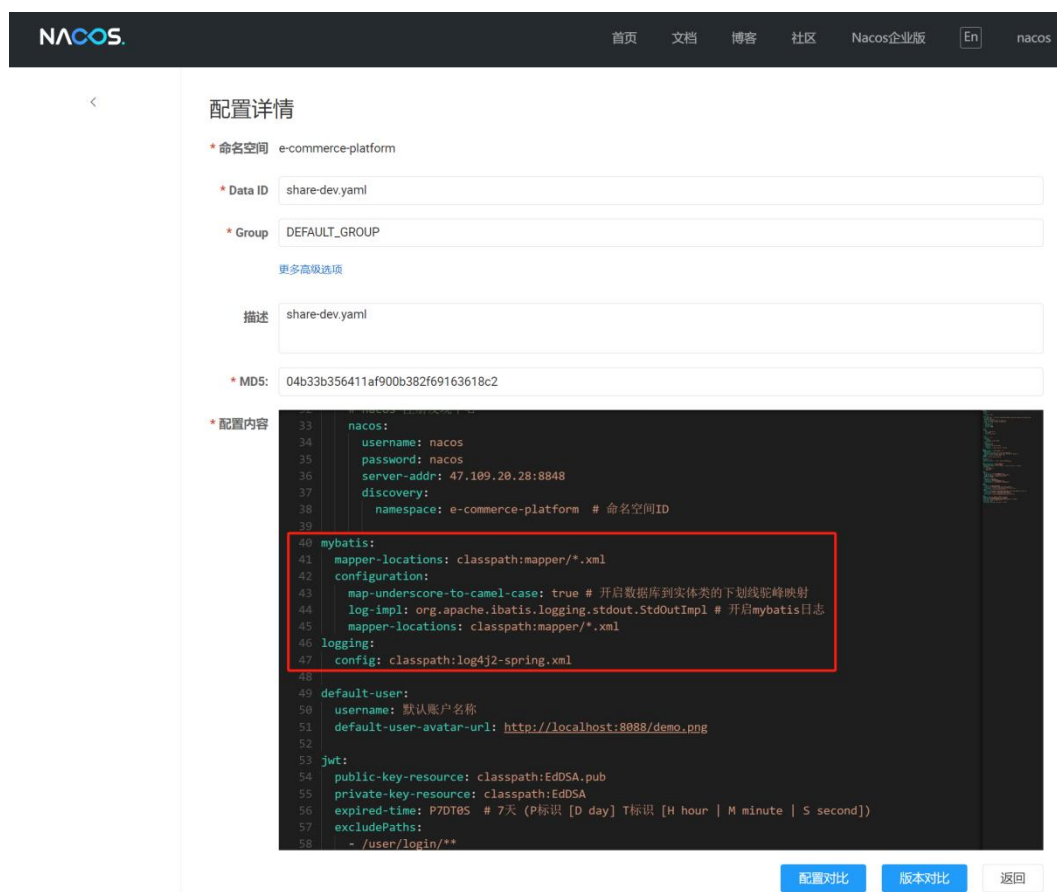


图 5.38 Nacos 端公共配置文件

Fig.5.38 Application.yaml Profile on Nacos

本系统通过 SQL 创建和 Navicat 可视化创建两种方式插入数据表,以下是商品模块中 SKU 信息的数据表创建 SQL 示例:

```
1. DROP TABLE IF EXISTS `product_spu_info`;  
2. CREATE TABLE `product_spu_info` (  
3.   `id` bigint NOT NULL AUTO_INCREMENT COMMENT '商品 id',  
4.   `name` varchar(200) COMMENT '商品名称',  
5.   `description` varchar(1000) COMMENT '商品描述',  
6.   `description_image` varchar(255) COMMENT '商品描述长图',  
7.   `default_image` varchar(255) COMMENT '默认展示图片',  
8.   `category_id` bigint COMMENT '所属分类 id',
```

```

9.  `tenant_id` bigint COMMENT '店铺 id',
10.  `weight` decimal(18, 4) COMMENT '商品重量',
11.  `publish_status` tinyint COMMENT '上架状态[0 - 下架, 1 - 上架]',
12.  `create_time` datetime COMMENT '商品创建时间',
13.  `update_time` datetime COMMENT '商品修改时间',
14.  PRIMARY KEY (`id`) USING BTREE
15.  ENGINE = InnoDB COMMENT = 'sku 信息' ROW_FORMAT = Dynamic;

```

图 5.39 SKU 信息创建 SQL 语句

Fig.5.39 SQL for SKU Information Creation

5.5 挑战问题解决

本论文在导论部分提出了六大工作和六大挑战。六大工作已在其他板块逐一实现，这里主要介绍六大挑战的解决方案。

1. 系统设计挑战解决：根据系统设计，将 19 大用例分为 8 个模块服务：用户服务、商品服务、购物车服务、订单服务、支付服务、搜索服务、仓储服务、优惠券服务。通过 OpenFeign 中间件完成他们之间的相互调用。设计统一返回类，保证接口响应数据的结构一致。

2. 数据管理挑战解决：通过 Seata 中间件保证不同服务的事务的一致性，当同一事务的不同服务未同时成功时，Seata 将通过 undo_log 将事务进行回溯，保证不同服务数据库的事务一致。

3. 网络延迟挑战解决：使用 HTTP/2 作为请求协议，以减少数据在传输过程中的时间消耗。使用 Spring Cloud LoadBalancer 分发流量，减少单点故障和延迟。将主服务器放置在同区域中，尽可能减少地域带来的网络波动。

4. 身份验证挑战解决：设置 Spring Cloud Gateway 服务，进行用户身份验证，所有的请求将首先通过该服务验证，通过后将用户信息加入到请求头中，携带进后续微服务请求中，由于集群属于内部集群，因此不再需要后续身份验证操作，后续服务在请求头中获取用户信息，即可知道用户信息。

5. 配置部署挑战解决：通过 Alibaba Codeup 平台进行自动化流水线的配置，设置流水线触发器，当代码被合入主分支时，自动触发流水线，进行新镜像的打包和部署，更新集群镜像，完成版本更替。

6. 可观测和监控解决：由于服务被部署到阿里云的 K8s 集群，直接通过阿里云控制台进行集群服务观测和监控即可。同时结合 Sentinel，实现流量的限流和熔断等操作。

5.6 本章小结

本章节主要描述了本系统开发过程中所使用的开发工具和开发环境，本系统的实现过程和呈现成果。通过划分功能的形式，将核心业务展示出来。同时，针对稍复杂的业务逻辑，给出了具体的实现方式。针对导论提出的六大挑战，给出实际解决方案。

第 6 章 系统部署

6.1 部署方式

本论文采用 Docker 容器技术进行服务的部署，通过 Alibaba Codeup 平台自动化 CI/CD，打包制品，并将制品上推到 Kubernetes 集群中去。通过设置触发器，当代码通过测试，合并到 master 发布主分支时，自动对最新的代码进行编译测试，打包成制品，并将制品推送至正式环境中。整体部署方案如下。

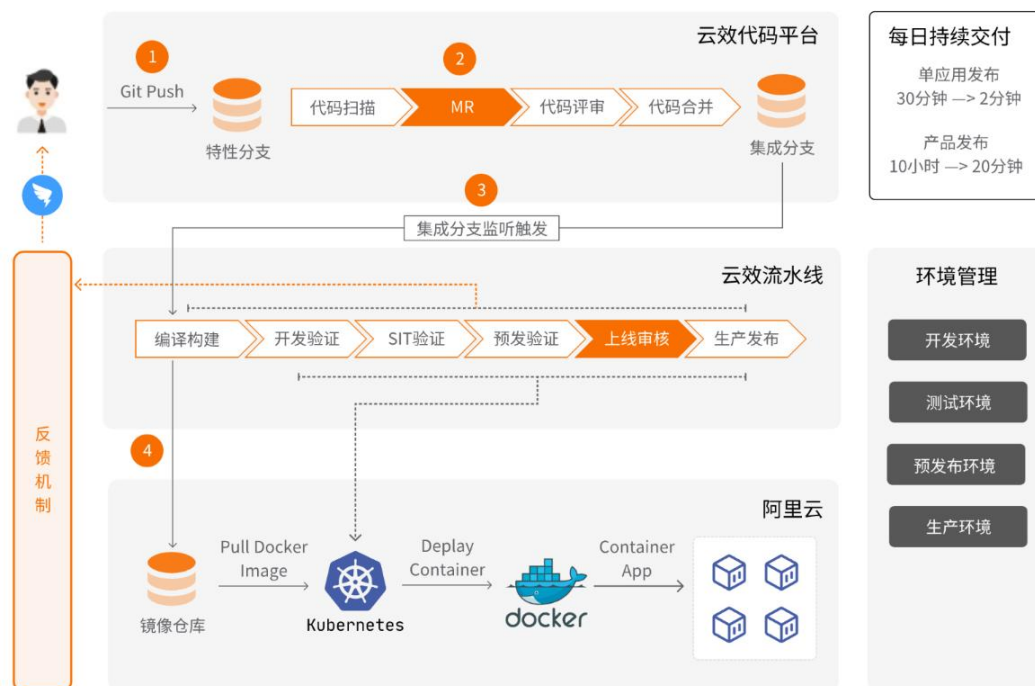


图 6.1 CI/CD 部署方案

Fig. 6.1 CI/CD Deployment Scheme

6.2 流水线配置

本论文所使用的流水线配置如下所示：



图 6.2 阿里云效流水线配置

Fig.6.2 Alibaba Flow Configuration

通过上述提到的设置触发器进行流水线自动编排。开启代码提交触发器后，阿里云效流水线自动添加 Webhook 到代码库源中，最终将镜像推送至 Alibaba Kubernetes 容器集群中去。最终流水线执行结果如下所示。

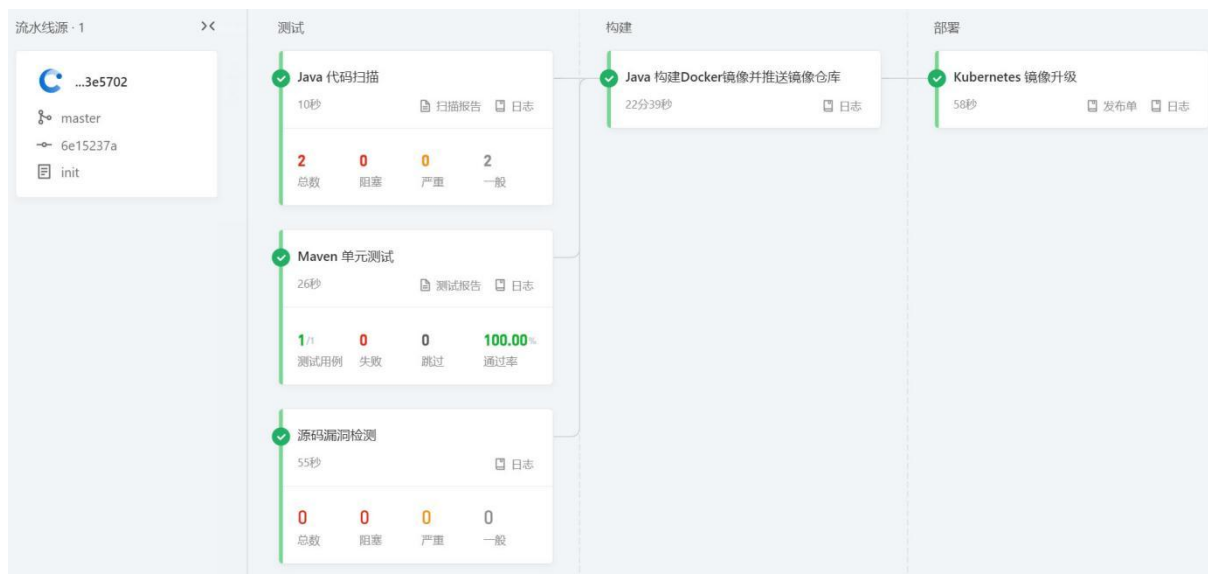


图 6.3 流水线最终执行结果
Fig.6.3 Result for Pipeline execution

6.3 本章小结

本章介绍了本论文所使用的系统部署方式和部署过程，没有使用主流的开源 Jenkins，而是通过 Alibaba Codeup 平台简化 CI/CD 操作，实现系统部署，最终完成了整个项目的部署工作。后续只需要将更新的代码合入主分支，就能够自动更新生产环境镜像。

第 7 章 系统测试

7.2 功能测试

本论文分别测试用户模块、商品模块、搜索模块、仓储模块、订单模块、优惠券模块、用户权限模块。支付模块由于个人无法接入生产环境微信和支付宝，因此暂时无法测试，只保留接入接口。所有的功能测试将以模块的形式分类列出。

表 7.1 测试功能总览

Table 7.1 Function Test Overview

模块分类	测试功能	获取	新增	修改	删除
用户模块	用户登录	密码、验证码、第三方			
	用户注册	密码、验证码、第三方			
	用户登出				
	用户信息	√		√	
商品模块	商品 SKU 列表	√			
	商品信息	√	√	√	√
	商品三级分类	√	√		√
	店铺信息	√	√	√	√
	商品评论信息	√	√	√	√
	商品属性	√	√		√
搜索模块	搜索商品				
	搜索店铺				
订单模块	订单信息	√	√	√	√
	订单发票信息	√	√	√	√
	订单退款信息	√	√	√	√
优惠券模块	优惠券信息	√	√	√	√
	优惠券分类信息	√	√	√	√
	商品满减信息	√	√	√	√
用户权限模块	角色信息	√	√	√	√
	权限信息	√	√		√
仓储模块	商品库存信息	√		√	
	订购信息	√	√	√	√

由于篇幅有限，这里仅以商品模块的三级分类功能为示例，针对其获取、增加和修改分别给出测试用例。以下先给出获取商品三级分类的测试用例：

表 7.2 获取三级分类测试用例

Table 7.2 Test Case for Getting Three-level Classification

主要元素	说明
------	----

1. 用例编号	1
2. 用例名称	获取三级分类
3. 测试目标	检查商品三级分类获取功能是否正常
4. 输入数据或操作	(1) 进入平台首页，获取商品一级分类 (2) 触碰一级分类，展示二级分类和下属三级分类
5. 系统的预期行为	(1) 系统展示商品一级分类 (2) 系统展示对应一级分类下的二三级分类
6. 执行情况记录	(1) 系统展示商品一级分类 (2) 系统展示对应一级分类下的二三级分类
7. 异常情况	无

通过上述测试用例清单，执行对应的测试步骤，得到了最终的测试执行情况，将具体情况记录到了上述表格中，并将测试结果图展示如下：



图 7.1 获取三级分类测试结果

Fig.7.1 Result for Getting Three-level Classification

针对添加三级分类测试，要考虑到输入的各种情况，设计了如下测试用例清单，添加三级分类的测试用例如下所示：

表 7.3 添加三级分类测试用例

Table 7.3 Test Case for Adding Three-level Classification

主要元素	说明
1. 用例编号	2
2. 用例名称	添加三级分类
3. 测试目标	检查商品三级分类添加功能是否正常
4. 输入数据或操作	(1) 输入合法的三级分类主节点。点击新增主节点按钮

	(2) 需要新增的地方点击添加，然后输入三级分类子节点 (3) 输入的三级分类名为 1 个字符 (4) 输入的三级分类名为 20 个字符 (5) 输入包含特殊字符的三级分类
5. 系统的预期行为	(1) 系统提示“添加成功” (2) 系统提示“添加成功” (3) 系统提示“三级分类名不能小于 2 个字符” (4) 系统提示“三级分类名不能大于 20 个字符” (5) 系统提示“三级分类名不能包含特殊字符”
6. 执行情况记录	(1) 系统提示“添加成功” (2) 系统提示“添加成功” (3) 系统提示“三级分类名不能小于 2 个字符” (4) 系统提示“三级分类名不能大于 20 个字符” (5) 系统提示“三级分类名不能包含特殊字符”
7. 异常情况	无

通过上述测试用例清单，执行对应的测试步骤，得到了最终的测试执行情况，将具体情况记录到了上述表格中，并将测试结果图展示如下：

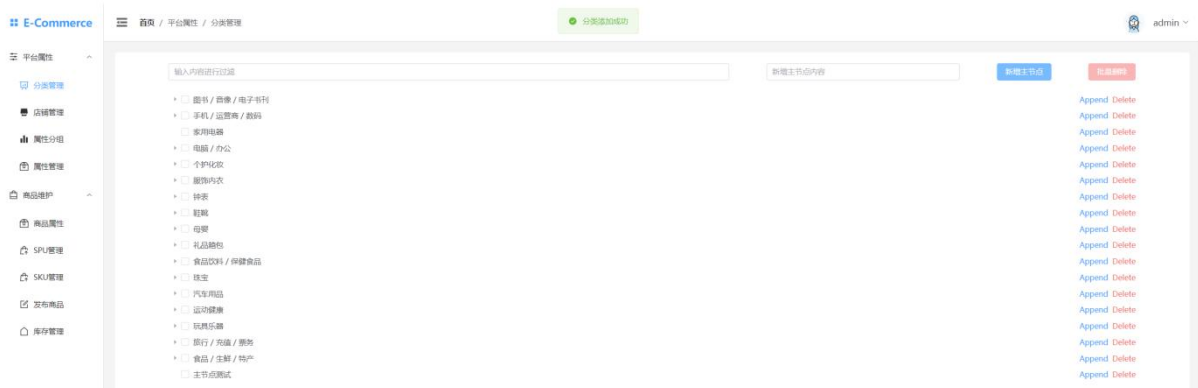


图 7.2 添加三级分类主节点测试结果

Fig.7.2 Result for Adding Three-level Classification Master Node

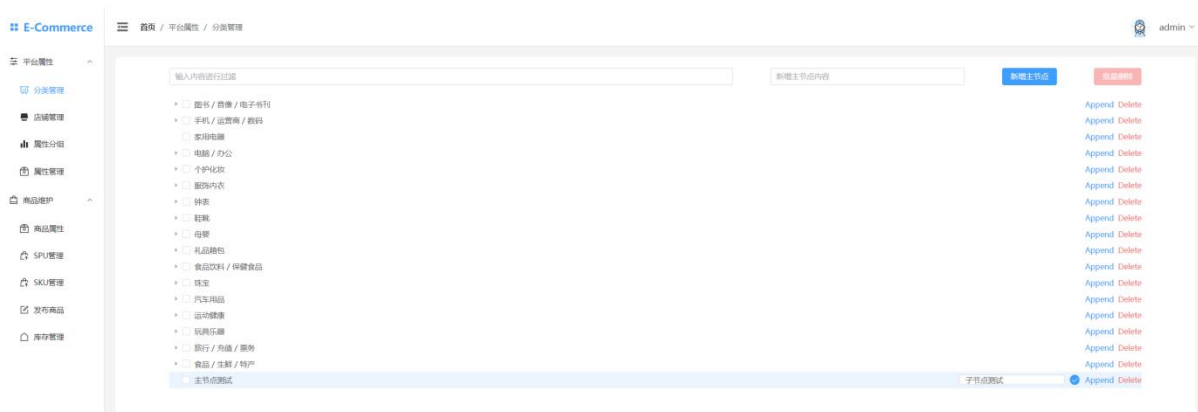


图 7.3 添加三级分类子节点输入时

Fig.7.3 Result for Adding Three-level Classification Child Node when input

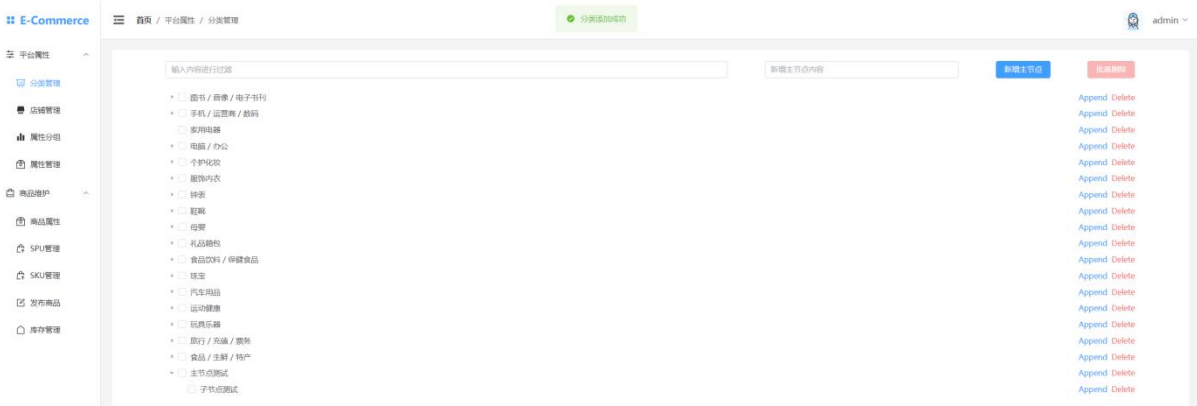


图 7.4 添加三级分类子节点测试结果

Fig.7.4 Result for Adding Three-level Classification Child Node

针对删除三级分类测试，要考虑到删除时候的误触可能，因此设计了如下测试用例清单，删除三级分类的测试用例如下所示：

表 7.4 删除三级分类测试用例

Table 7.4 Test Case for Deleting Three-level Classification

主要元素	说明
1. 用例编号	3
2. 用例名称	删除三级分类
3. 测试目标	检查商品三级分类删除功能是否正常
4. 输入数据或操作	(1) 点击删除按钮
5. 系统的预期行为	(1) 系统提示“删除成功”
6. 执行情况记录	(1) 系统提示“删除成功”
7. 异常情况	无

通过上述测试用例清单，执行对应的测试步骤，得到了最终的测试执行情况，将具体情况记录到了上述表格中，并将测试结果图展示如下：

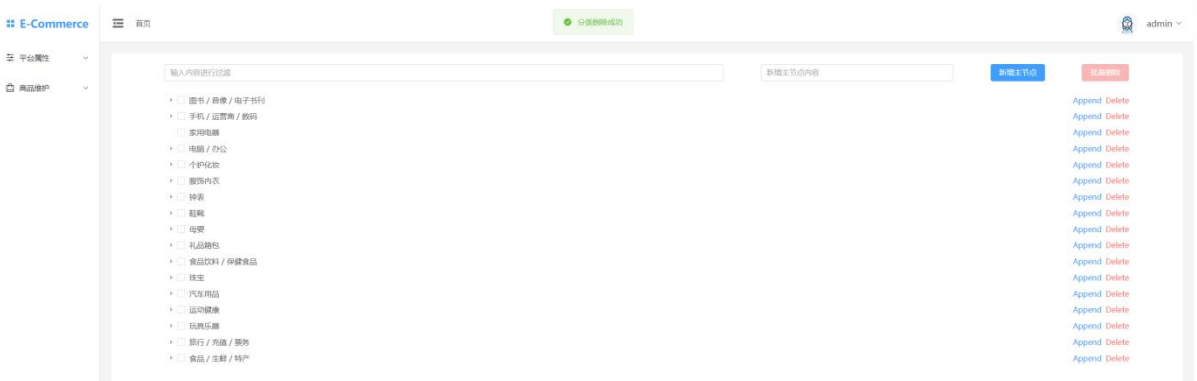


图 7.5 删除三级分类测试结果

Fig.7.5 Result for deleting Three-level Classification

通过 Apifox 继续实现该功能的自动化测试，使用自动化测试覆盖整体功能测试，便捷测试过程。

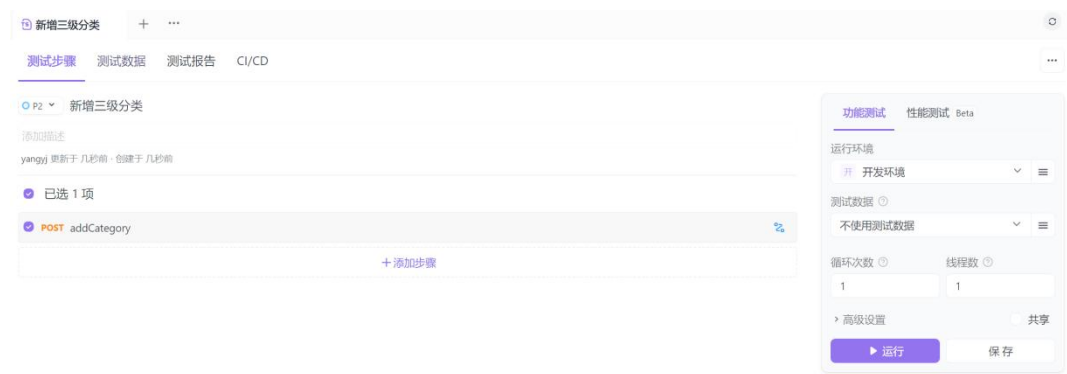


图 7.6 Apifox 自动化测试示意
Fig.7.6 Apifox Automated Test Schematic

7.3 非功能测试

7.3.1 性能测试

本论文将分别对数据库性能,Elasticsearch 性能和接口性能进行测试。使用 JMeter 对数据库进行性能测试，测试响应时间等^[18]。本次 MySQL 集群测试采用容器式 MySQL，配置为 4 核 8G，运行命令如下：

```
1. docker run -d -e MYSQL_ROOT_PASSWORD=123456 -p 3306:3306 --cpus=4 -m 8G --name mysql L-cluster MySQL
```

图 7.7 容器 MySQL 启动命令
Fig.7.7 Container MySQL Start Command

针对该容器 MySQL 基于单线程和多线程（10、50、100）进行查询和插入测试，最终测试结果如下表所示：

表 7.5 单线程查询测试
Table 7.5 Single Thread Query Test

编号	测试用例	响应时间（总数据量/万）		
		100	500	1000
1	在索引键上做查询 (select * from test where id=4000000)	0.026 s	0.022 s	0.023 s
2	整表分页查询 (select * from test limit 100 offset 4000000)	0.128 s	0.834 s	2.086 s
3	在非索引键上做查询 (select * from test where phone="xxx")	0.180 s	1.222 s	2.526 s
4	在非索引键上做模糊查询 (select * from test where phone like "12%")	0.181 s	1.166 s	2.575 s

表 7.6 多线程查询测试（10 线程）

Table 7.6 Multi-threaded Query Test (10 Threads)

编号	测试用例	响应时间（总数据量/万）		
		100	500	1000
1	在索引键上做查询 (select * from test where id=4000000)	0.021 s	0.015 s	0.019 s
2	整表分页查询 (select * from test limit 100 offset 4000000)	0.134 s	1.867 s	4.774 s
3	在非索引键上做查询 (select * from test where phone="xxx")	0.150 s	3.545 s	8.145 s
4	在非索引键上做模糊查询 (select * from test where phone like "12%")	0.167 s	3.610 s	7.858 s

表 7.7 多线程查询测试（50 线程）

Table 7.7 Multi-threaded Query Test (50 Threads)

编号	测试用例	响应时间（总数据量/万）		
		100	500	1000
1	在索引键上做查询 (select * from test where id=4000000)	0.017 s	0.010 s	0.010 s
2	整表分页查询 (select * from test limit 100 offset 4000000)	1.357 s	15.679 s	36.721 s
3	在非索引键上做查询 (select * from test where phone="xxx")	2.553 s	23.235 s	45.016 s
4	在非索引键上做模糊查询 (select * from test where phone like "12%")	2.413 s	22.861 s	50.289 s

表 7.8 多线程查询测试（100 线程）

Table 7.8 Multi-threaded Query Test (100 Threads)

编号	测试用例	响应时间（总数据量/万）		
		100	500	1000
1	在索引键上做查询 (select * from test where id=4000000)	0.011 s	0.007 s	0.008 s
2	整表分页查询 (select * from test limit 100 offset 4000000)	1.865 s	32.006 s	101.024 s
3	在非索引键上做查询 (select * from test where phone="xxx")	5.845 s	51.306 s	140.206 s
4	在非索引键上做模糊查询 (select * from test where phone like "12%")	5.607 s	50.131 s	131.160 s

表 7.9 单线程插入测试
Table 7.9 Single Thread Insert Test

编号	单次数据量(条)	平均响应时间（总插入数据量/万）		
		100	500	1000
1	10	0.149 s	0.152 s	0.161 s
2	50	0.574 s	0.521 s	0.542 s
3	100	1.076 s	1.103 s	1.112 s

表 7.10 多线程插入测试（10 线程）
Table 7.10 Multi-Threads Insert Test (10 Threads)

编号	单次数据量(条)	平均响应时间（总插入数据量/万）		
		100	500	1000
1	10	0.175 s	0.173 s	0.178 s
2	50	0.932 s	0.942 s	0.939 s
3	100	1.752 s	1.783 s	1.741 s

表 7.11 多线程插入测试（50 线程）
Table 7.11 Multi-Threads Insert Test (50 Threads)

编号	单次数据量(条)	平均响应时间（总插入数据量/万）		
		100	500	1000
1	10	0.217 s	0.210 s	0.235 s
2	50	0.954 s	0.993 s	0.981 s
3	100	1.901 s	1.857 s	1.932 s

表 7.12 多线程插入测试（100 线程）
Table 7.12 Multi-Threads Insert Test (100 Threads)

编号	单次数据量(条)	平均响应时间（总插入数据量/万）		
		100	500	1000
1	10	0.244 s	0.239 s	0.256 s
2	50	1.130 s	1.144 s	1.140 s
3	100	2.350 s	2.217 s	2.421 s

使用 Apifox 对 Elasticsearch 进行性能测试。以搜索商品为例，当并发用户量为 100，数据量为 100 万时，持续反复进行商品查询 1 分钟，测试结果如下所示。



图 7.8 Elasticsearch 查询性能测试结果

Fig.7.8 Elasticsearch Query Performance Test Result

对比数据库查询，Elasticsearch 平均查询时间为 8ms，数据库平均查询时间约为 1572ms，消耗约为数据库查询的 0.5%。

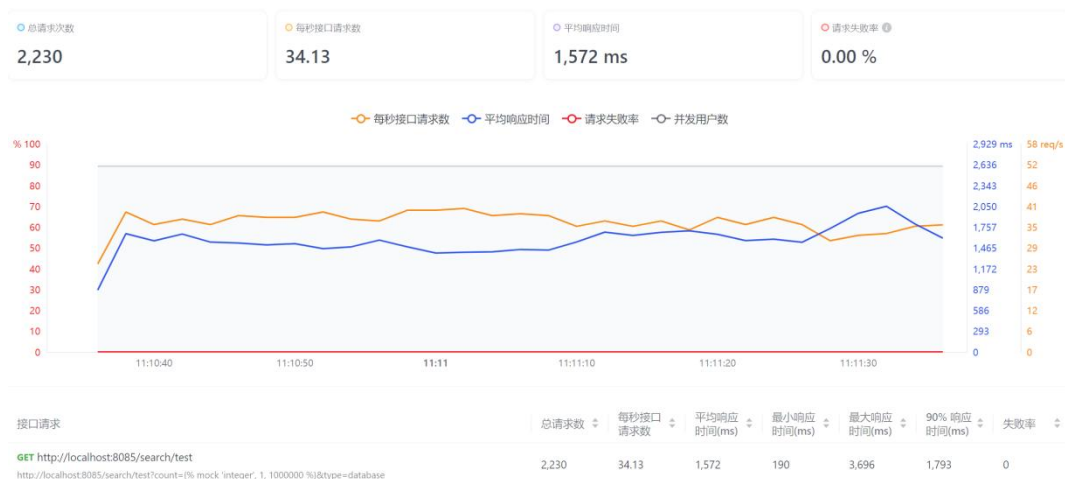


图 7.9 数据库查询性能测试结果

Fig.7.9 Database Query Performance Test Result

使用 Apifox 对系统功能接口进行性能测试，以登录账户作为示例，设置并发用户量为 100，持续反复进行登录 1 分钟，性能参数和测试结果如下所示。

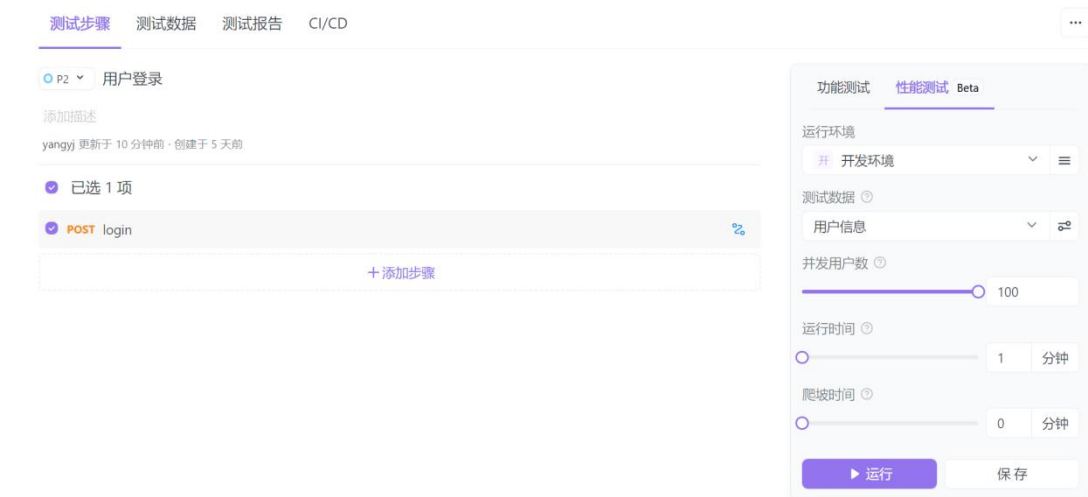


图 7.10 登录用户性能测试参数

Fig.7.10 User Login Performance Test Parameters

在 100 个并发用户的情况下，本系统表现出色，平均响应时间为 132ms，最大响应时间为 565ms，90%的响应时间都在 160ms 以下。



图 7.11 登录用户性能测试结果

Fig.7.11 User Login Performance Test Result

7.3.2 安全性测试

使用 Alibaba Codeup 代码安全性检测方案对代码进行检测，分别针对源码漏洞、依赖包漏洞、敏感信息、Java 代码安全和代码补丁进行检测，检测方案如下所示。



图 7.12 Alibaba Codeup 代码安全性检测方案

Fig.7.12 Code Security Detection Scheme from Alibaba Codeup

最终该安全性检测方案发现安全性问题 142 个，所有安全性问题都属于警告问题，最终检测结果如下：



图 7.13 Alibaba Codeup 代码安全性检测结果

Fig.7.13 Code Security Detection Result from Alibaba Codeup

7.3.3 容错性测试

本论文将针对下述三种情况进行容错性测试，分别是服务器断电、由网络中断或超售引起的事务未运行完成和请求未运行服务的接口。由于表现一致，都是由 Seata 监控事务完成与否进行业务回调，因此以请求未运行服务的接口为例作为说明。

在请求未运行服务的接口测试中，设置一个订单模块微服务不运行，其他模块正常运行，当请求下单商品时，Spring Cloud LoadBalancer 自动查询到该微服务状态，将请求转发到正常运行的订单模块微服务上。当设置全部订单模块微服务不运行的时候，

Seata 监控到此事务异常，自动进行回调。

7.3.4 兼容性测试

本论文仅对软件兼容性进行相关测试。每一个浏览器和版本支持的特性上都有细微的差别,在不同的操作系统上表现也有所不同。一个网站可能在某浏览器的某个版本上表现极佳，但是在另一种环境中就存在许多问题甚至无法显示。为了保证很好地为预定的客服服务，本文设计了一个测试矩阵表。

表 7.13 兼容性测试矩阵
Table 7.13 Compatibility Test Matrix

浏览器	Windows		Non-Windows		
	10	11	Ubuntu	Centos	OS X
IE 11	√	√	√	√	√
Firefox 3	√	√	√	√	
Safari					√
Chrome	√	√	√	√	√
Edge	√	√	√	√	√

通过 WSL（Window SubSystem Linux）完成 Ubuntu 和 Centos 的测试，由于 Centos 不属于 WSL 原生镜像，因此自行获取 Centos 镜像进行安装。针对 Windows 和 OS X，则使用 Windows 和 Mac 电脑进行测试。最终测试结果符合预期。

针对不同设备的分辨率问题也进行了兼容性测试。兼容手机，当媒体宽度在 1200px 以下的时候，收起分类列表，自动适应调整应用，页面属性根据平面宽度进行调节。

7.4 本章小结

本章对在线销售系统进行了较为详细且全面的测试，通过上述测试可以得出以下结论：在功能测试中，针对七大模块的测试都表现良好、符合预期；在性能测试中，数据库和接口性能都有较为良好的表现；在安全性测试中，代码通过 Alibaba 静态代码扫描并通过安全性编译；在容错性测试中，Seata 和 LoadBalancer 表现稳定，对系统有良好的维护能力。同时结合自动化测试，系统化、简便化功能测试流程和速度，加快效率，由于自动化测试的存在，在每开发过一个功能后，只需要自动运行测试即可过完全部功能测试。但同时也发现了以下不足：在兼容性测试中，发现该系统并未适配全部浏览器。在未来需要继续做出改进。

第 8 章 总结与展望

8.1 结论

本论文首先深入探讨了现代在线销售平台的现状，及其开发与运维所面临的巨大挑战。明确了微服务体系架构引入智能化时代超大规模在线销售平台的必要性的同时，决定融入业界最新大模型 Alibaba 通义前问以提供智慧化服务。

本平台构建过程基于模型软件工程方法，针对分布式事务、数据管理、网络延迟、身份验证、自动部署和服务监测六大挑战，围绕着需求规约、服务拆分、系统设计、系统实现、系统部署、系统测试六大任务，开展了研究和开发工作。最终完成了全部工作，并针对相关问题提出了解决方案。主要经历了以下阶段：

需求分析阶段，通过构建用例模型、结构模型和行为模型，完成对系统功能的全方面描绘。累计开发了 8 大模块共计 19 个用例以刻画系统的功能性需求，并在这些用例的基础上，继续开发了它们的概念类模型以刻画系统功能的数据结构，开发了它们的交互模型（用例序列图）以刻画用户与系统间的宏观交互行为。同时针对系统行为，为了使得系统交互更为形象，构建了基于霍尔三元组的系统操作契约。

系统设计阶段，结合了用例序列图和操作契约后置条件，构造了对象交互模型（即对象序列图），通过对象序列图展示了每个用例中的类与类之间的逻辑交互流程和数据交互。并基于需求分析阶段的概念类模型进一步完成了数据库设计，引导后续系统实现中的代码逻辑编写和数据库实现。

系统实现阶段，使用了 Spring Cloud Alibaba 作为微服务开发框架，应用了 Nacos 做配置下发和微服务注册与发现。结合 OpenFeign, Zipkin, Spring Cloud Gateway, Sentinel, Seata 等主流微服务组件分别实现微服务的远程调用、链路监控、网关路由控制、流量控制和分布式事务的管理。基于 Docker + Kubernetes 进行项目的部署，通过 Alibaba Codeup 进行 DevOps，部署流水线实现了自动化部署，完善了从代码合入主分支到自动化打包制品和上推镜像的一体式流程。使用了 Alibaba Codeup 进行代码管理，代码风格完全满足阿里巴巴风格的代码规范，共计完成代码超 2w 行，这里以后端服务为例，代码检测结果如下图所示，共计解决含魔法值等超 400 个相关问题。

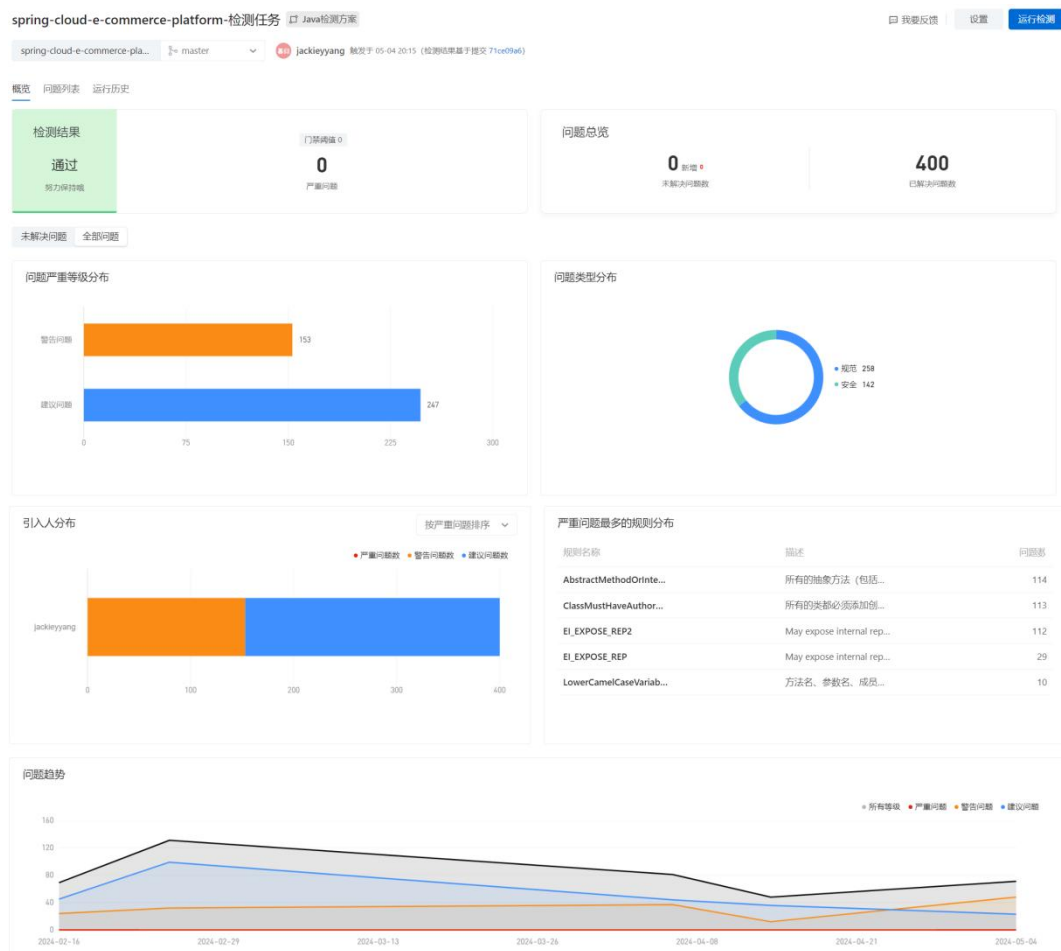


图 8.1 Alibaba Codeup 代码检测最终结果

Fig.8.1 Code Detection Final Result from Alibaba Codeup

系统测试阶段，针对该系统在功能和常见非功能上进行较为全面的测试，系统接口性能表现良好，数据库集群性能表现良好，证明了微服务架构在提高在线销售平台功能正确性、并发特性和高可用性方面的潜力。部分数据库测试表现欠佳，但通过合理的索引优化，访问效率能够得到显著提高。

总体而言，本课题成功设计并开发了一个融合 Spring Cloud Alibaba 微服务架构与 Alibaba 通义千问为一体的智慧化微服务体系架构在线销售平台，有望进一步推广到实际生产应用中。

8.2 展望

尽管本论文在微服务架构的在线销售平台的开发设计方面有一定成果，但还有着许多能够改进和深入分析的地方。

由于人员和时间因素，原计划完成销售平台 H5 网页端、销售平台小程序端和商家/平台管理后台，目前只完成了销售平台 H5 网页端和商家/平台管理后台，销售平台小程

序端还待开发。

在业务分析方面，并未考虑秒杀等场景，这也是在线销售的核心业务之一，并在实际环境中有着非常好的促销效果。后续能够在这方面进行进一步的分析和完善。

在技术实现方面，主要有以下三点展望：

1. 支付模块未接入。由于限制，本系统并未实现相关支付方式，只能接入支付宝沙盒，采用模拟余额的方式，进行用户扣款。后续希望能够进一步开发支付服务，集成支付接口，对接真实业务环境。

2. 部署不完善。由于服务器资源限制，对于自动化部署方面，只跑通了整体流程，实现代码合入主分支后的流水线触发与集群镜像更新，未将整体业务部署到集群中去。后续希望能够继续完善分服务部署流程，将整个销售平台业务完整部署到集群中去。

3. 测试可优化。由于在 WSL 中使用 Docker 进行测试，并关闭 Docker 的 ip-tables 来达到 window 可以直接访问 WSL 中的 Docker 的效果，因此网络延迟因素被排除在外。实际测试中应当考虑该因素的影响，使得测试结果更加真实可靠。

希望后续能够进一步深入和完善该项目。

参考文献:

- [1] 陈建海,陈淼,浦云明.基于微服务架构 B/S 系统的性能分析[J].计算机系统应用,2020,29(02):233-237.DOI:10.15888/j.cnki.csa.007285.
- [2] 龚森. 基于微服务架构的电商平台设计与实现[D].北京交通大学,2022.DOI:10.26944/d.cnki.gbfju.2021.000970.
- [3] 顾涵文.基于 UML 的知识分享系统设计与实现[J].科学技术创新,2022,(07):81-84.
- [4] 国家统计局. 中华人民共和国 2023 年国民经济和社会发展统计公报[EB/OL]. [2024-02-29]. 国家统计局, https://www.stats.gov.cn/sj/zxfb/202402/t20240228_1947915.html.
- [5] 胡根铭. 基于大数据技术的供应链物流管理平台的设计与实现[D].西南大学,2024.DOI:10.27684/d.cnki.gxndx.2023.001002.
- [6] 李春霞.微服务架构研究概述[J].软件导刊,2019,18(08):1-3+7.
- [7] 李强. 基于 CAS 和 OAuth 的统一认证授权系统设计[J]. 信息技术与网络安全, 2021, 40(06): 83-88.DOI:10.19358/j.issn.2096-5133.2021.06.014.
- [8] 李清波. 基于 PHP 和 AJAX 的高校科研管理系统设计与实现[D].北京邮电大学,2011.
- [9] 李志威, 王巍, 王恺. 基于 OAuth2.0 协议的门户系统设计与实现[J]. 中国数字医学, 2023, 18(12): 47-51.
- [10] 林龙健.基于 UML 用例图的企业网站系统需求建模[J].电子技术与软件工程,2016,No.91(17):64-66.
- [11] 林满山, 郭荷清. 单点登录技术的现状及发展[J]. 计算机应用, 2004, (S1): 248-250.
- [12] 刘梅. 基于 docker 的持续集成及发布平台的设计与实现[D].中国科学院大学(中国科学院沈阳计算技术研究所),2019.
- [13] 刘万里.多环境下的 CI/CD 自动化集成部署设计[J].现代计算机(专业版),2019(04):83-87.
- [14] 潘博.计算机软件数据库设计的重要性以及原则研究[J].计算机光盘软件与应用,2013,16(08):291-292.
- [15] 邵维忠,梅宏.统一建模语言 UML 述评[J].计算机研究与发展,1999(04):2-11.
- [16] 时子庆, 刘金兰, 谭晓华. 基于 OAuth2.0 的认证授权技术[J]. 计算机系统应用, 2012, 21(03): 260-264.
- [17] 孙克,鲁泽霖.人工智能在电子商务中的应用发展趋势研究[J].贵州社会科学,2019(09):136-143.DOI:10.13713/j.cnki.cssci.2019.09.020.
- [18] 陶幸辉,宋志刚.软件系统测试类型及测试用例设计[J].科技经济市场,2011(06):3-5.
- [19] 田杨锋,王振.基于 K8s 的 PaaS 架构及业界典型产品的调研分析[J].科学技术创新,2018(06):97-98.
- [20] 汪明洋.电商 B2B2C 模式下多店商城平台的构建[J].中国商论,2018(24):15-18.DOI:10.19699/j.cnki.issn2096-0298.2018.24.015.
- [21] 魏峻,王栩,李京.一种基于对象序列图的组件交互协议设计方法[J].软件学报,2001(07):996-1006.DOI:10.13328/j.cnki.jos.2001.07.007.
- [22] 席剑霄.OAuth2.0 协议及其应用模式研究[J].软件导刊,2015,14(07):23-26.
- [23] 辛园园,钮俊,谢志军等.微服务体系结构实现框架综述[J].计算机工程与应用,2018,54(19):10-17.
- [24] 颜炯. 基于 UML 的软件统计测试研究[D].国防科学技术大学,2006.
- [25] 杨涛. 基于 Node.js 的高并发电商购物系统设计与实现[D].北京邮电大学,2022.DOI:10.26969/d.cnki.gbydu.2021.002805.
- [26] 袁中臣,马宗民.基于语义的 UML 类图的集成分类[J].计算机工程与应用,2021,57(12):257-262.
- [27] 袁中臣,马宗民.基于语义和结构的 UML 类图的检索[J].东北大学学报(自然科学版),2020,41(01):23-28.

- [28] 郑七凡. 软件需求文档的 UML 用例图抽取研究[D].东北林业大学,2021.DOI:10.27009/d.cnki.gdbl u.2020.000948.
- [29] 中国互联网络信息中心. 中国互联网络发展状况统计报告[EB/OL]. [2023-08-28]. 中国互联网络信息中心, <https://www.cnnic.cn/n4/2023/0828/c88-10829.html>.
- [30] 朱光辉,王喜文.ChatGPT 的运行模式、关键技术及未来图景[J].新疆师范大学学报(哲学社会科学版),2023,44(04):113-122.DOI:10.14100/j.cnki.65-1039/g4.20230217.001.
- [31] 朱亚兴, 余爱民, 王夷. 基于 Redis+MySQL+MongoDB 存储架构应用[J]. 微型机与应用, 2014, 33(13): 3-5+9. DOI:10.19358/j.issn.1674-7720.2014.13.002.
- [32] Craig, L. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. London: Pearson.
- [33] Hatma Suryotrisongko, Dedy Puji Jayanto, Aris Tjahyanto. Design and Development of Backend Application for Public Complaint Systems Using Microservice Spring Boot[J]. Procedia Computer Science,2017.
- [34] Newman S.Building microservices:designing fine-grained systems[M]. [S.I.]: O'Reilly Media, Inc, 2015.
- [35] The Object Management Group® Standards Development Organization. Unified Modeling Language: 2.5.1[S]. [2017-12-05], <https://www.omg.org/spec/UML/2.5.1/About-UML>.

致 谢:

自此，论文的写作就告一段落，完成此论文的过程中，首先要感谢我的指导老师刘波老师，论文定题时，在专业文献上指导并提出建议，指导我使用基于模型的软件工程方法构建软件需求和设计。该方法为我打开了一道崭新的大门，将需求刻画到实现强链接到了一起。其次感谢我大学期间遇到的每一位课程老师和辅导员老师，他们的谆谆教诲令我受益匪浅。要感谢参与我论文评审和答辩的各位老师，对我论文不当之处提出的宝贵意见和建议，和答辩时的认真倾听。祝老师们身体健康、万事胜意、平安喜乐。

我还要感谢我的家人们，这么多年的求学之路上对我的支持、理解和陪伴，见证了我的成长和努力。站在他们的肩膀上，我见识到了更广阔的世界。也正因为有他们在物质和精神上的双重支持，我能够无忧无虑地完成我的学业，追逐我的梦想。感谢我的每一位朋友，在人生的道路上不同阶段的陪伴。很开心能看见他们都有了各自的出路，也将会在自己的领域闪闪发光。

人生味履，砥砺前行，一路走来有欣喜有失落，感谢那个哪怕道路荆棘，也依旧坚持不懈，努力向着目标前行的自己；感谢那个并不完美且总是尽全力做到最好的自己；感谢自己能够成为自己最好的朋友，黑夜残灯亦有孤影相伴。

未来我将心怀感恩之心，继续迈向人生新的阶段。愿我们都能不忘初心，继续书写新的篇章。